SOCIAL NETWORKS

NPTEL Online Course | Prof. Sudarshan Iyengar, IIT Ropar

Week 12 - Lecture Transcripts

Lectures 150-166 | 17 lectures | 141 pages (source transcript pages 1215-1354)

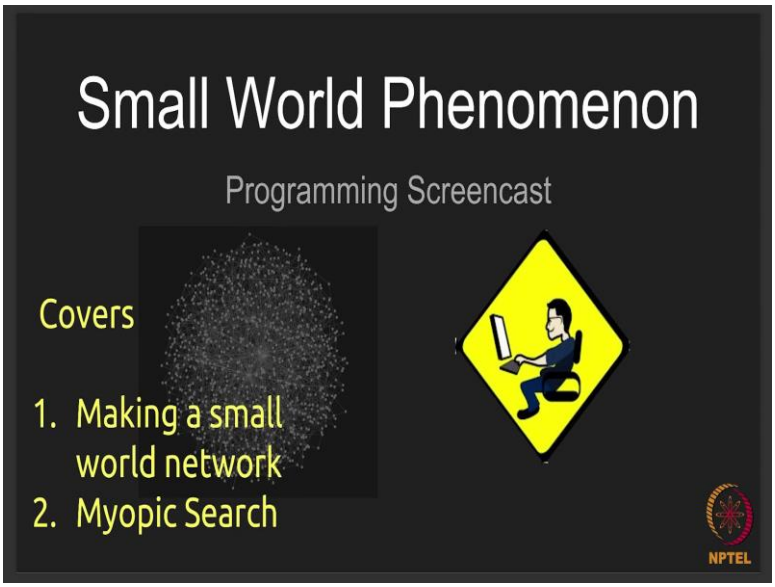
INDEX

LECTURE	TOPIC	PAGE IN THIS FILE
L150	Programming Illustration - Small World Networks: Introduction	2
L151	Base Code	11
L152	Making Homophily Based Edges	14
L153	Adding Weak Ties	27
L154	Plotting Change in Diameter	31
L155	Programming Illustration - Myopic Search: Introduction	34
L156	Myopic Search	40
L157	Myopic Search Comparison to Optimal Search	56
L158	Time Taken by Myopic Search	62
L159	PseudoCores: Introduction	74
L160	How to Be Viral	79
L161	Who Are the Right Key Nodes?	83
L162	Finding the Right Key Nodes (the Core)	85
L163	Coding K-Shell Decomposition	96
L164	Coding Cascading Model	108
L165	Coding the Importance of Core Nodes in Cascading	123
L166	Pseudo Core	136

Social Networks
Prof. S. R. S. Iyengar
Department of Computer Science
Indian Institute of Technology, Ropar

How to go Viral on Web
Lecture - 150
Programming illustration- Small world networks: Introduction

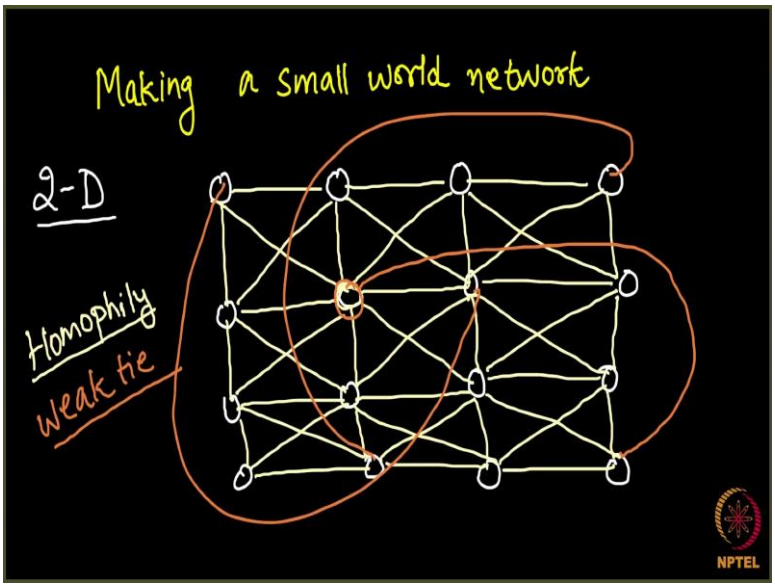
(Refer Slide Time: 00:05)



In the previous week, we have looked at the Small World Phenomenon. This week we are starting off with a programming screencast based on small world phenomenon and then we will take up another concept. The first starting off with a programming screencast of small world phenomenon, we are going to do two things there. First of all we are going to look at how do we make a small world network. So, I am sure all of you remember how we make a small world network, I will quickly recap it. And secondly, will do how do one does a myopic search on this network, how do one does a decentralised search on this network.

Now, before starting off with the programming screencast, I will quickly recap both of these concepts; how do we make a small world network and how do we do a myopic search on it ok.

(Refer Slide Time: 00:59)



Let us look at the first one. So, the first one was you want to locate how do we code, how do we make a small world network; so making a small world network making a small world network. So, here let us talk about the model which we have looked in the lecture. So, you will remember, how do we modulate in two dimensional in a two-dimensional space. We have certain nodes here. So, let us put some nodes here.

So, let us say I put some 16 nodes over here. So, I have some 16 nodes over here and then we have looked at, how do we make the connections between these two nodes. So, connections between these two nodes were based on two concepts. The first one was homophily. So, in homophily what we have done is, every node was connected to the nodes which are geographically closer to it. For example, this node here it can be connected to this node graphically closer this node, this node and this node.

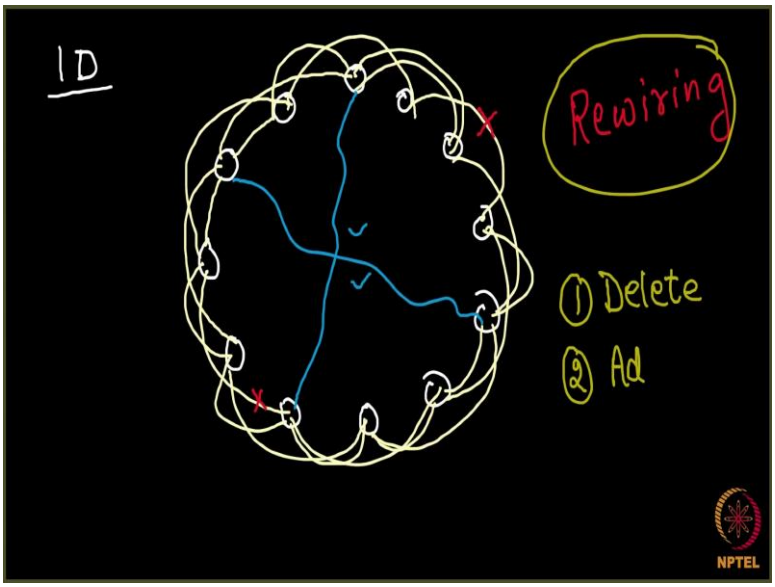
So, one on it is top bottom right and left. And similarly, for all of these nodes we can do the same thing and all of these links are made because of homophily. So, we get a grid something like this and what even we can do next is we can even connect these nodes to their diagonally opposite partners, because they are also geographically closer. So, we can make a grid like this as well. So, the grid which we had previously, this one is also correct and this one is also correct where we link the diagonally opposite nodes as well.

So, all these links are because of homophily and then we have look that what makes this world small is the presence of weak ties. So, the second reason the second concept which

gave rise to the small world network was weak ties where every node for example, this node will be randomly connected to some of the node which is quite far away from this. So, this node might be connected to somebody here, this node might be connected to somebody here, this node might be connected to somebody here.

So, these weak ties make our world small. So, this was how we can portray our world. So, gets small in two dimensions and rather then we looked at we can portray the small world network with the model with model which got.

(Refer Slide Time: 03:25)



So, gets give in one dimension which is rather more interesting. So, in one dimension, we can assume that the nodes in the network they are in the form of a ring. So, here we have some nodes in this network.

So, these are the nodes and again we have certain connections because of homophily. So, because of homophily the nodes are connected to other nodes which are geographically closer to them. So, we can take let us say 1 on the right-hand side and another on the left-hand side or rather we can take 2 on the right hand side and 2 on the left hand side. And again, this node here is connected to 2 on the right-hand side and 2 on the left hand side. And similarly, all the nodes connected to 2 on the right-hand side and 2 on the left hand side. So, let me make this connection quickly.

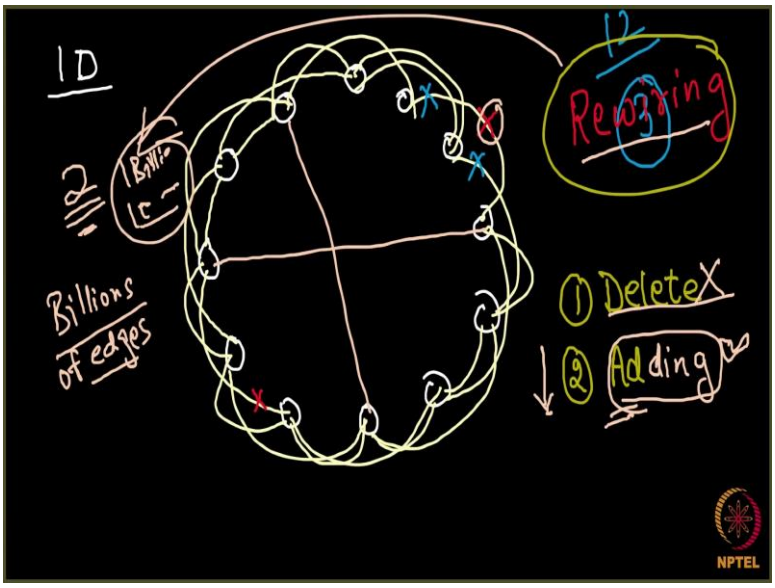
So, something like this. Every node is connected to 2 nodes on its left side and 2 nodes on its right side. So, let us say something like this and so on. So, these are the ties which exists because of homophily and again to make this world small, we should have somewhat long range ties weak ties. So, what we have looked previously in the chapter in the previous week was the concept of rewiring was the concept of rewiring. What we did in rewiring was we deleted of some of the edges from here. So, let us say this edge can be deleted and instead of this edge, we randomly put an edge somewhere else in the network.

Similarly, this edge can be deleted, and we can randomly put an edge at some other place in the network and so on. I make a for our coding for our programming screencast, I will make a small change here. And the change which I am doing here is instead of rewiring so, by the basic model which what so, gets has given it is actually rewiring where what you do it consist of two steps. So, the first is step is first of all you delete an edge from the network, you delete a random edge from the network and second you add a random edge in the network.

So, delete one edge and add one edge, but for our programming screencast I am doing a small change to this. What I am going to do is instead of this rewiring so, here instead of this rewiring, what am I going to do is, I do not rewire the edges here rather like in the previous model how did we see I am just simply going to add some long range contacts. And it will not make a as you will see; it will not make much change to the number of edges as you will see just on the addition of two edges, we will have did use the diameter of this network.

So so, let me elaborate it know what i want to say. What we are interested in is looking at the diameter of the network. Let us say we have this network somewhat like this and here can you tell me what the diameter of this network is. So, assume that how many nodes are there, 1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 11, 12 say 12 nodes are here.

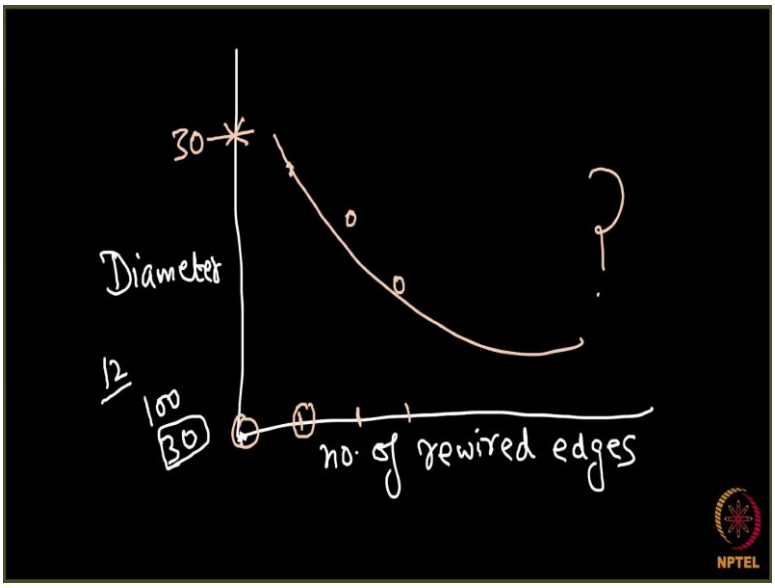
(Refer Slide Time: 06:55)



So, when there are 12 nodes, we can see that the diameter of this network is 6 or a node 6 actually it is going to be little bit less than 6. So, so, let us say from these are the two nodes which are at the maximum distance. So, the diameter will be 1, 2 and 3, right. So, the diameter of this network is 3. And what will happen when you are actually going to rewire some edges? So, let us say you rewire this edge and you put an edge here and then you delete this edge and you put an edge here, what it will do it will reduce your diameter.

So, now if you want to arrive from this node to let us say this node, the distance instead of 3 is just 2. So, you can jump from this node to here and you can jump from here. So, the diameter is reduced to 2. So, more the wiring you do, the diameter decreases accordingly.

(Refer Slide Time: 07:59)



What is our aim our aim is to look at if I plot a. So, if I plot a graph let us say, I plot a graph and on the x axis I have the number of rewired edges. So, I rewire 1 edge, I can rewire 2 edge and so on. Here are my number of rewired edges is and on the y axis I have the diameter.

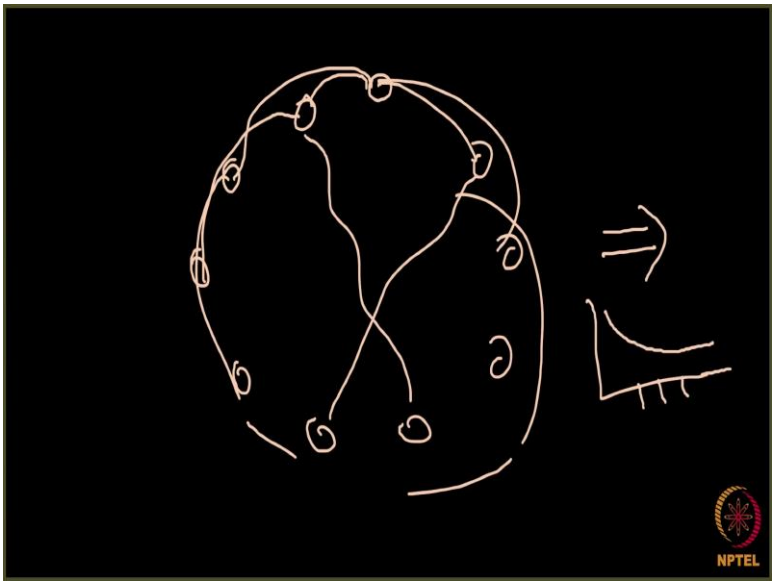
How will this plot look like? So, initially we know let us say instead of 12, let us say our network consisted of some 100 nodes and what will its diameter be? Its diameter let us say was something like 30. So, initially the diameter of my network was 30. So, here when I have 0 number of rewired edges; here when I have 0 number of rewired edges, the diameter of my network was 30. And then I rewire 1 edge and when I rewire 1 edge, what will happen? My diameter will reduce maybe somewhat here and then I rewire another edge and then again my diameter can reduce and again I rewire my diameter can reduce.

How will this curve look like? We want to study. So, as we do more and more rewiring of edges, how does the diameter of a network reduce is what we aim at is what we aim it studying. And while doing this as I discussed with make a small difference to a network over here and what is a difference is instead of deleting an edge here and adding an edge here we remove this step of deletion and we just add some rewired edges. And as you will see, the number of edges will not change much. So, you will see that on just the addition of let us say 2 or 3 edges.

On just the addition of 2 or 3 edges, the diameter will go down drastically and hence this whether you do rewiring or whether you do simply addition of an edge, these 2 are not these 2 are not very different concepts. The number of edges in the final network is actually going to be almost the same. So, let us say that you take a very big network on let us say some millions of nodes and billions of edges.

And on this network having billions of edges whether you rewire when you require 2 edges so, the number of edges will remain 1 billion only. According to the rewiring principle the number of edges will remain, 1 billion only, but if you add 2 edges, the number of edges will be 1 billion plus 2; 1 billion and 2 which are almost the same. So, here is a little change that we make to this network to this is framework of building your small world network.

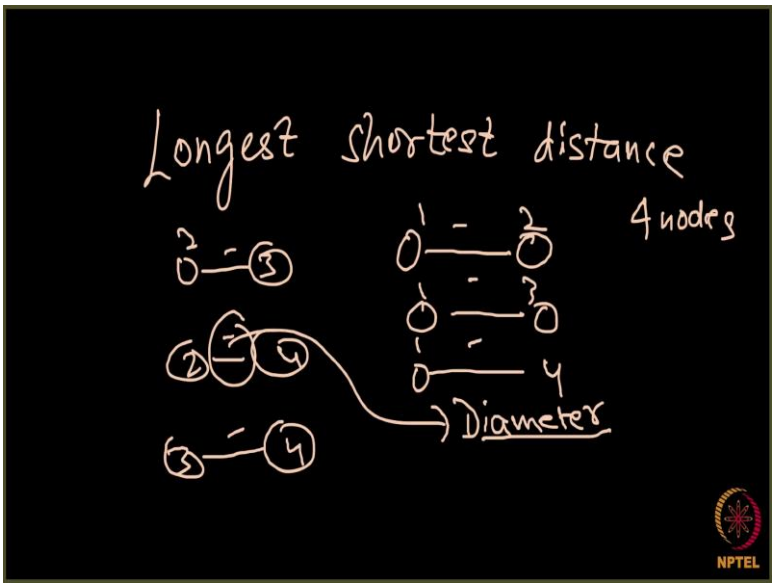
(Refer Slide Time: 10:37)



So, we have this ring network. And on this ring network, every node is connected to 1 node on the left side sorry 2 nodes on the left side and 2 nodes on the right side and so on. So, every node is following the same principle and at the end what we are going to do is we will add some extra edges there. And we will look at as we add these extra edges over here, how does the diameter of this network reduce.

And I hope will you remember; what is the diameter of a network.

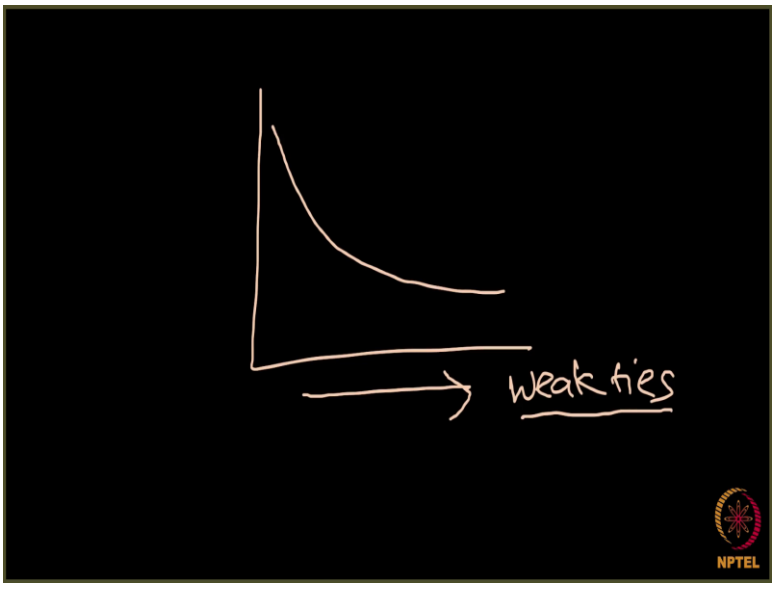
(Refer Slide Time: 11:09)



Diameter of a network is nothing but the longest, shortest, distance; longest shortest distance. So, you take every possible pair of nodes in the network, you find the distance between them distance between them. So, let us say there are some 4 nodes in this network. So, you take 1, 2, 1 3, 1 4 and then and then you can take 2 3, 2 4 and then you have 3 4. And you look at their distance right the shortest distance between these two nodes and the maximum over here is your diameter.

So, we have discussed it previously, I will not going to the details of this.

(Refer Slide Time: 11:57)



So, overall the first part of this programming screencast is going to deal with when we make a small world network as we increase the number of extra edges; as we increase the number of weak ties or the short range or the long range contacts how is this diameter going to reduce.

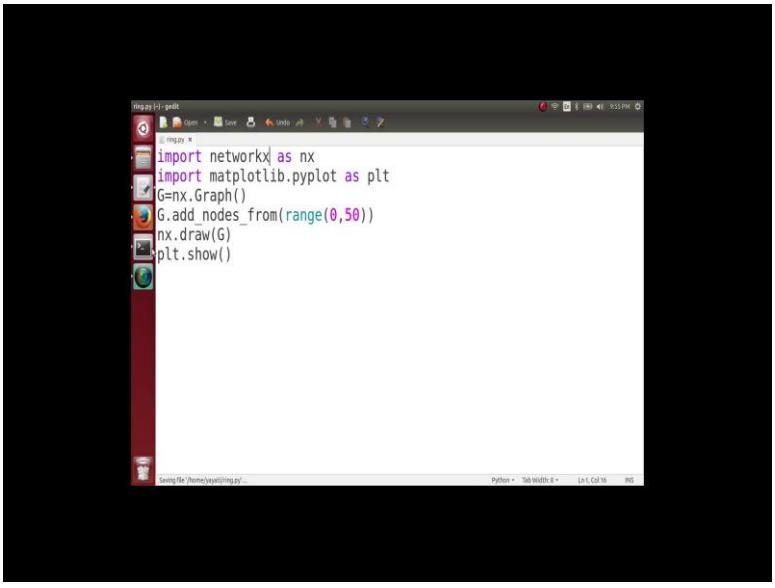
Social Networks
Prof. S. R. S. Iyengar
Department of Computer Science
Indian Institute of Technology, Ropar

How to go Viral on Web
Lecture - 151
Base Code

So, now we are staring of with the programming screen guard for creating a small world network. What we are going to do is as we discussed we are going to create a one dimensional small world network in the form of a ring. Here every node is connected to 2 nodes on the left side and 2 nodes on the right side and there are certain weak ties in between. And then we will look and how does the diameter of this network reduces as we add more and more numbers of weak ties more and more numbers of long range contacts.

So, first of all what do we need is we need a network having 50 nodes.

(Refer Slide Time: 00:43)

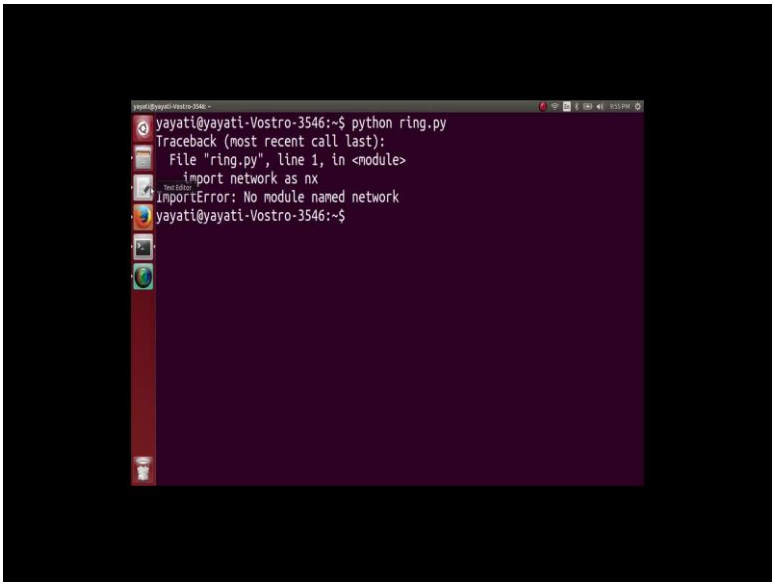


So, we know the code for that is very simple import network excess as nx after importing the module. So, will like to visualize this graph so, we import maplotlib.pyplot as plt and then G = nx.graph for creating a graph.

And then what we want to do is we want to add 50 nodes to this network. So, what do we do G.add_nodes_from and inside this we pass a list having numbers from 0 to 49? So, we have added the nodes, and then we can simply visualize this graph and nx.draw(G) and then plt. show.

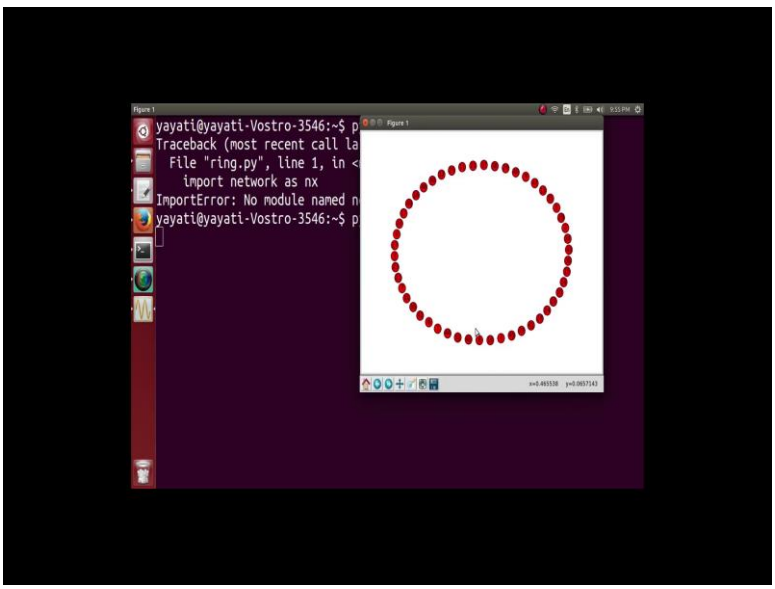
So, I am going to name this file as ring. py and let us execute and see.

(Refer Slide Time: 01:41)



So, python ring.py ok. So, import network x as nx.

(Refer Slide Time: 01:57)

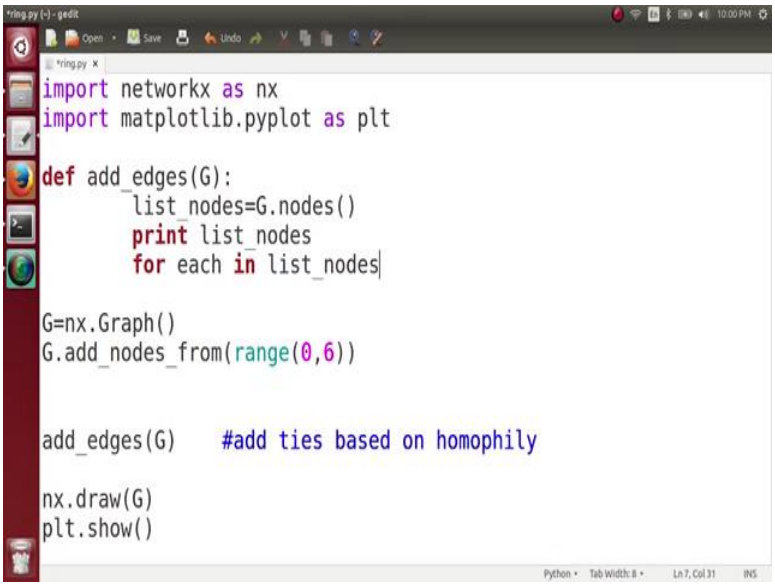


And then you can see that here, we have got a simple graph having 50 nodes label from 0 to I think that the maximum should be 49; labeled from 0 to 49 here. And you can see that the nodes here are noting a particular order. So, this is a graph which we are getting.

Social Networks
Prof. S. R. S. Iyengar
Department of Computer Science
Indian Institute of Technology, Ropar

How to go Viral on Web
Lecture - 152
Making homophily based edges

(Refer Slide Time: 00:07)

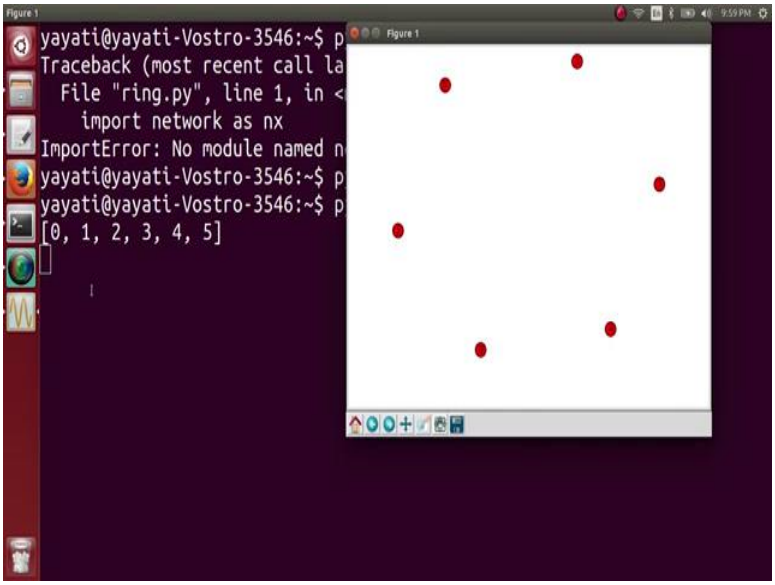


Next what we are going to do is, we want to add the edges corresponding to the homophily on this network and we have seen that according to the homophily, we are going to connect each node to 2 nodes towards its left and 2 nodes towards its right. And, we have also seen that when we were portraying this network, drawing this network the nodes were not in any particular order. So, for the time being to test the validity of the code I will simply change the number of nodes to 6.

So, we will be having 6 nodes number from 0 to 5 and next what I am going to do is I have to add homophily based edges to it. So, I call a function add_edges(G) here. So, what this function is going to do is add ties based on homophily to this network and how do I do that. So, that is simple. So, what I have to do? I have to connect node 0 to 2 nodes towards its right which are 1 and 2 and 2 nodes towards its left you can see that those 2 nodes are node minus 1 and minus 2, rather goes nodes are this node number 6 here and that node number 5. So, how do we do that?

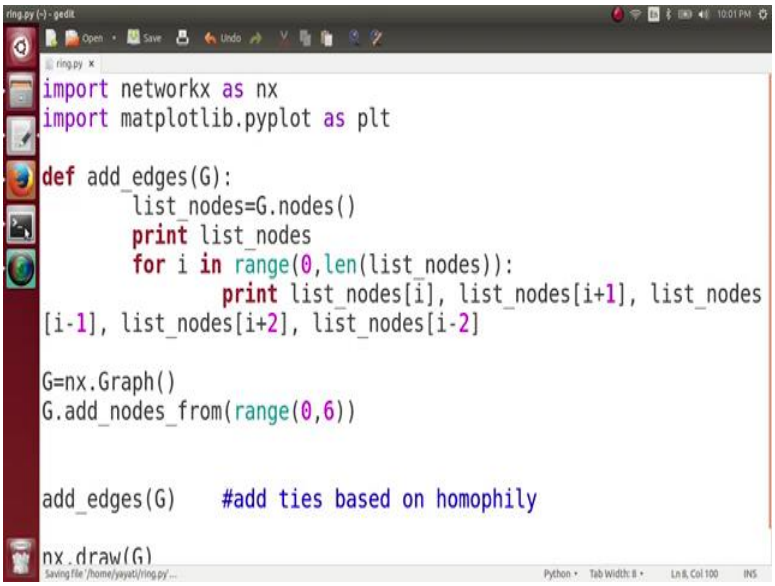
So, what we can simply do is we obtain a list of a. So, let me define the function here, define add edges and I will pass the graph G here and what I do is I obtain a list of nodes here; list of nodes equals to G.node which are simply the list_nodes equals to G.nodes which are simply the nodes in this graph G. And, we can actually print these list_nodes and see what it is giving to us.

(Refer Slide Time: 02:03)



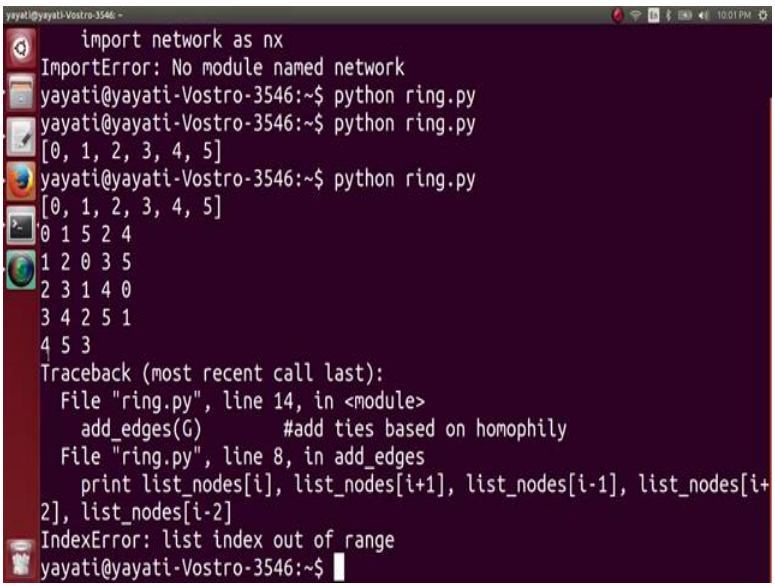
So, when I execute this code, we can see that we got here 5 nodes 0, 1, 2, 3, 4, 5 well and fine. Now, what we have to do is ok, we have to connect each node. So, we have to take every element in this list_nodes and every element is to be connected to 2 elements towards its left and 2 elements towards its right.

(Refer Slide Time: 02:39)



So, how do we do that is for each in list_nodes, what do I want is I will print each and rather now each in list.node for i a for i in range. So, we need to index in this is here for i in range 0 to length of list_nodes ok. We are going to print, this we are going to print list_nodes[i] right and where this list_nodes[i] should be connected to is list_nodes[i + 1], list_nodes[i – 1], list_nodes[i + 2] and list_nodesp[i – 2] right. So, this thing is very clear. So, we have seen previously whenever the in this is of a list goes in minus it starts back from 0. So, let us see whether it is working or not.

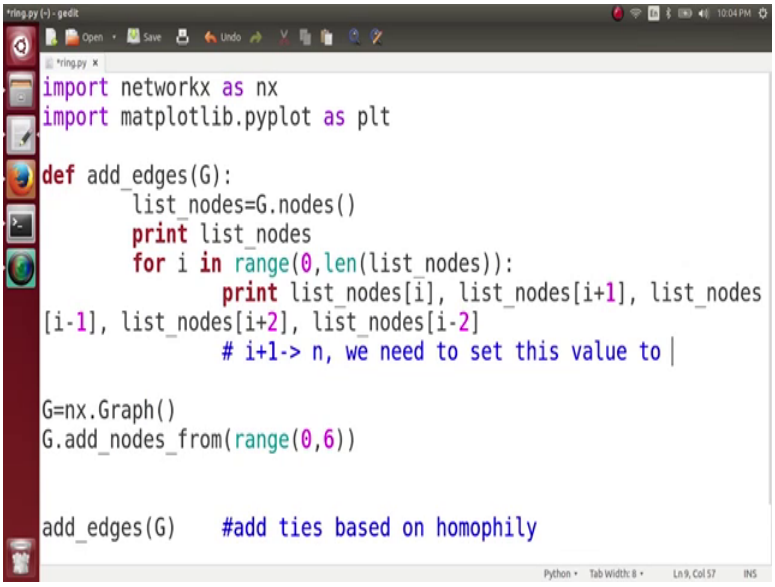
(Refer Slide Time: 03:47)



So, let us see what is it printing. So, you can see here 0 gets connected to 1 5 2 and 4, 1 gets connected to 2 0 3 and 5, 2 gets connected to 3 4 1 and 0, 3 gets connected to 4 5 2 and 1. Little bit problem with 4, it gets connected to 5 3 and then 4 gets connected to 5 and then 3 and then 5 3 and then 4 plus 2 is 6 ok. So, you so, you note the problem here. So, here when the index of a list whenever the index of a list goes in minus there is no problem, it starts up from the last element, but there is no element named as list_nodes[i + 2].

So, what we can do here is little bit of manipulation is required here. So, list_nodes[i] we can use and there can be a little bit problem with this i + 1 and i + 2, i - 1 and i - 2 are perfectly fine. So, with i + 1 and + 2 there is a little bit of problem. So, we will just resolve that ok. How do we resolve that? When will i + 1 first problem, you can say that i + 1 will create problem when you go to this element.

(Refer Slide Time: 05:31)

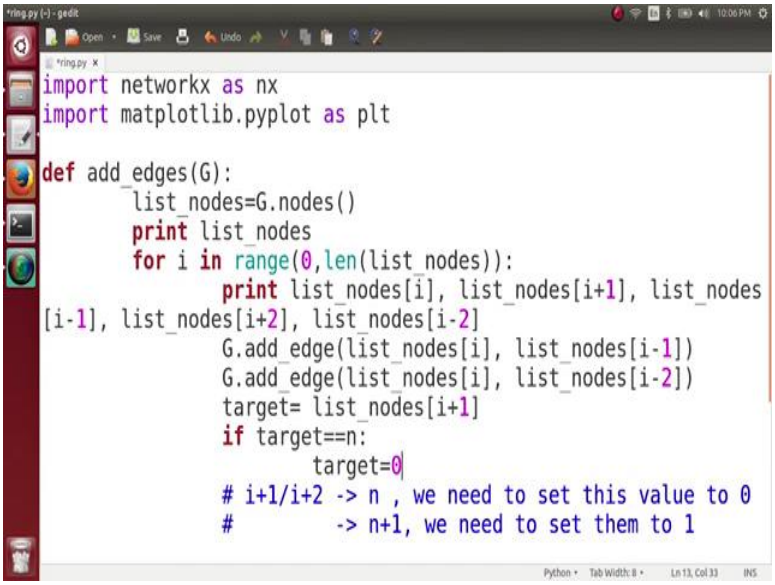


So, the last element of my list will be n - 1, where n is the number of nodes. The last element is n minus 1 and it will create problem when it has to connect to list_nodes[5 + 1] which is n. And, there is no list notes of i + 1 because the length of the list is total length. So, we cannot access this element here. So, we can simply put a loop here.

So, what we are going to do is whenever are and then also now this one thing here before doing that list_nodes[i] is actually nothing, but i right list_nodes of 0 is 0 list_nodes[1] is 1 and so on. So, whenever my list_nodes[i + 2]. So, whenever i + 1 has a danger,

whenever i + 1 has a danger of becoming n, whenever i + 1 has a danger of becoming n we need to set this value to what we go back from the starting.

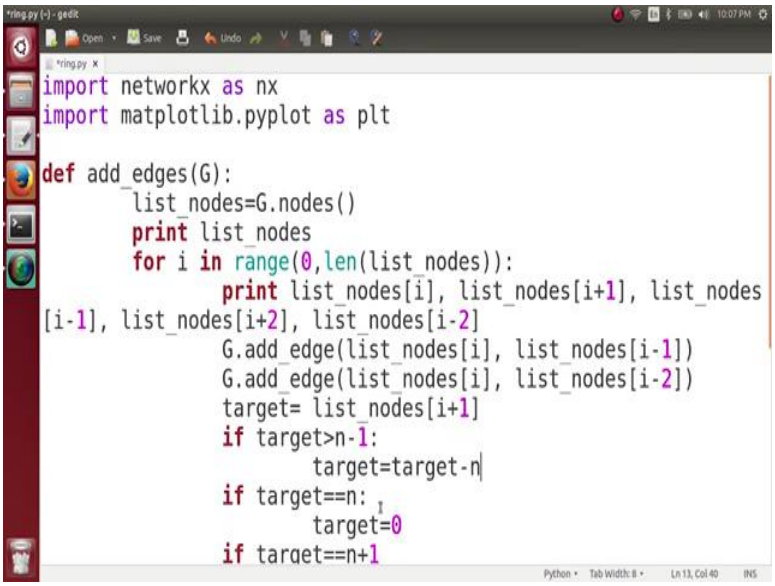
(Refer Slide Time: 06:57)



So, whenever it is n it has to be set back to 0 and whenever i + 2 it tends to become ok, it can be n or whenever i + 1 or it can also be i + 2 whenever they tend to become n we tend to set that value to 0. And, whenever these two values they tend to become n + 1 we need to set them to 1, who let us quickly see how we can set it. So, we can know that this node list_nodes[i] has to be connected to list_nodes[i + 1], [i + 2], [i – 1] and [i – 2], [i – 1] and [i – 2] are perfectly fine.

So, there can be a problem when we are connecting them to i + 1 and i + 2. So, what we are going to do is, what we have to actually do is we have to G.add_edge. And, from where to we have to add_edge(list_nodes[i], list_nodes[i – 1]) can be simply added and list_nodes[i] 2 nodes of i - 2 can be simply added. And, now what we are going to do is let us say target list node of i + 1 and what we are going to do is, if target equals to equals to n what we will do is target equals to 0 and rather ok.

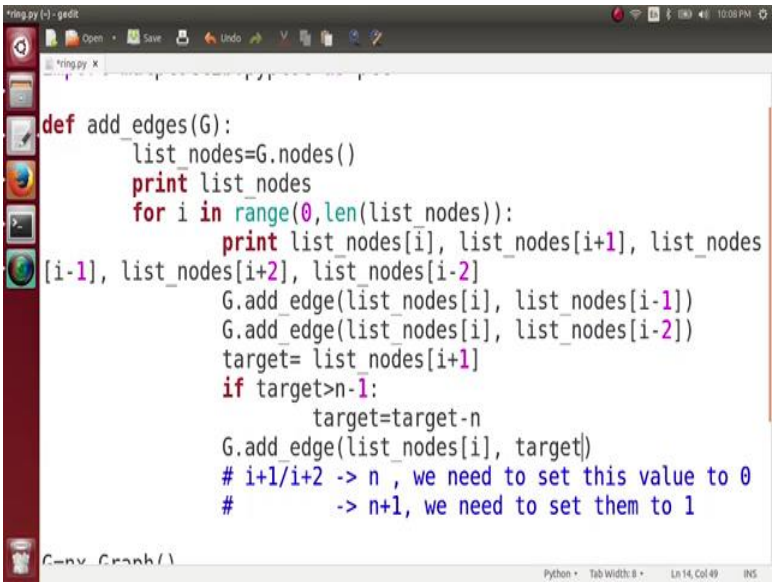
(Refer Slide Time: 09:13)



And then if target equals to equals to n plus 1, what we are going to do is target equals to 1; we can simply it. So, we can write both of these statements as 1 statement; what can that be? We can write if target is greater than n - 1, what we are going to set the target to be? So, target is going to be set to, then I write target equals to n - target. So, if target is greater than n - 1, now if target is n it should be set to 0 and if target is n plus 1 it should be set to 1.

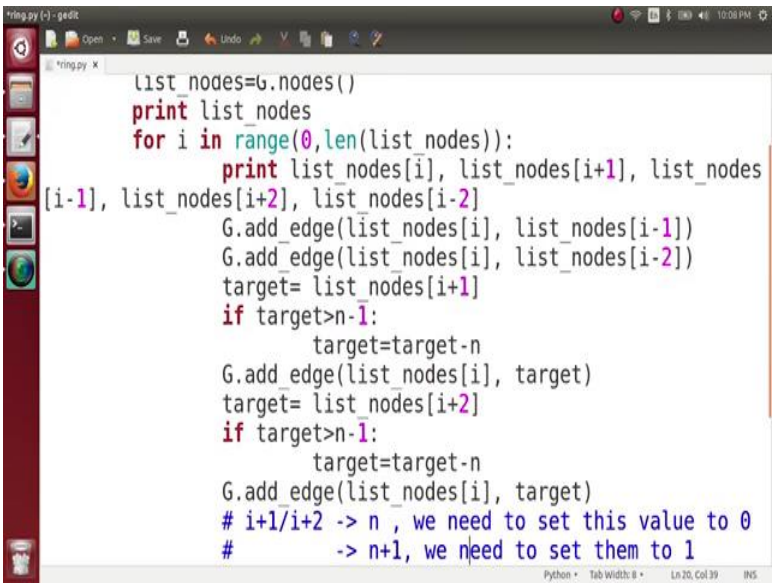
So, target is nothing, but target minus n. So, can I do target equals to target minus n, what will we do if target is n it will set target is 0 and if target is n + 1 target will be set to 1 ok. So, target equals to list_nodes[i + 1] and then this happens.

(Refer Slide Time: 10:27)



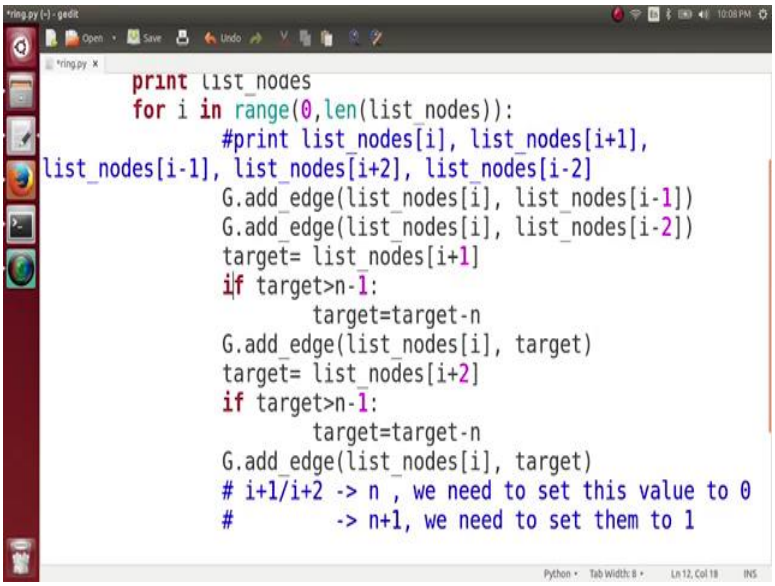
And then what I can simply do is G.add_edge from list_nodes[i] to target right and now this target can be list_nodes[i + 1] and this target is also going to be list_nodes[i + 2].

(Refer Slide Time: 10:53)



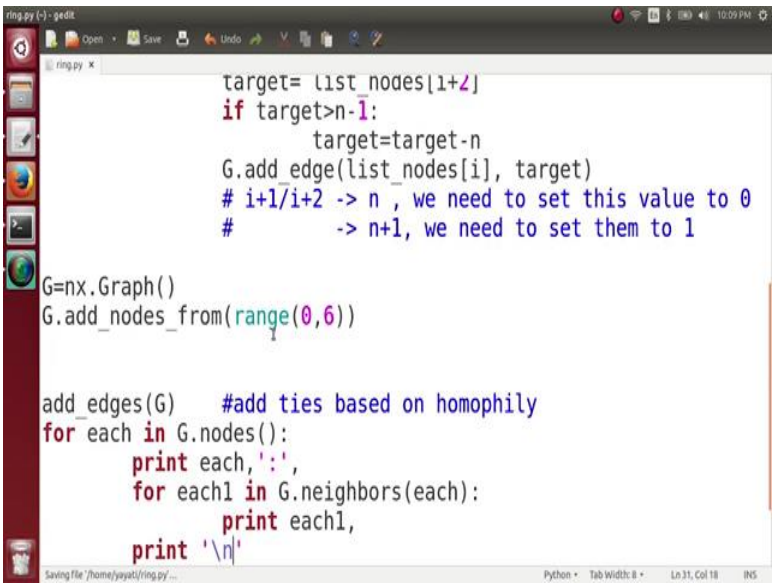
So, essentially going to replace this whole thing with if target equals to list_node[i] 2 and then this code remains the same and, then G.add_edge(list_nodes[i], target) right and then we can comment this statement.

(Refer Slide Time: 11:09)



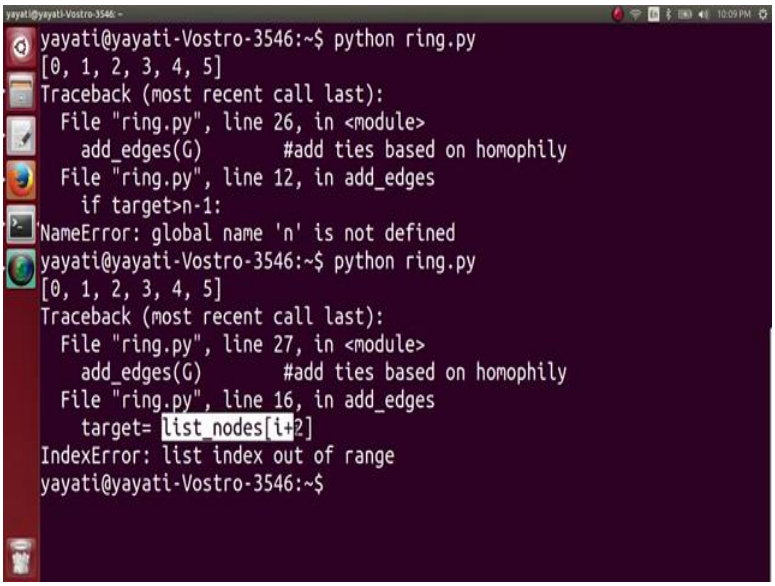
Now, to see whether this code is working fine or not, what I am going to do here is I am going to look at the neighbors of each nodes.

(Refer Slide Time: 11:13)



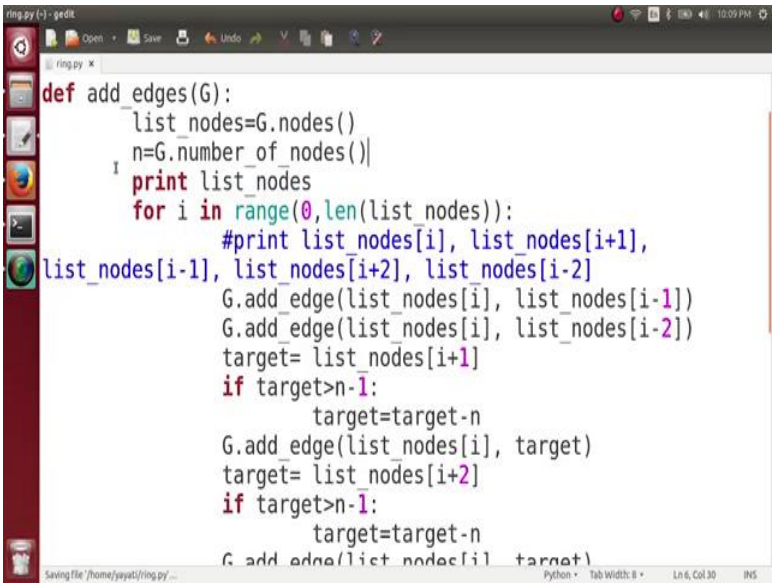
So, for each in G.nodes, I am going to print each comma and then for each 1 in G.neighbors I am going to look at all the neighbors of this node each here. And, I am going to print this neighbors one by one, print each1 then I print the new line here; let us now execute this code and see ok.

(Refer Slide Time: 11:53)



So, we get an error name n is not define. So, we have again and again used n here and we have not define what is n. So, it is easy what is n as we have discuss, n is nothing but the number of nodes in the network.

(Refer Slide Time: 12:09)

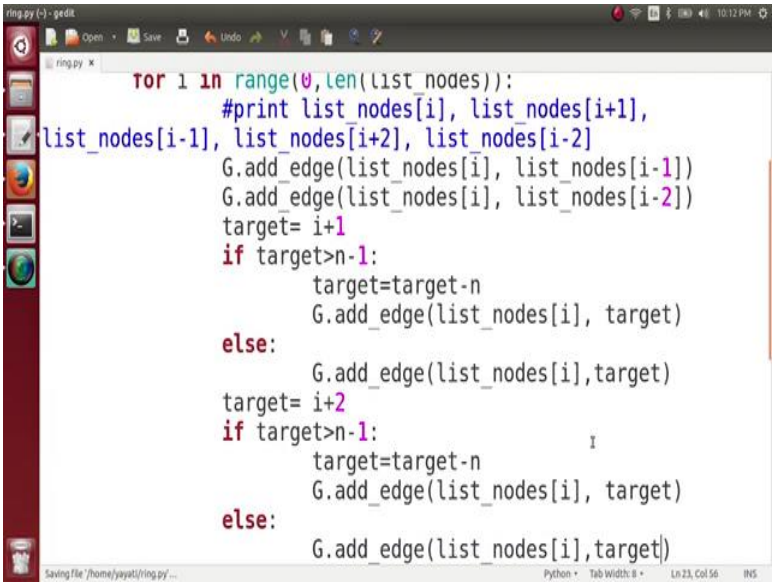


So, n is G.number_of_nodes ok, here is again some problem target equals to list_nodes[i + 2] line number 16 ok. So, what is going to happen here is when we obtain the target equals to list_node[i + 1] just simply showing as an error here. Now, what can we do

here is will be here is remover. So, we will node add list_nodes[i] is namely nothing, but i until and unless this value here becomes in negative.

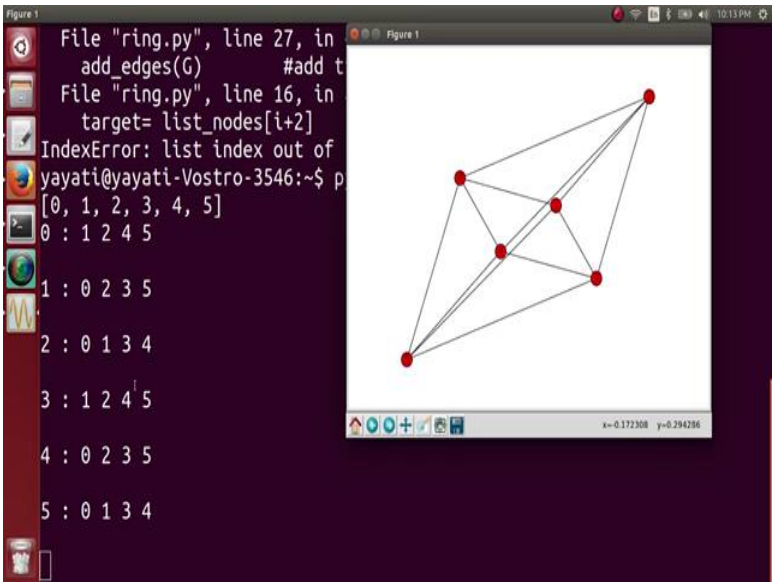
So, list_nodes[i – 1] did not be i - 1, it can go back and start from the last element in list, but list_nodes[i] is essentially going to be i. Now, what I can do here is instead of list_nodes[i + 1]. So, the maximum index in this list is n sorry n - 1. So, if it the here becomes the value n so, it cannot determine the value for list underscore nodes n. So, what I am going to do here is target = i + 1.

(Refer Slide Time: 13:39)



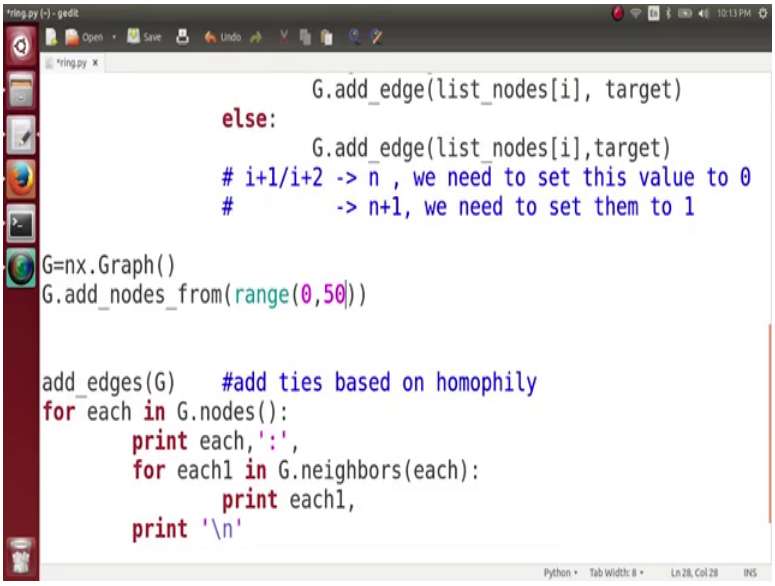
So, if target = i + 1. So, now, if target is greater than n - 1 we are going to change the target to target minus n and then we are going to add the edge. Otherwise what we are going to do is, else G.add_edge otherwise there was no problem and we can simply add an edge from list_nodes[i] to target. And, similarly here so, what I am going to do here is target = i + 2 and if target is greater than n - 1, this things happens, else we can simply put an edge from list_nodes[i] to target without changing target. So, that has now run this code and see ok.

(Refer Slide Time: 14:51)



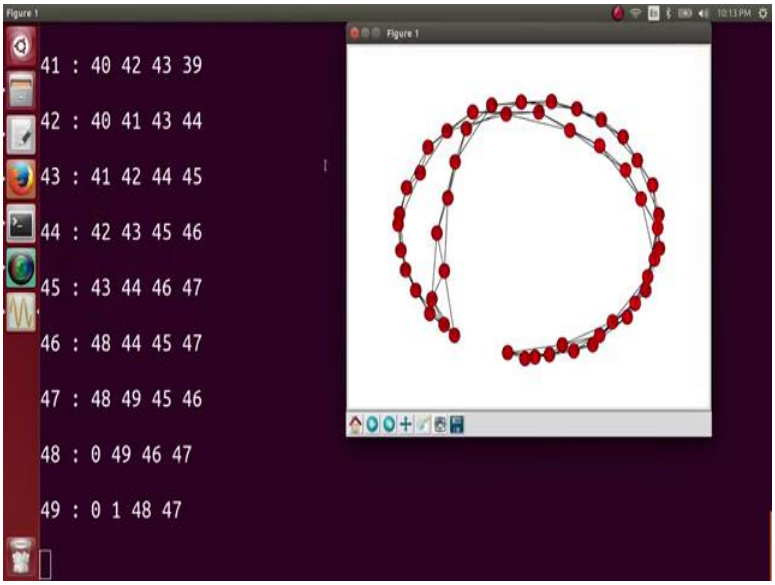
So, we say here we got a graph here 0 is connected to 1 2 and then back towards 4 and 5, 1 is connected to 2 3 0 and 5, 2 is connected to 0 1 3 4, 3 is connected to 1 4 3 is connected to 1 2 4 5, 4 is connected to ok; little bit problem with 4 here. 4 should be connected to 3 2 0 and 5 right, yeah 4 is connected to 3 2 0 and 5 perfectly fine. And, 5 should be connected to 2 towards its left which are 3 and 4, 2 towards 6 right which are 0 and 1. So, this is perfectly fine. So, we have created the homophily based links.

(Refer Slide Time: 15:43)



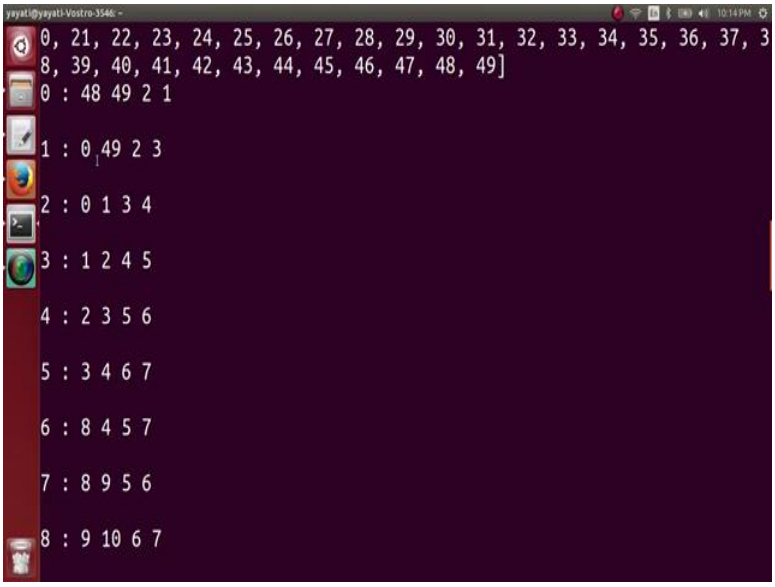
And, what we can do is we wanted a graph here for 0 to, let us say we want 50 nodes here. So, we I will make it 50 here and then we can execute it and see.

(Refer Slide Time: 15:47)



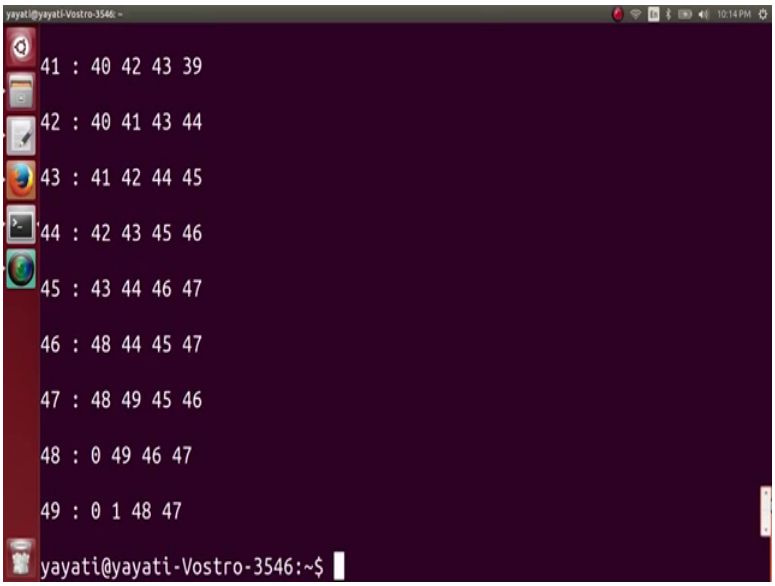
It might look here that this graph is ok, you cannot see a complete cycle here, but you can test the edges here and ensure that they all are correct.

(Refer Slide Time: 15:57)



So, you can see that 0, 1 is 0 is connected to 1 to 48 and 49, 1 is connected to 2 3 0 and 49, 2 is connected to 0 1 3 4.

(Refer Slide Time: 16:11)

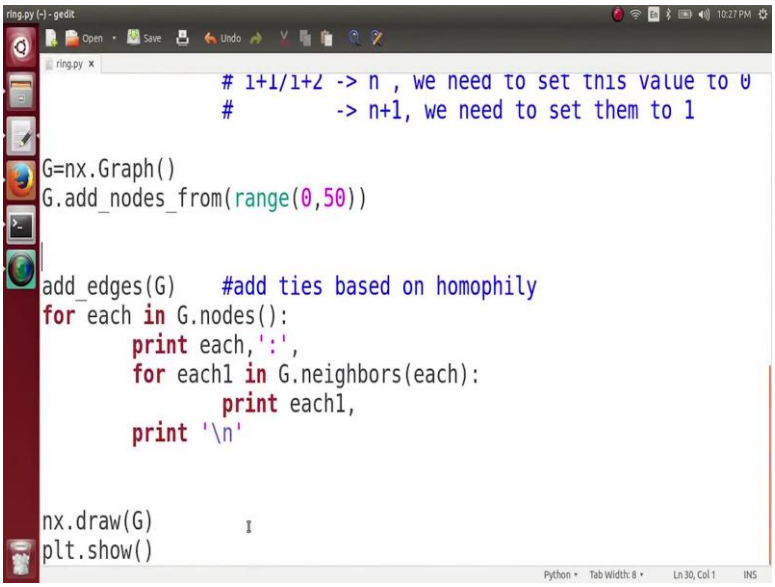


So, you can verify all these edges. So, verifying all these edges you conform that we are getting a perfect cycle here, where every node is connected to 2 nodes towards its left and 2 nodes towards its right.

Social Networks
Prof. S. R. S. Iyengar
Department of Computer Science
Indian Institute of Technology, Ropar

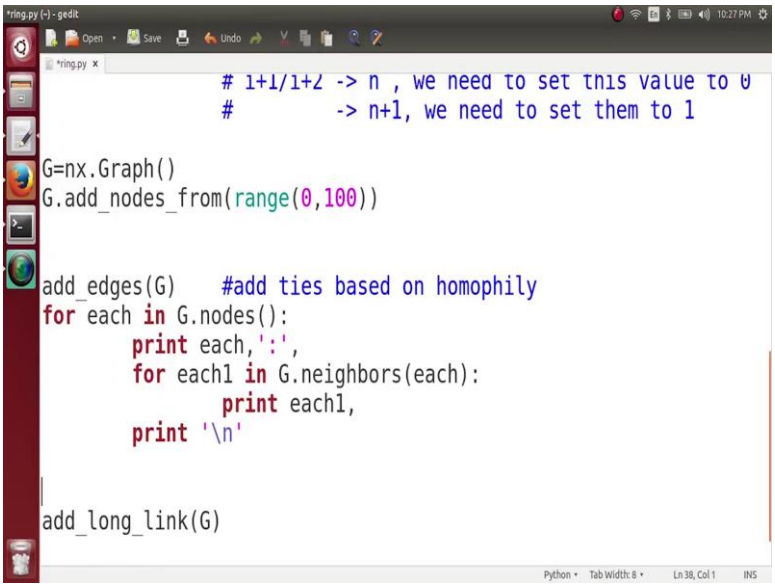
How to Go Viral on Web
Lecture - 153
Adding Weak Ties

(Refer Slide Time: 00:05)



So, now we have a network which is having 50 nodes, and which is having homophily based contexts where every node is connecting to 2 nodes towards its left and 2 nodes towards its right ok.

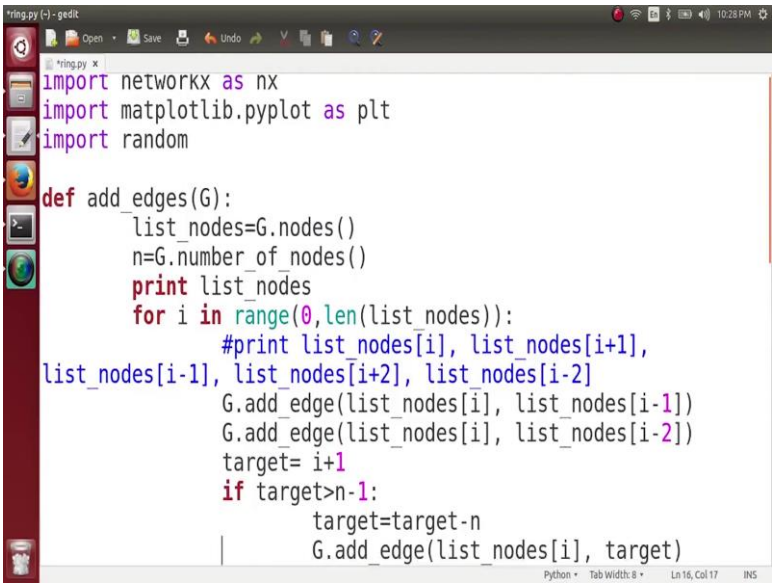
(Refer Slide Time: 00:23)



Let us take 100 node instead of 50. Next what we want to do is I am going to create a simple function which is going to create a weak tie. So, when we call this function ones it creates one weak tie, we are going only to add these weak ties. And we know that this is very easy, we have done it many times before from going to take a function here let us say add long link its basically add a long range link in G and it is going to be pretty easy, isn't it?

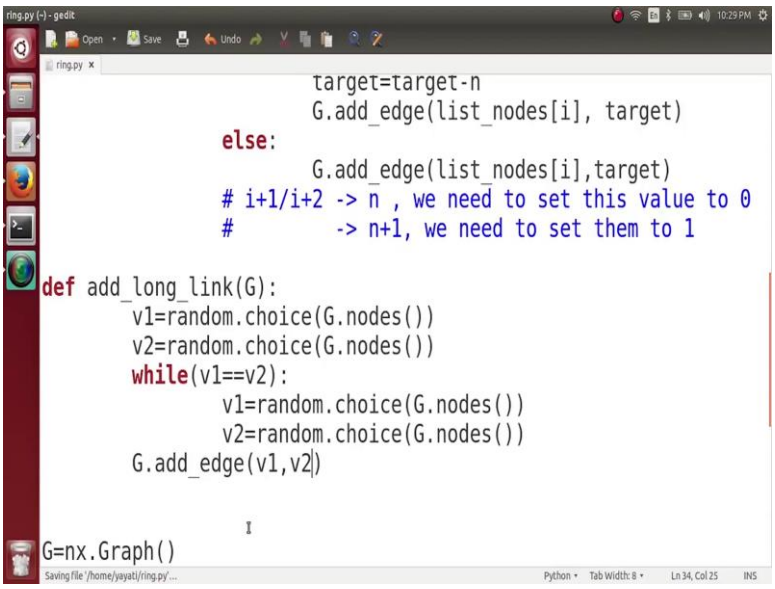
So, we what we have to do? We have to randomly pick 2 nodes in this method and put an edge between them.

(Refer Slide Time: 01:03)



So, we can actually import the function random here.

(Refer Slide Time: 01:09)

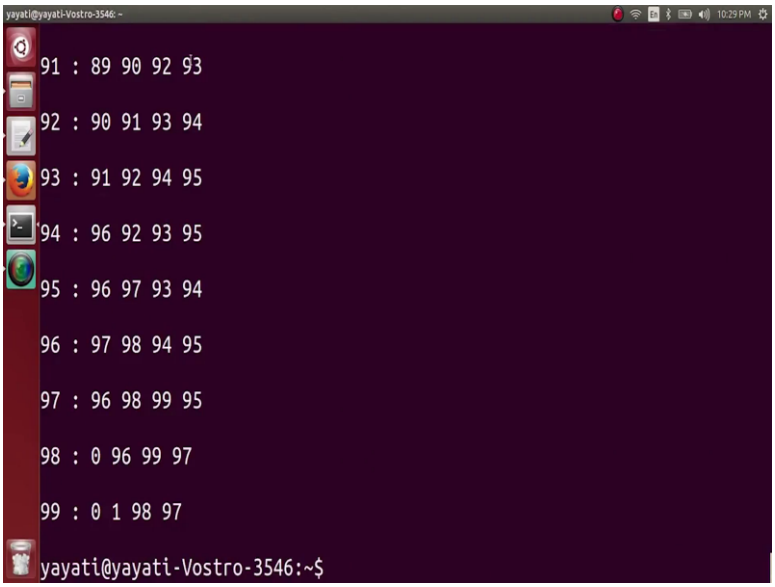


And then what we have to do is define add long link G. And what we are going to do is let us take the first vertex it is nothing but random node choice for random element from the list G.nodes, and then v2, it also equal to random node choice G.nodes. And now, these two nodes can be same. We are going to put an edge between them only then these two nodes are different.

So, what I am going to do for that is why v1 == v2. So, if v by chance v1 become equal to v2 what I am going to do is I am going to repeat the same process again till I get to different values for v1 and v2. And as soon as I get to different values f or v1 and v2, I put an edge between them G.add_edge(v1, v2).

So, when I call this function once add long link it adds two edge it adds one edge between 2 randomly picked word this is in my network.

(Refer Slide Time: 02:33)

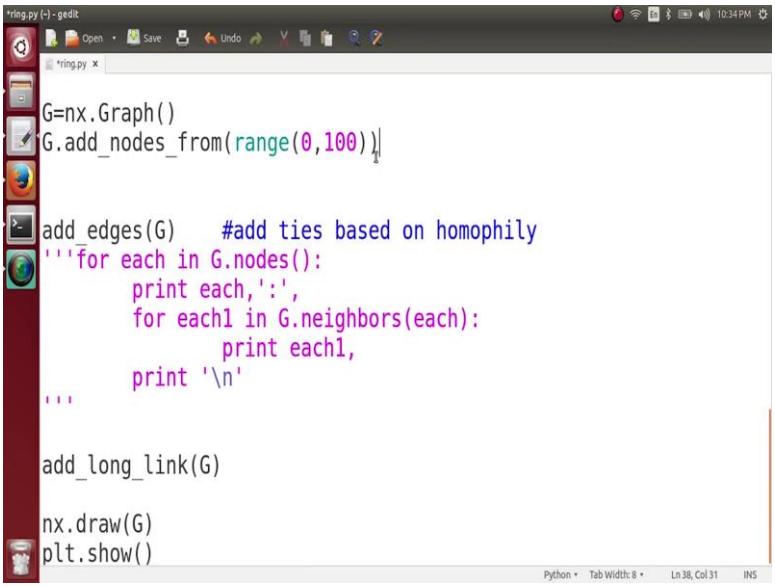


And we just check whether this code running fine or not, there is no error. So, this code is running fine.

Social Networks
Prof. S. R. S. Iyengar
Department of Computer Science
Indian Institute of Technology, Ropar

How to go Viral on Web
Lecture - 154
Plotting changes in diameter

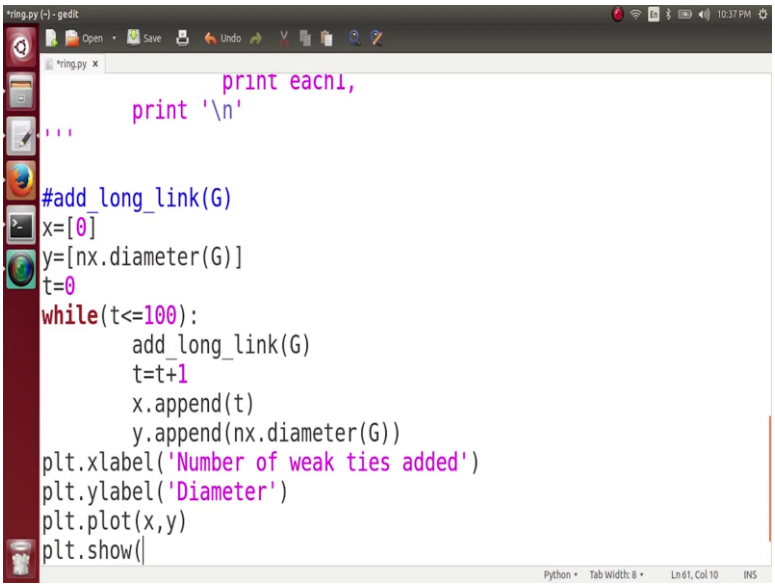
(Refer Slide Time: 00:05)



So, what we have done till now is we have created a network having 100 nodes and, on this network, we have created edges based on homophily, that is what we can say is strong ties. Next, we want to look at we have created a function which adds 1, when we call this function once we get 1 long range link in this network. Now, what we want to do is we want to see as we add more and more number of long range ties in this network, how does the diameter of this network reduces.

And, that is easy what we want is on the x axis we want the number of links, number of long range links which have been added to the network and on the y axis we want to plot a diameter of the network. And, please note since we are only adding the edges and initially this network is connected. So, this network is always going to be connected and hence, the diameter will be a positive number which is not infinity ok.

(Refer Slide Time: 01:15)



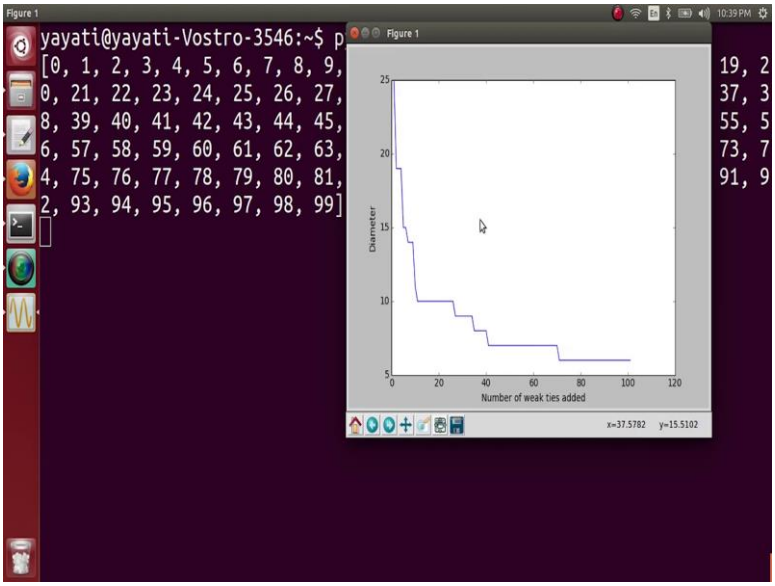
So, what I am going to do now is we need 2 axis 2 let 2 lists; one for the x axis and another for the y axis. So, this is the list for the x axis and we know that the first value for x axis will be 0; x axis shows the number of long range links added. So, when x is 0 that is when there is no long range link added in the network we know that at that time what is the diameter of my network. So, I am going to commend this over here, we have added no long range tie as of now and here the value which will be appended to the y axis is nothing, but the diameter of the network. So, this is the function which tells me the diameter of the graph G.

So, we got 1 1 value for x and y and now we starts from times equals to 0 and at every time step I am going to add 1 long rang link to this network and let say we repeat this process for 100 times. So, when t is less than 100 less than or equals to 100. So, for 100 times since I am going to do this process, what I am going do is I am going to add 1 long link in this network. And, as soon as I add 1 long link in this network what I am going to do is, I am going to increment t by 1. And, then I am going to append t to y sorry t to the list x which means that 1, which means which shows me how many long-range links have been added.

So, at this time when t is 1, 1 long range tie has been added here and simply we know what has to append to the y axis is nothing, but nx.diameter(G). So, we can get here x axis and y axis and we can simply plot them. So, let me set the labels so, plt.xlabel is

nothing, but what we are having on x axis is the number of weak ties added. And, what we are having on y axis is the diameter of the network and then we are simply going to plot plt.plot(x, y) and then plt.show ok.

(Refer Slide Time: 03:49)



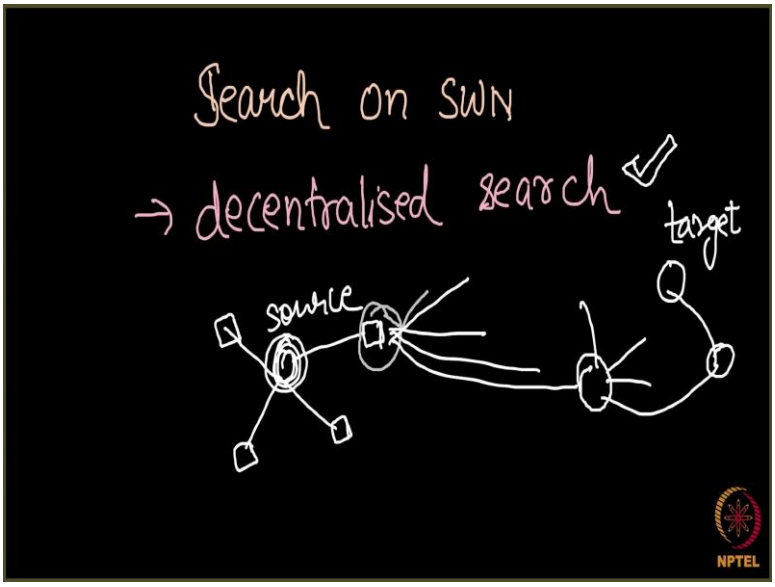
Let execute it and see ok. So, we can see here what is happening the initially you know the diameter of this network was 25. We know it is going to be 25 because, there are 100 nodes and every node is connecting to 2 nodes towards the left and 2 nodes towards the right. So, we can make a jump of 2 in we can make a jump of 2 hopes in 1 hope. So, in 25 jumps we can reach from one end of the network to another and you see that just on the addition of 1; just on the addition of 1 or 2 weak ties the diameter drastically reduces to 18. And, then and we can see that as we add more and more long range ties the diameter reduces drastically, it is not something linear. It reduces very quickly right and then the diameter of the network finally, reaches some 4 or 3 and we have added some 80 weak ties. So, this is how the curve which we wanted to look that looks like.

Social Networks
Prof. S. R. S. Iyengar
Department of Computer Science
Indian Institute of Technology, Ropar

How to go Viral on Web
Lecture - 155
Programming illustration- Myopic Search: Introduction

The next part of our programming screencast is we will see how we do search on the small world networks, search on the small world networks.

(Refer Slide Time: 00:11)



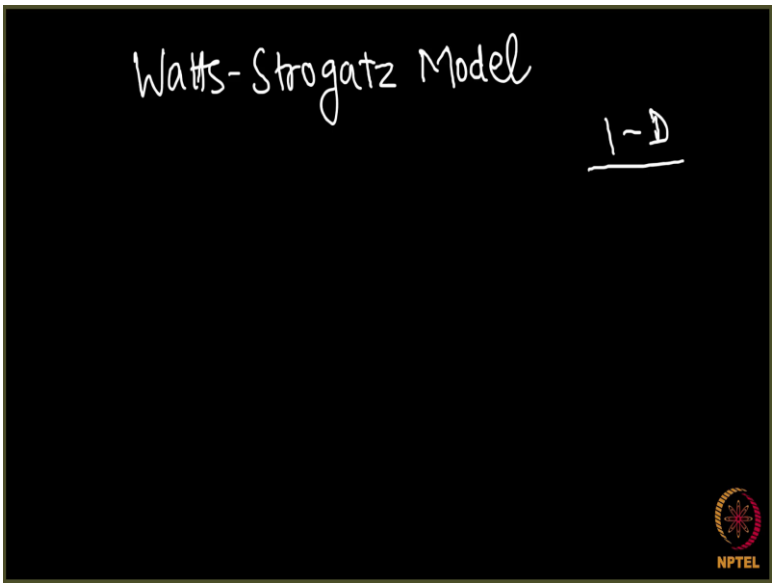
And, as we have discussed before the search on the small world networks is known as a decentralised search. Decentralised search because, it is a collaborative effort, there is no central authority no central node in the network which help you searching. And, we are have we have briefly looked at how do we do this decentralised search right.

So, how do we do this decentralised search was something like here is a node in this network and let us say this is the source node and it wants to find a path to some node, let us say the target node, let us say it wants to pass on some later do this target node. So, how this source was going to work is, it was going to look at all of its neighbours and for all of its neighbours it has an estimate of the distance of its neighbours from the target. So, the source looks at what is the distance of this neighbour from the target, what is the distance of this neighbour from this target this neighbour and this neighbour.

And it might find out that this neighbour here is closest to the target. So, it chooses this neighbour and this neighbour again here this neighbour, again here looks at all of its neighbours and chooses the one which is closest to the target. And, this one looks at all its neighbours and chooses the one which is closest to the target.

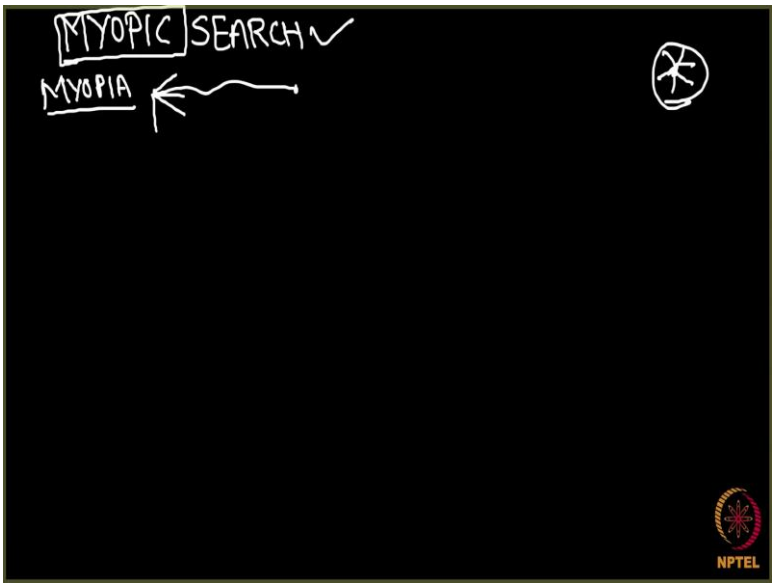
And, hence the letter in this process the letter reaches the target. Now, what we are going to do this? We are going to look at how do we do this decentralised search on a small world network, on a Watts-Strogatz model which we have studied before.

(Refer Slide Time: 01:53)



So, given a Watts-Strogatz model how do you perform a decentralised search on this model. So, let us say the model which we are talking about is that the is the ring model the one-dimensional Watts-Strogatz model and we want to see how do we perform a decentralised search over there.

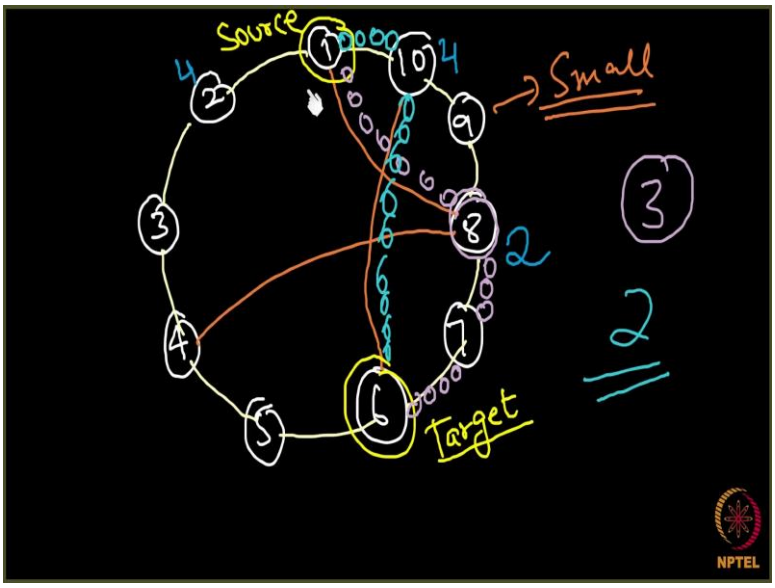
(Refer Slide Time: 02:19)



So, let me tell you this kind of search is known as the myopic search as well. So, we call this decentralised search as myopic search. Why myopic? Do you remember myopia we are studied it when we were in like 6th or 7th or 9th class, I do not remember? So, myopia; myopia is nothing, but the short sightedness; short sightedness means that you can only look at a small distance you cannot look at a long distance.

And, that is what happens in the decentralised search right, you do not know about the entire network, you know only about your locality. And, based on your locality you decide where you want to pass a letter and hence myopia. And, hence this search is known as the myopic search. Let us quickly see how we perform a myopic search before starting with the screencast. So, let me take a 1-dimensional Watts-Strogatz model here.

(Refer Slide Time: 03:13)



And, let us say I take here ten nodes arranged in a ring 1 2 3 4 5 6 7 8 9 and 10. Now, here write it 10 nodes which are arranged in the form of a ring and for the sake of simplicity I am going to connect each node. So, the homophily links each node 1 edge towards its left and 1 edge towards its right; now these are the homophily links in this network in this homophor ring and next my aim is to lay some weak ties in this network. So, I am simply instead of rewiring as we have discussed before I am simply going to add some edges here.

So, let us say I add an edge from 10 to 6 and then I add an edge from 1 to 8 let us say and let us say one from 4 to 8. And, these are be 3 weak ties in this network which makes this world small, which makes it a small world network. Now, how do we do a decentralised search over here? So, let us say our node 1 here is the source which has the letter and let us say the node 6 here is the target. We want to pass on the letter to this target which is the node 6. Now, how do our myopic search, how do our decentralized search work here.

We have seen before how does decentralised search work every node is going to look towards its neighbours and choose the neighbour which is closest to the target node 1 here. So, node 1 here it is supposed to pass on the letter to the node 6. So, according to our myopic search decentralised search it will first look at node 10 and then look at the distance of this node 10 from the target. Now, here I would like to tell you when you

look at the distance of your neighbours from the target you do not have information about the weak ties which your neighbours is connected to.

So, I might know my neighbour and I would have an estimate of the distance to the target. I will not be knowing the weak range the weak range ties of my neighbours the weak ties I will not know, I am very sorry long range contacts and a weak ties of my neighbours I will not know. I will know my weak ties, my long range contacts, but not the long range contacts of my neighbour. So, the node 1 here knows that he can pass on the letter to the node 10 and he can pass on the letter to node 8 or to the node 2.

And, then he calculates the distance for node 2 what is the distance from the target, it is 1 2 3 and 4, the distance of node 2 from the target is 4. And, then he looks at the node 1, looks at node 10 and what is the distance of node 10 from the target. Since, it does not know the long range contacts of node 10 so, we calculate the distance of node 10 to be 1 2 3 4. So, the distance of node 10 from the target is 4 and then node 1 also no more 8 whose distance from the target is 2.

So, what will happen according to decentralised search or the myopic search, the node 1 will pass on the letter to this node 8 over here. And, now this node 8 over here has the letter and then this node 8 looks at all of its neighbours node 9; whose distance from the target is 3. And, then node 1 whose distance from the target is 5 and then node 4, whose distance from the target is 2 and then node 7, whose distance from the target is 1 and it will simply pass on the letter to node 7.

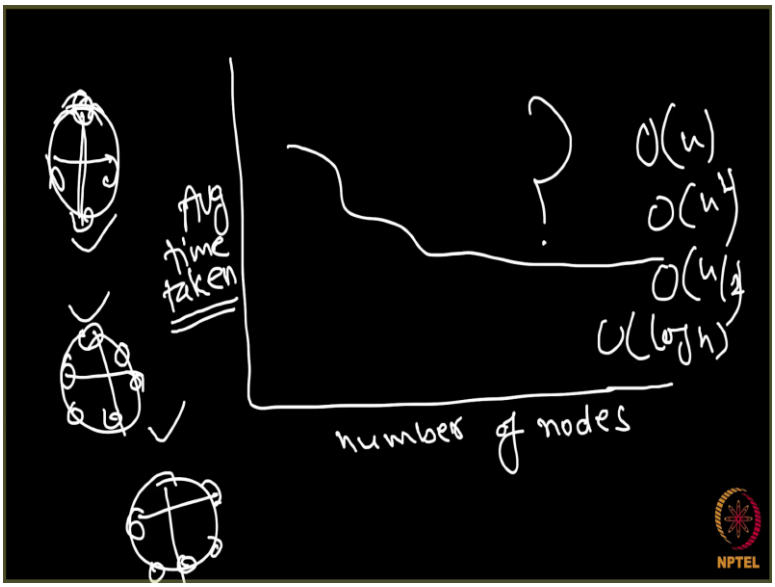
And then node 7 will look at all of its neighbours and the target is its neighbour. So, it will pass on the letter to the target. So, we can see here that this is the path which our decentralised search will find, and the length of this path is simply 3. My next question to you, now we understand what decentralised search is, what is myopic search. Do you think this search policy is optimal? What do I mean by optimal? Optimal means do you think that it has found out the best path, can you find a path from this node 1 to node 6 in this graph path from node 1 to node 6; whose distance is 3 and its actually possible.

So, if you looked at this path over here you start from node 1 and then you come to node 10 and from 10 you can directly go on to this node 6 over here. So, there is a path of length 2 from the source to the target in this network, but our myopic search has found out the path of length 3. So, it confuses that our myopic search our decentralised search

is not optimal it is not optimal, but still it gives us a path of a quite less distance with a less knowledge. So, one we were working out the optimal path from node 1 to node 6 we have assumed that we have the global knowledge about this network.

So, anybody who is knowing this entire network can find out this path, but if you know only the information about your locality, about your neighbours which is few in the real world which is difficult to get an optimal path. So, you follow this greedy approach which is called the decentralized search and find out this path. So, we will be doing this and what we will be looking at in this programming screencast is let us say here are the number of nodes in our network.

(Refer Slide Time: 09:09)



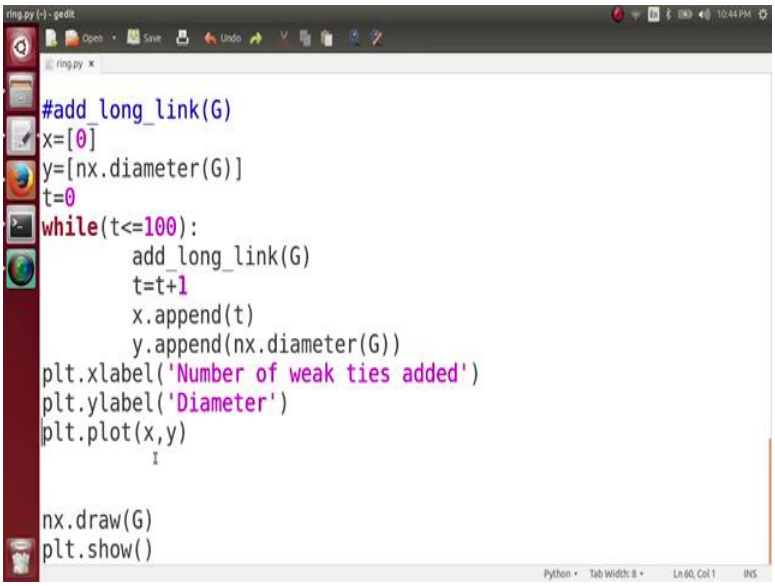
Say here are the number of nodes, mainly you can make a ring having 100 nodes, you can make a ring having 200 nodes, you can make a ring having 300 nodes and so on. So, different sized rings we are going to take of course, with the weak ties over here. So, different small world networks one dimensional small world networks we are going to take. And, then we are going look at we are going to do decentralised search on these networks and will be looking at what is the average time taken.

What is the average time taken to find the distance to find the path from the source to the target? How many steps are taken and will see at how does this plot look like, whether it is order of n whether it is order of n square or order of n by 2 or order of log n or order of log square n; what is it you will be looking at this in the second programming screencast.

Social Networks
Prof. S. R. S. Iyengar
Department of Computer Science
Indian Institute of Technology, Ropar

How to go Viral on Web
Lecture - 156
Myopic Search

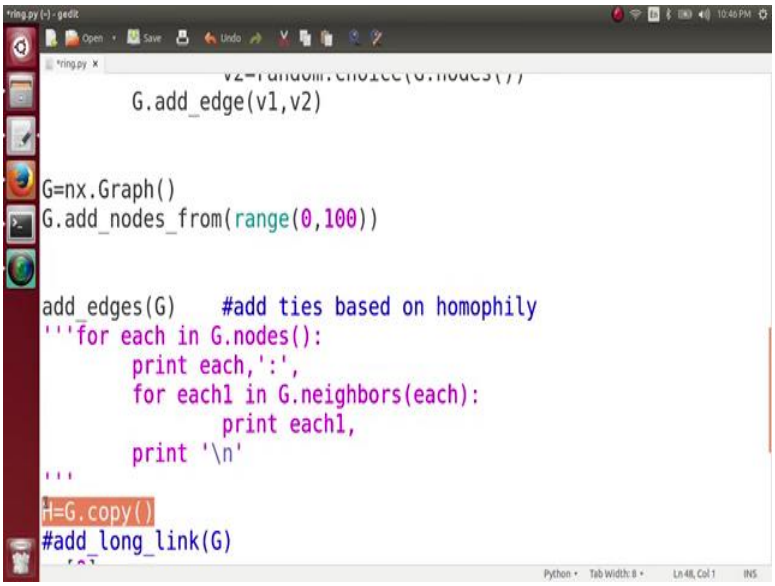
(Refer Slide Time: 00:07)



So, we have seen that how we can write a code for making a small world network in 1 dimension. What is the next aim is, we want to implement the decentralized search or myopic search on this network and look at how does this search confirm. So, what I am going to do in this code is, we look at how do we do myopic search on this network. So, for doing myopic search we remember what we have to do is we take a node and then we look at all of its neighbors. And, then we look at the distance of all these neighbors from the target and do you remember when we look at this distance, we assume that we have no knowledge about the long range context of these neighbors.

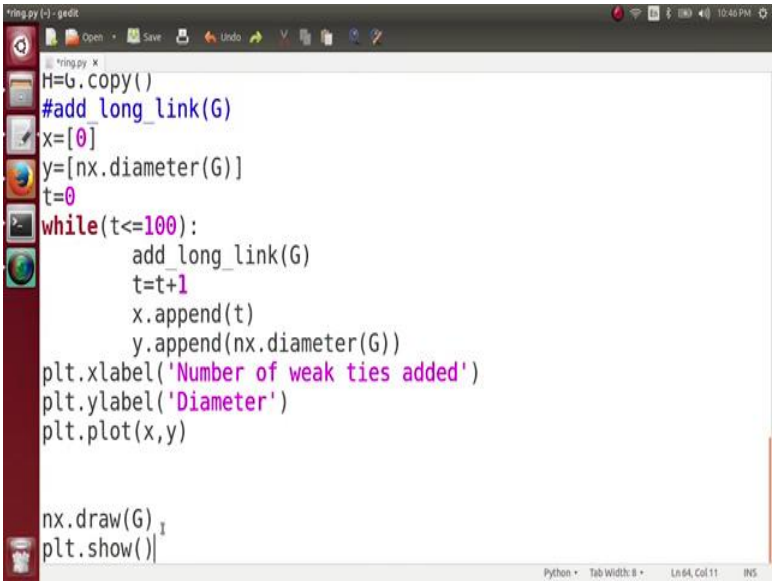
So, we have to look at the distance of these nodes across the cycle. So, while looking at that distance we assume that there is no long link in the network and then we find their distance.

(Refer Slide Time: 01:07)



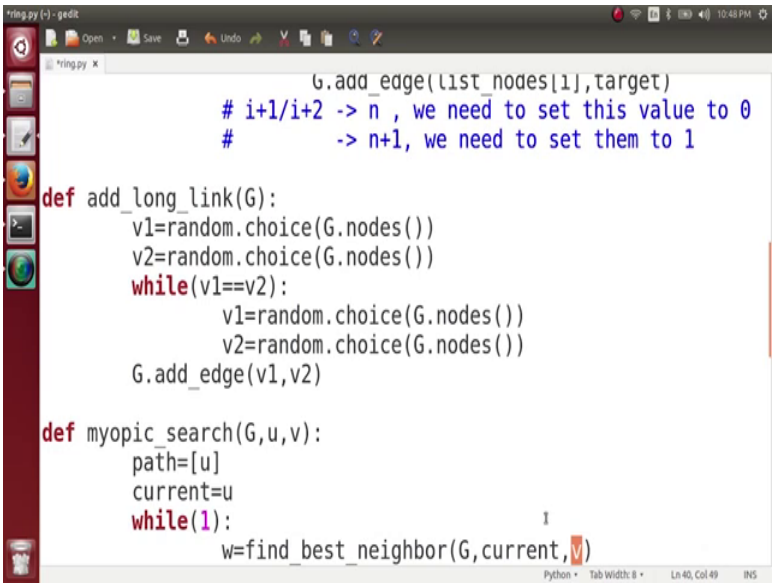
So, for that purpose what I am going to do is the graph G here and then I have added the edges here. So, before adding any long link in this network I will take a copy of this graph here H = G.copy. And, why I have taken this copy here is actually very simple, the reason is very clear what is the reason when I am going to do myopic search. So, in that myopic search I have to find the distance of my neighbors from the target and by finding the distance, I do not have to see the long range ties in the network.

(Refer Slide Time: 01:45)



So, when I have to find the distance, I look in this network edge, this network edge is this network edge. This network edge here is simply my cycle, the cycle which we have formed which consists of only the ties based on homophily and has no long range link. And, then this network G over here, G is the network which consists of the homophily based links as well as the weak ties. So, whenever I do my myopic search, I will performing the myopic search on my network G, but when finding the distance of my neighbors from the target; I will be using this network edge over here ok. So, how do we proceed is I define a function myopic search over here.

(Refer Slide Time: 02:37)

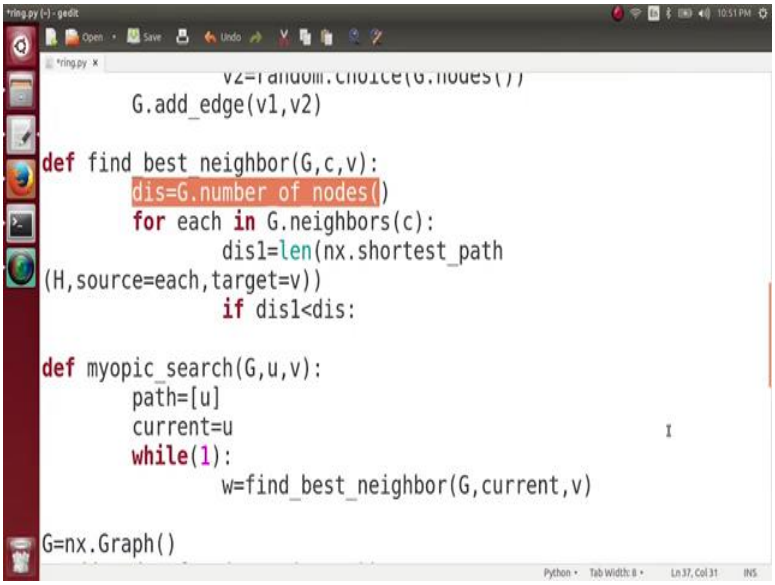


Define myopic search, I am these needs are graph the source vertex and a target vertex. So, u is my source and v is my target and we have to find the path from source and target. I take this path as a list and this path initially consists of only the node u, which is the source, current is my variable which takes care of where I am standing currently on the network. So, currently we are standing on u and while 1 what we are going to do is I take a loop over here, what we do is we have to find the next best neighbor where we have to move on.

So, I call a function here find best neighbor find best neighbor G current where, have to find the best neighbor from current and for finding the best neighbor I will be needing the target. So, what I am going to do is for finding the best neighbor this current node will be looking at all of its neighbors and we choose the one which is closest to the target

in the graph H, not in the graph G. In the graph H where there are no long-range ties. So, before proceeding further let me define this function here.

(Refer Slide Time: 04:01)



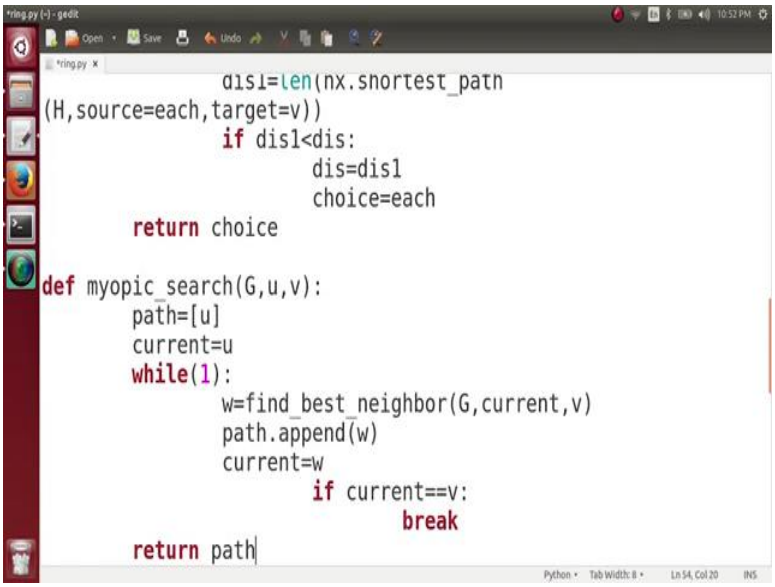
Define find best neighbor, define find best neighbor G current node and the target. So, I get a variable over here distance equals to G.number_of_nodes, you will soon understand why I have taken why I am taking this variable over here. So, what do I do next is I look at each of the neighbors of this node current here? So, for each in G.neighbors(c) so, for each in G.neighbors(c) what I am going to do is, I am going to calculate again a distance, distance 1 which is the distance of this neighbor from the target. So, for finding a distance of the neighbor from the target we can use the function nx.shortest_path which gives us the shortest path between 2 nodes in a graph; nx.shortest_path and the graph H.

So, you understand why we are taking H here because, we have to look at the distance across the cycle not across the complete graph, we have to ignore the long range ties. So, H and what is my source. So, here this is the source for my shortest path. So, the shortest path function requires three parameters the graph, the source that is the first node and the target which is the second node; approximate I am finding the distance. Please do not confuse it with a source and target over for our myopic search, the source and target for myopic search are different and these source and target are different. So, when we were doing the myopic search, we were at a node current and now this node current is

interested in finding the best neighbor and for all of its neighbors it is calculating their distance from the target.

So, here the source here is nothing, but the neighbor of my node current. So, the source is what? Source here is each which is the neighbor and target here is v. So, the target is actually the same the source changes, now what I am going to do is, if this distance is less than the previous value of distance which was quite high. So, we want this value to be quite high and if my distance 1 it is less than distance; what I am going to do is distance equals to distance 1.

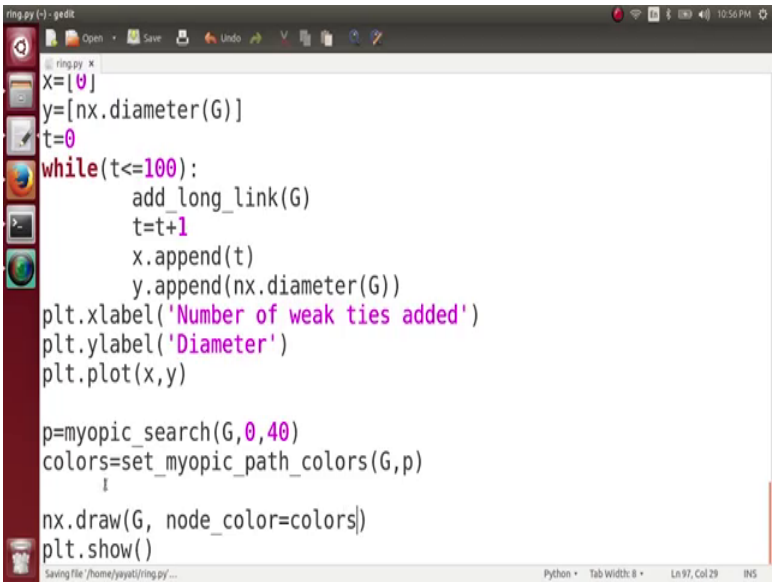
(Refer Slide Time: 06:41)



So, this is a simple method in coding which we used to find out the maximum value or the minimum value. And, I set the value of choice the best neighbor to be equal to each and at the end I return choice. So, I think that this is pretty clear and then I return back to my code for myopic search. Here I have find out the best neighbor for my node current and after finding the best neighbor I append this best neighbor to my list to my path.

And, then the value of current becomes equals to this neighbor which I have found and also by doing this if the value of current becomes equals to v which is the target; then what we have to do is simply come out of this while loop and then we can return the so, we can return path h v. So, we return path here ok. So, this is the code for our myopic search and we return path here ok. How do we execute it?

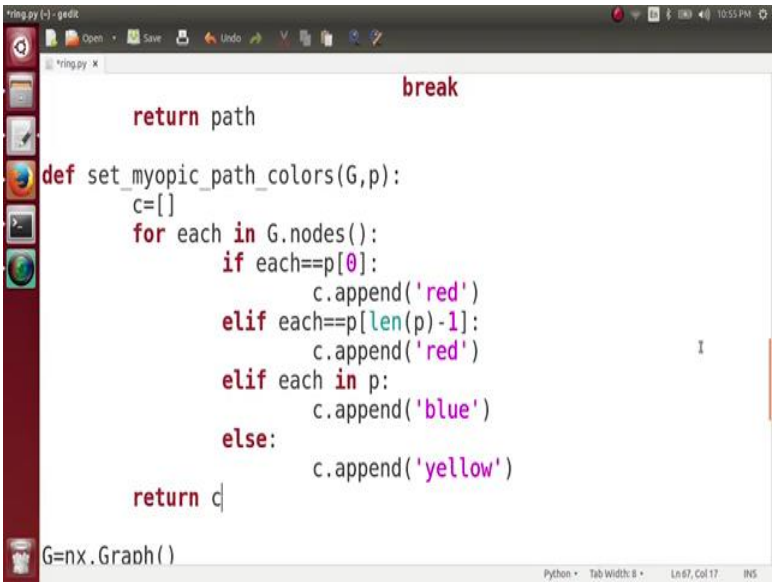
(Refer Slide Time: 07:53)



So, let us say p equals to the path which I wanted to find out and myopic search and let us say I have performed the myopic search, let us say from the node 0 to node 40. And, I wanted to see how this path looks like; I am going to play around with colors little bit here what I am going to do is, I am going to call a function here set myopic path colors why no.

So, I guess you know what I am going to do here. So, when we are going from this node from 0 to 40 I want to see how does this path look like. So, what I am going to do is I am going to define a function here, function for set myopic path colors. And, here I have to pass my graph G and the path p and it gives me an array called colors and let us define this function here.

(Refer Slide Time: 09:05)

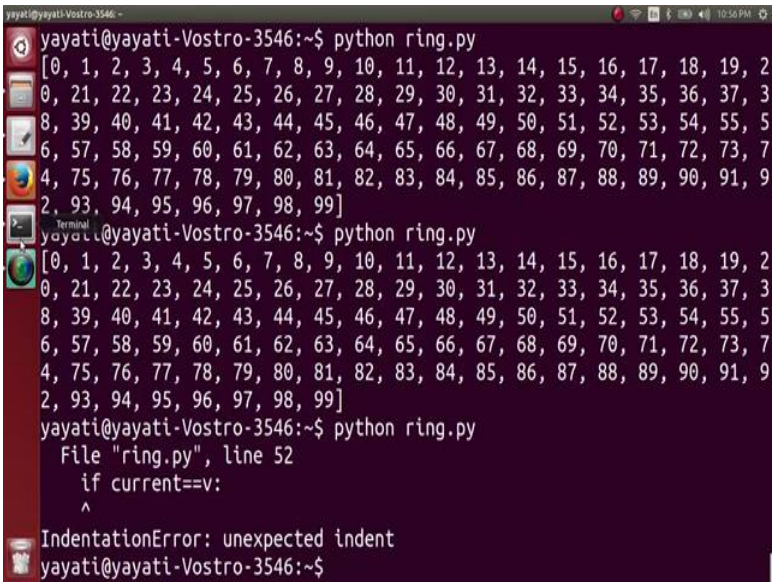


Define set myopic path colors and what the function is going to do is for I have a array here c, which is for colors for each in G.nodes. What I am going to do is, if each equals to let us say p 0 which is the starting node of our path; what I am going to do is for each in 0th node, if each is a starting node of our path and going to append a let us say red color here. And, if each equals to equals to the last element of my path then also I am going append red. What I am going to do is the source and the target here are going to be red.

The source and target here are going to be red and are going to be red we call it is actually not else if, it is elif and elif each in p ok. So, if it is the starting node of this path p, we append a red here. If it is the end path of this list p of this path p we append a red here, if it is neither starting node and but exists in p. So, what I am going to append there is let us say blue and what do I do for rest of the nodes I make them yellow.

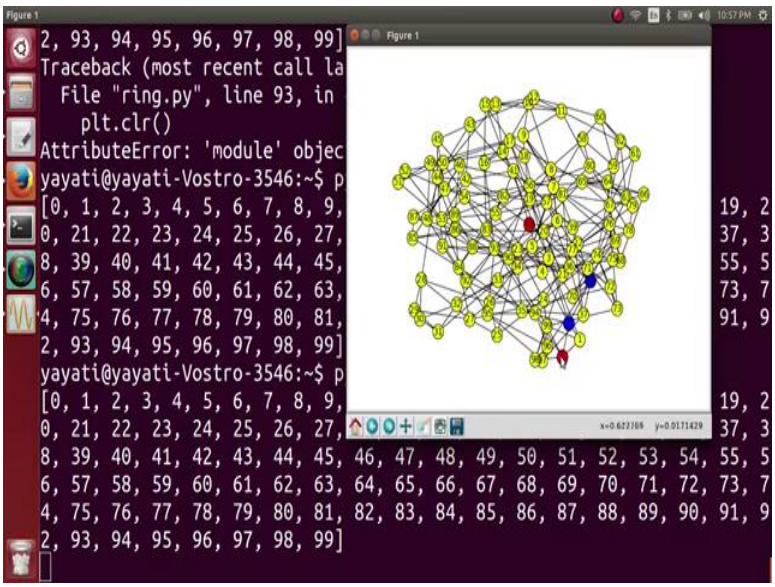
So, all the nodes in my graph are now going to be displayed in yellow, the starting and the ending nodes are going to be red and the rest of the paths on my; rest of the nodes on my path are going to be blue. And, then I return c here you will soon understand why I am using colors here and then we got colors here nx.draw(G) node underscore color equals to colors.

(Refer Slide Time: 11:35)



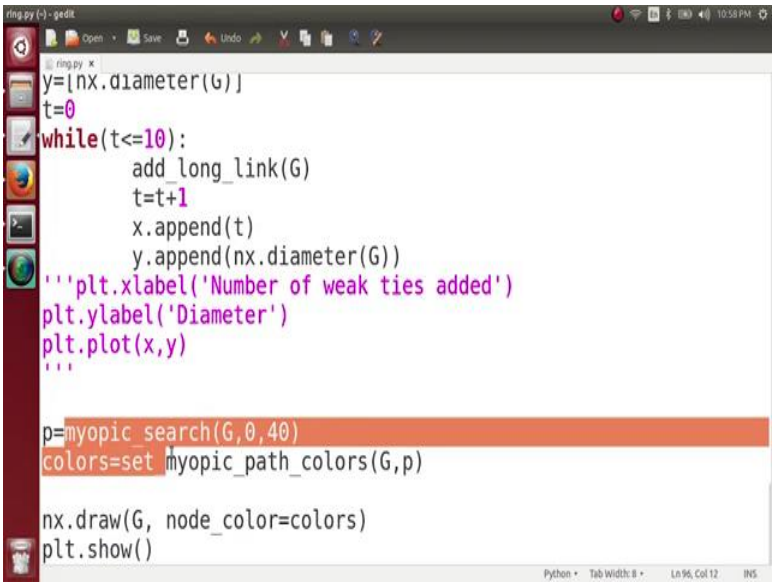
And, let us now see execute it and see line 52 shows an error. So, I have commented this portion over here and let us now execute it and see.

(Refer Slide Time: 11:47)



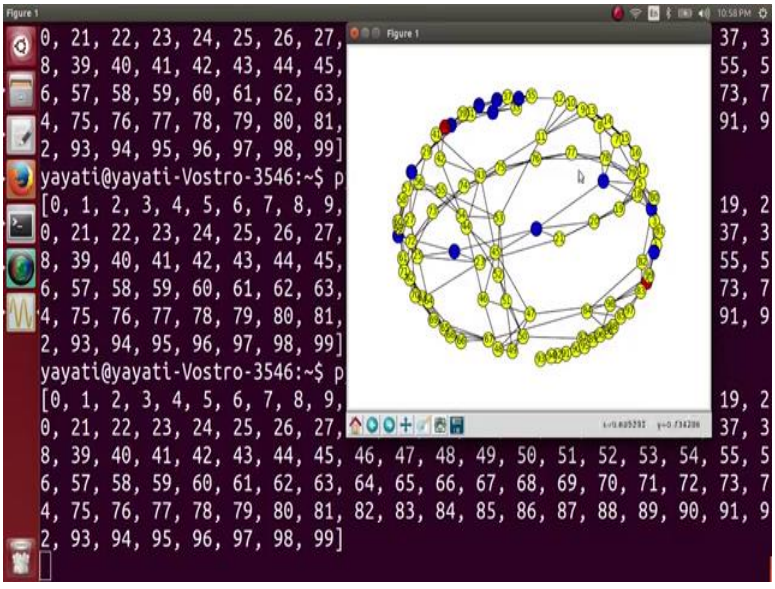
You can see that when we have to find a path from this node 0 here to 40. So, we can go from 0 to 2, 2 to 38 and 38 to 40. So, this is the path which my myopic search gives you and the network here looks quite random. So, we know why it looks random over here because, we have added a lot of long-range links to this network. So, what I going to do is, I am going to change this value of t here to let us say 10.

(Refer Slide Time: 12:13)



So, let us add only 10 long range links over here and we know that what was the initial diameter of this network was nothing, but 25. From that 25 diameter after that I have added 10 long range links and then I find my path from the node 0 to 40 using my myopic search and let us see ok.

(Refer Slide Time: 12:39)

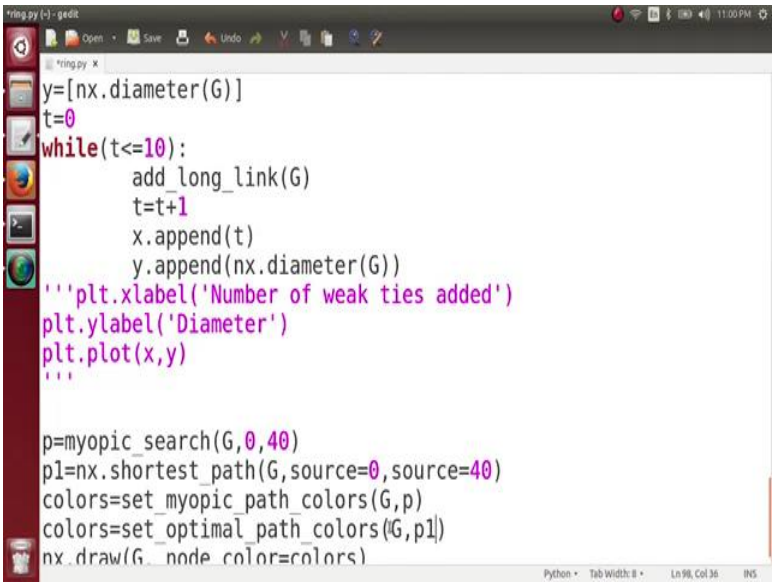


Now you see the path length has increased quite a lot and this is my ring network although it does not appear here quite like a ring. So, it starts here from node 0 and there

are 1 2 3 4 5 6 7 8 9 10 11 12 nodes in between. So, within so, the path length here is 12. So, the myopic search here finds a path of length 12.

What next? I am going to do is we have seen that the myopic search is not an optimal search. Let us now compare the myopic search to my optimal search and what we are going to do now, what we are going to do now is we will find the optimal distance let us say from this node 0 to 40 only.

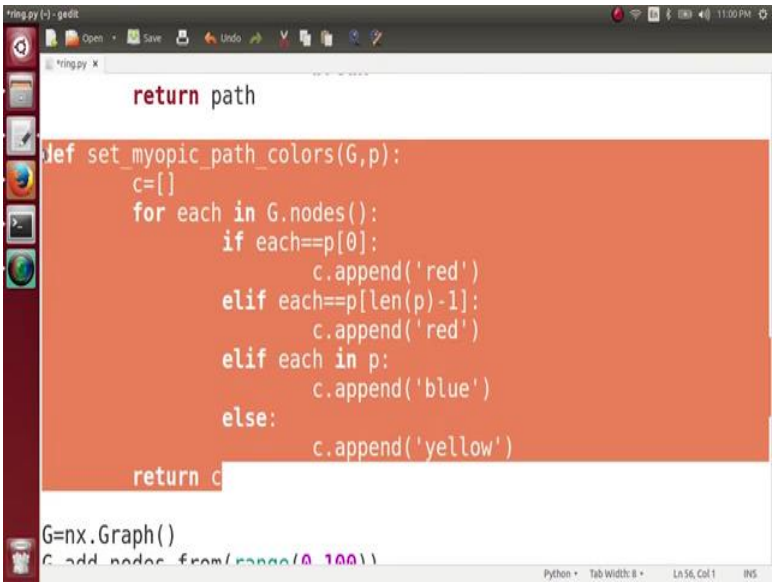
(Refer Slide Time: 13:27)



So, what I am going to do is I am going to find another path and this path is nothing, but the shortest path, the optimal shortest path on my graph G from node 0 to 40. So, you see here I am applying a shortest path on the graph G.

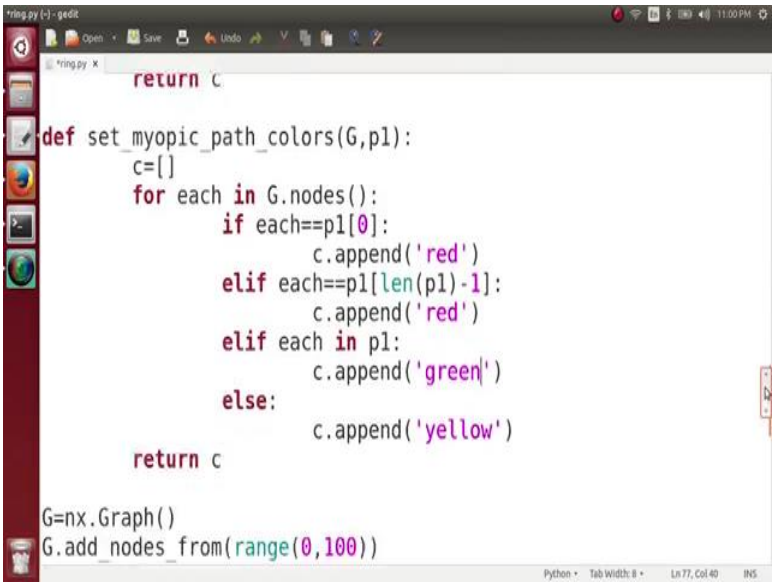
So, this graph G here is consisting of the long range link also and then I find p 1 and what actually we can do here is I want to see a different path also. So, here I am going to, what I am going to do is set optimal path color. So, I am also going to set optimal path colors here based on the path obtained from here from G to p 1 and then this is going to be quite easy.

(Refer Slide Time: 14:21)



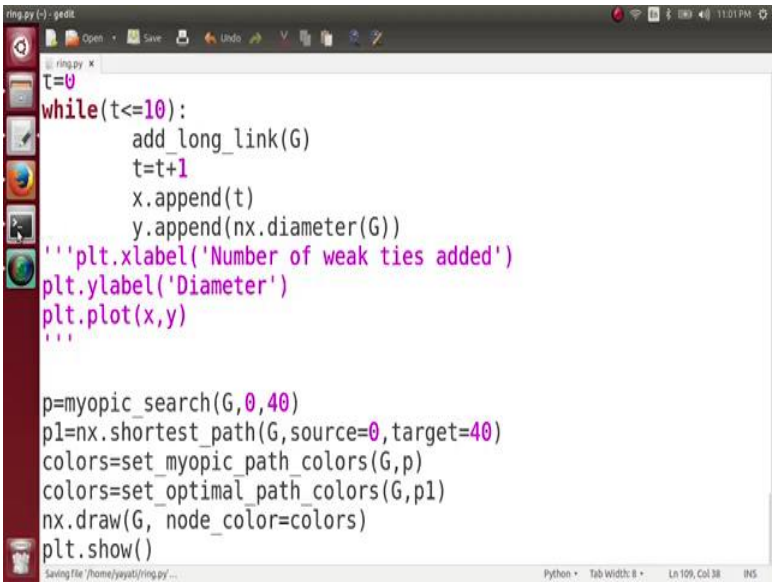
So, we are going to use the same code which is here.

(Refer Slide Time: 14:29)



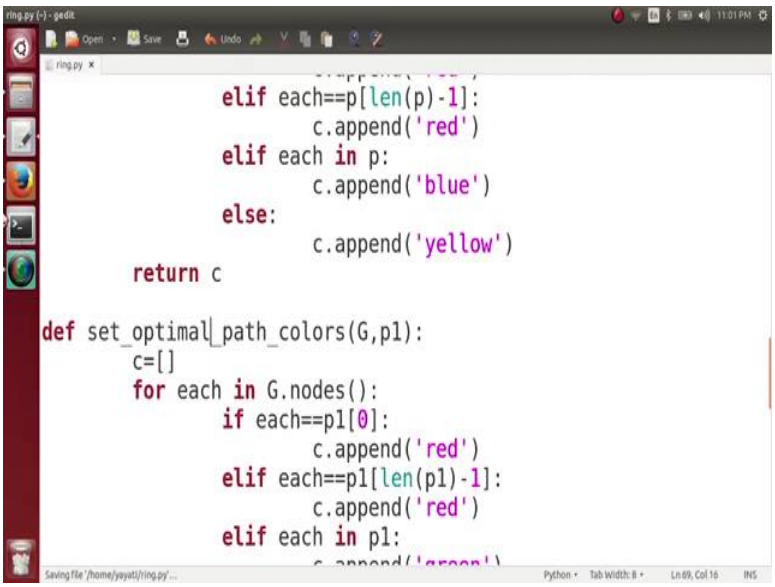
And, let us say so, here it is (G, p1) and then p1, p1, p1, p1 and what I am going to do is the starting node is red the ending the target node is also red. The middle nodes on this optimal paths let us color them green and the remaining nodes in the network are yellow. And then we can see and let us now execute this code and see the difference between both the paths.

(Refer Slide Time: 14:59)



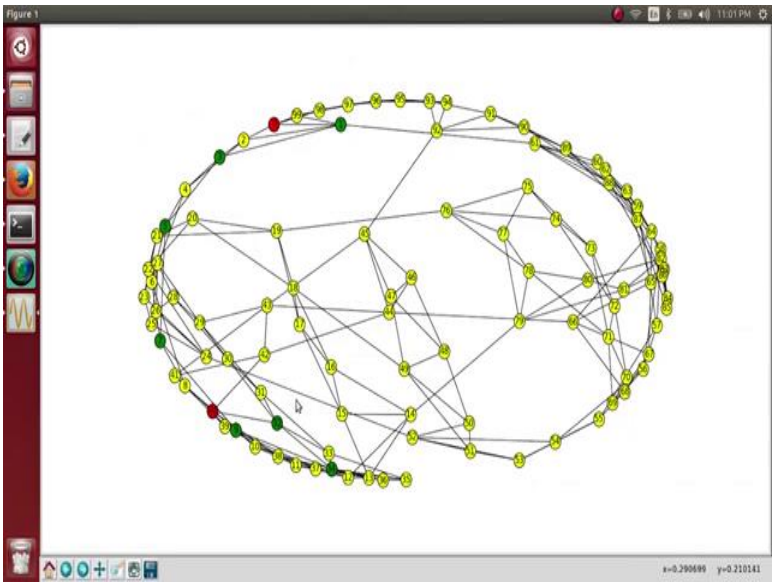
Source I am very sorry source equals to 0 and this is target equals to 40, let us execute it and see set optimal path colors.

(Refer Slide Time: 15:19)



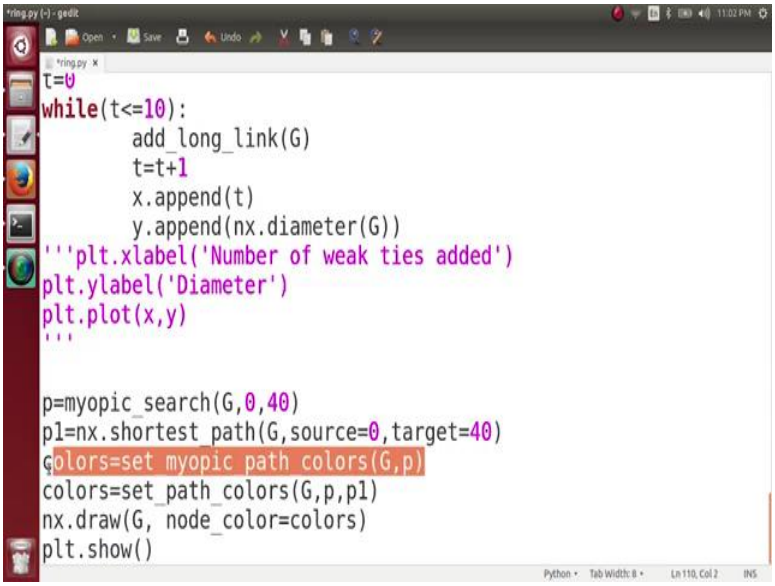
This is set optimal path colors.

(Refer Slide Time: 15:23)



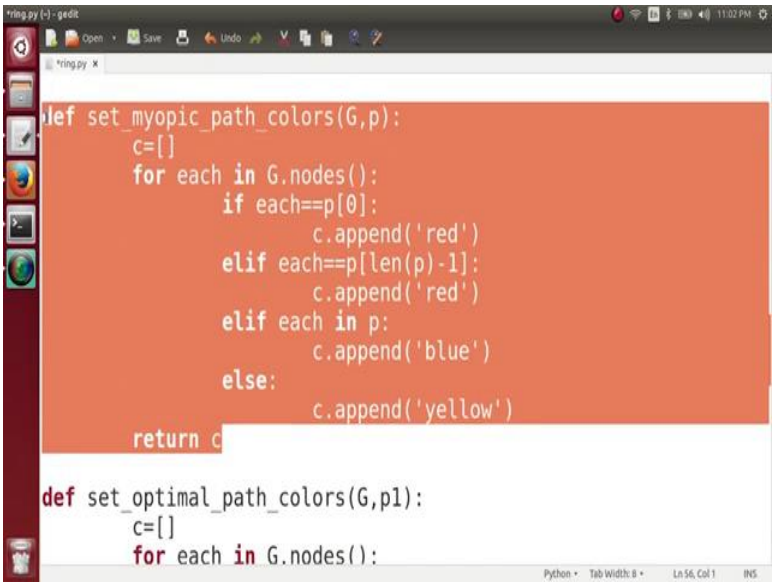
Now, you see this network over here ok. So, here is an error. So, what is the error? When I am doing the set optimal paths colors, I have recolor the nodes, which I have found in the myopic path. So, we have to be a little bit careful so, what we will do is here when we have to find colors, we will actually have to combine both these functions.

(Refer Slide Time: 15:55)



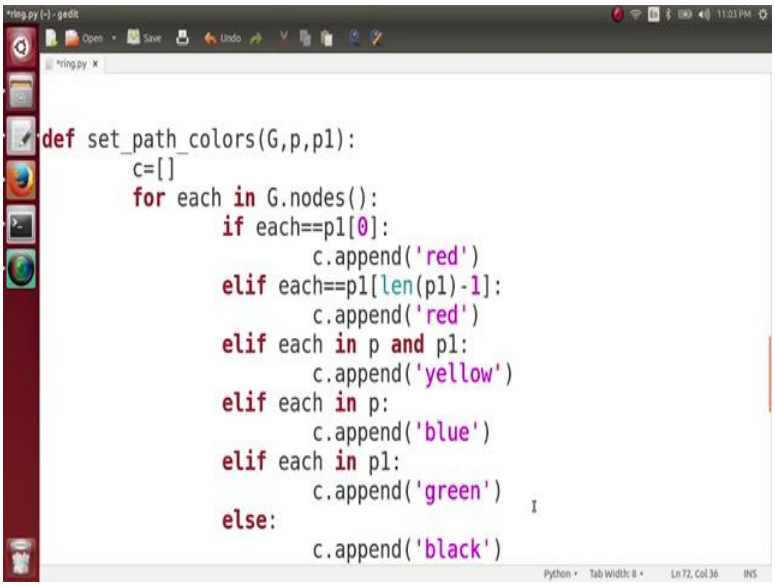
Let us say set path colors (G, p) and p1 ok, (G, p) and (p1) and then we need only one function here.

(Refer Slide Time: 16:05)



Set path colors.

(Refer Slide Time: 16:09)

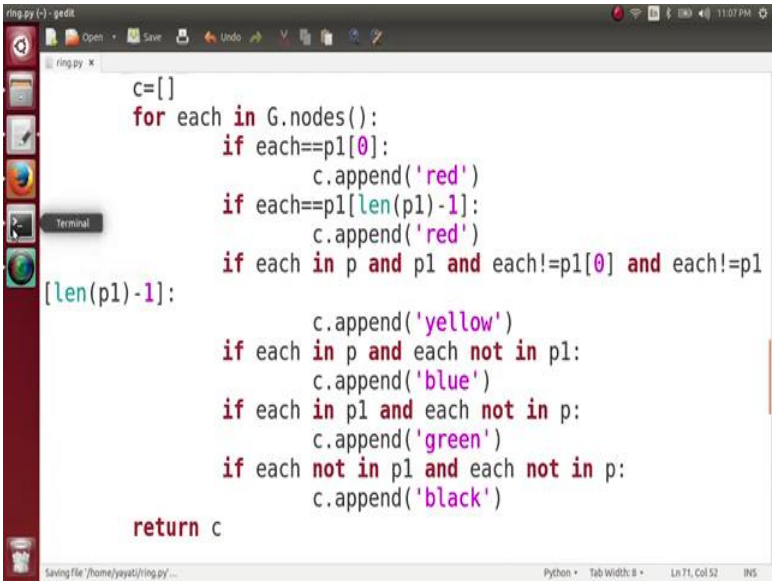


(G, p, p1) and what I am going to do is; obviously, what is if each is p1[0] it is p[0] also because, the source node is same red and then here it is red. And, it is each in p1 when we are making it green and you see here one node can be both in p and p1 right. So, if there is a node which is present in both p as well as p1 and it is neither the source node not the target node we are coloring that ok.

Please be careful with the colors here, we are coloring that node to be yellow ok. So, yellow is a node which occurs in both these paths p and p1 elif each in p, p is the path for myopic search what we are going to append here is; for myopic search we are going to use blue. And, elif each in p1 it is then we are going to append green over here and else it is in none, we are going to append a black over here. So, please see all the nodes in this network are going to be black, the source node is red the target node is red.

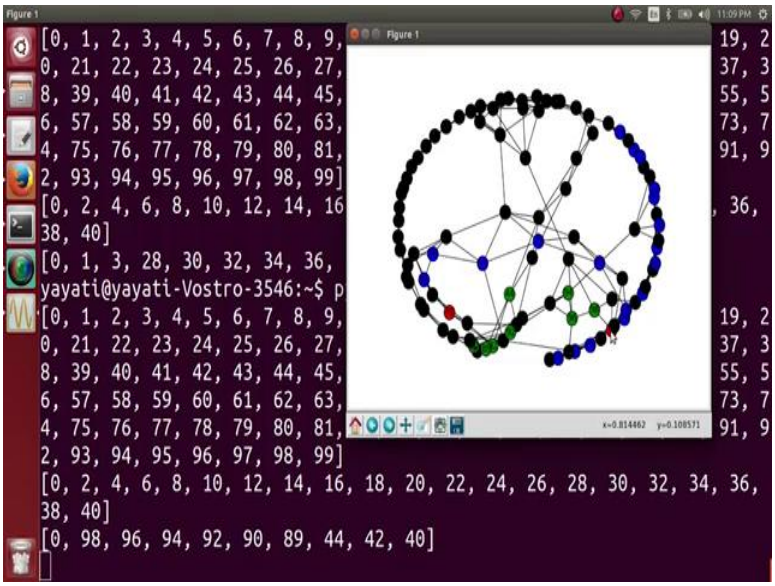
The nodes which are in myopic in the path for myopic search are blue, nodes which are in path for optimal search are in green and the nodes which are in the paths for both the myopic search and the optimal search are in yellow. So, we are going to make it a little bit more rigorous and more understandable, what we are going to do is ok.

(Refer Slide Time: 17:55)



If each in p and p1 and each is not equals to p1[0] and each is not equals to p1[len(p1) – 1] which clearly says that if each in p and p1 it is in both the paths. But its neither equal to the source vertex not equal to the target vertex then we are going to append our yellow here, if each in p and each not in p1 then we are going to make it blue. And, if each in p1 and let us say each not in p then we are going to make it green. So, instead of an elif here we can use a if here, here also we can use a if and at last if each not in p1 and each not in p we are going to make it black.

(Refer Slide Time: 19:09)



Now, let us execute it and see, from here you see the there is no node common between both of the paths. If you look at the myopic search its start from 0 and close to 40, it takes this path 0 to 2 to 4 ok; it is actually little bit difficult to make out where this path is going. So, before 30 there is 28 ok. So, this path goes something like this, the ending path is here ok. Let us see from 0 it goes to 2, 2 is somewhere here, from 2 it goes to 4 which is here and then to 6 8.

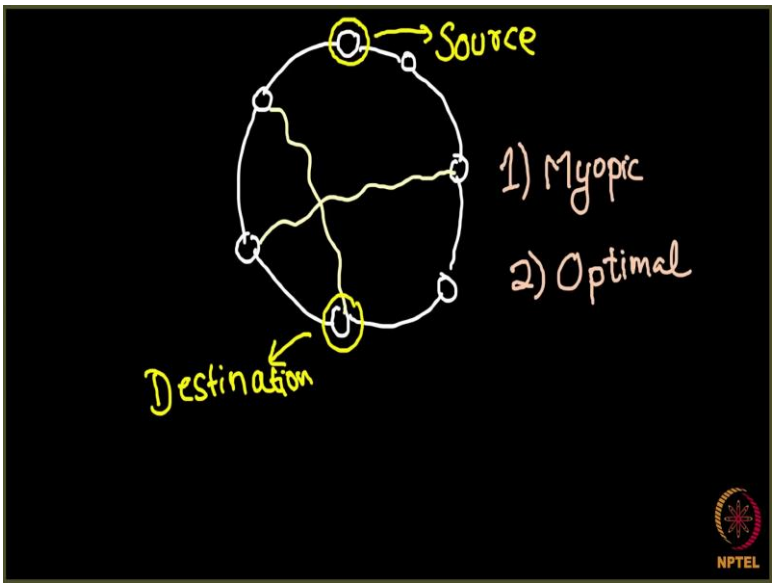
So, it goes all the way here and then it comes here, here and here and reaches 40 and why an optimal path is starts from 0 and goes like this. So, we can see that optimal path is much shorter than what are myopic search is finding out, which clearly shows that a myopic search is not the optimal search.

Social Networks
Prof. S. R. S. Iyengar
Department of Computer Science
Indian Institute of Technology, Ropar

How to go Viral on Web
Lecture - 157
Myopic Search comparison to optical search

So, what we are going to do in the first programming screencast is, we are going to take a small world network in 1 dimension which we have discussed already what is that, that is a first of all a ring.

(Refer Slide Time: 00:15)



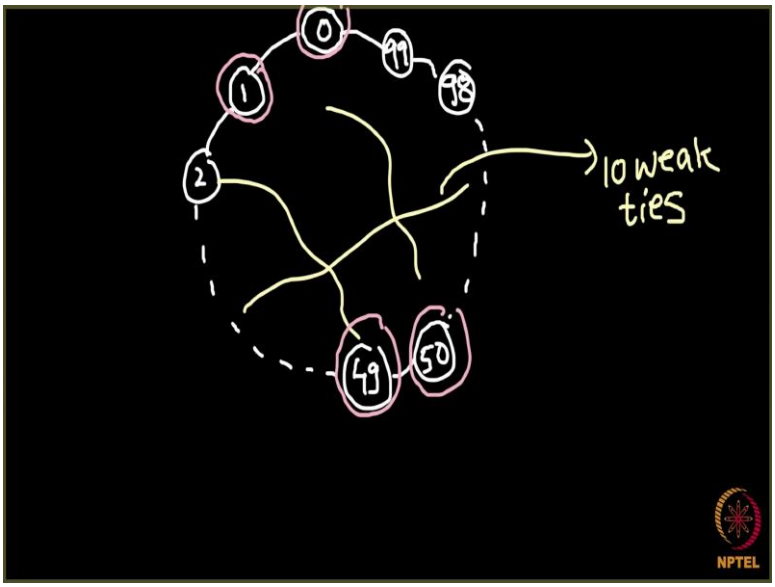
So, I am going to take a simple ring, a simple ring I mean 1 node on this ring for example, this node here is connected to only 1 node toward its left and 1 towards its right. So, I am going to take this simple ring and then what we are going to do is, we are going to perform 2 kind of search on this network. So, first of all by the side of these ties which are homophily base there will be weak ties something like this.

So, let me put 2 weak ties in this network. So, these yellow lines are the weak ties here and these are the homophily base ties. What we are going to do next is we are going to choose 2 nodes. So, I choose here 2 nodes let say this and I call this node as source and then I choose let say this node and then I call this node as destination. So, for going from

source to destination here we have 2 kind of searches which we have discussed before. So, we are going to implement those 2 kind of searches.

The first one is the myopic search and the second one is the optimal search; from our previous discussion we know that myopic search is not always optimal. In this particular case yes the myopic search is optimal. So, here myopic search will also give you the result we go from here and the, you moved here, and optimal search is also going to give you the same result. However, there can be cases where their results differ as we have seen an example previously. What we are going to do in this programming screencast is we are going to take a small world network one dimensional ring on 100 nodes.

(Refer Slide Time: 02:13)

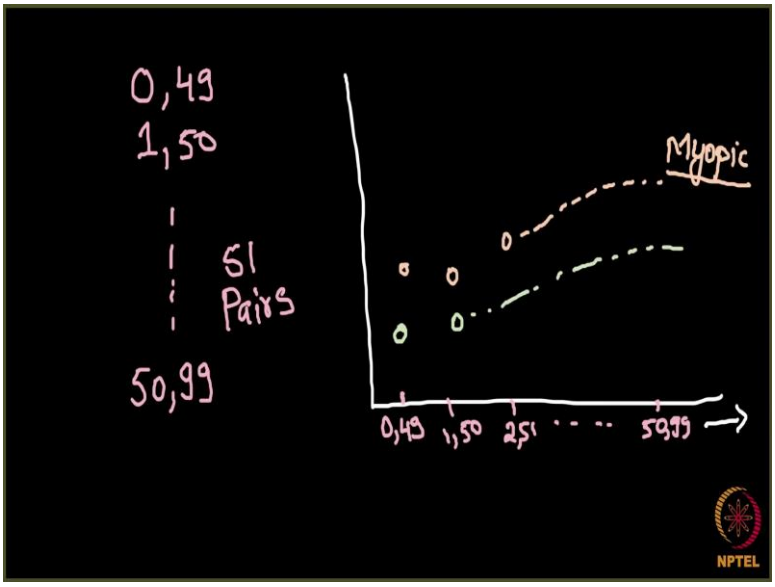


So, let say while label start from 0 so, 0 1 2 so on and something like 49 here and then 50 and then there comes 98 99. So, I am going to take this ring first of all. So, these are my homophily base ties and we are going to put across 10 weak ties here. So, I put 10 weak ties here 10, 10 long range context 10 weak ties. Next what is our aim? Our aim is to compare the myopic search with the optimal search for various different pairs.

So, which pair do you choose? So, we want many pairs for source and destination, and we are going to check for all these pairs; what is the difference between the path taken by the myopic search and the path taken by the optimal search. We think that the best pairs for doing this are the pairs which are diametrically opposite. Why? Because, along this

ring they are at the highest distance from each other; so, we can take 0 and 49 and then 1 and 50. So, these kind of pairs we are going to take. So, what we will be doing.

(Refer Slide Time: 03:31)

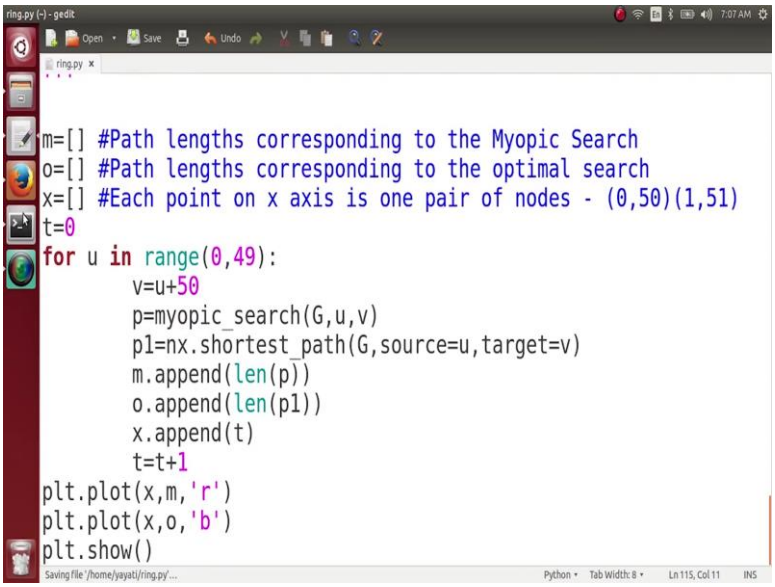


We will be taking the pairs 0 comma 49 1 comma 50 so on and so forth up to 50 comma 99. So, these are going to be I guess 51 pairs. So, we are going to take this 51 pairs and then what we are going to do is we are going to draw a plot. What is this plot? On the x axis here are my pairs. So, this point here can correspond to my pairs 0 comma 49, this point can be 1 comma 50, 2 comma 51 so on and so forth, 50 comma 99.

So, the x axis denotes these pairs and across the y axis we are going to draw 2 results. So, the first is corresponding to the myopic search. So, we will take this pairs 0 and 49 and, on this ring, we will calculate what will be the length of the path taken by myopic search. So, it be myopic search goes from 0 to 1, 1 to 2 and 2 to 49; in this case the length of the path will be 3 right. So, we will see what is the length of the path taken by myopic search and plot that some where here and then 1 comma 50, 2 comma 51 so on and so forth.

So, we are going to see what answer a myopic search gives us and we are also going to draw a plot corresponding to the optima search on the same plot. So, for 0 comma 0 comma 49, what was the path of the length given by optimal search for 1 comma 50 so on and so forth. And, we will be observing how this plot looks like in reality ok.

(Refer Slide Time: 05:17)



So, what I am going to do now is I will be taking 1 array. So, this array is for my path length for a corresponding to the myopic searching, path lengths corresponding to the myopic search. And, I will be having an array o and this array o is for the path lengths path lengths corresponding to the optimal search, corresponding to the optimal search. And, then I will be having an array x and this x array simply for the x axis.

And, as we have see previously each point on x axis is represent each point on x axis is nothing, but 1 pair of nodes and this 1 pair of nodes is something like it will be 0 and 49 or it will be 1 and 50 so on and so forth. So, it will be a pair of diametrically opposite nodes. So, that is my x axis, what I will do next is I will take a new array 0, sorry not array appoint t here; t is initially equal to 0.

And, next what I am going to do is the use of t array come to now soon, what I will do next is I know my first node here. So, we are going to change. So, previously we have what we have done was, we have looked at the distance from 0 to 40. Now, we have to looked the distance between all the diametrically opposite nodes.

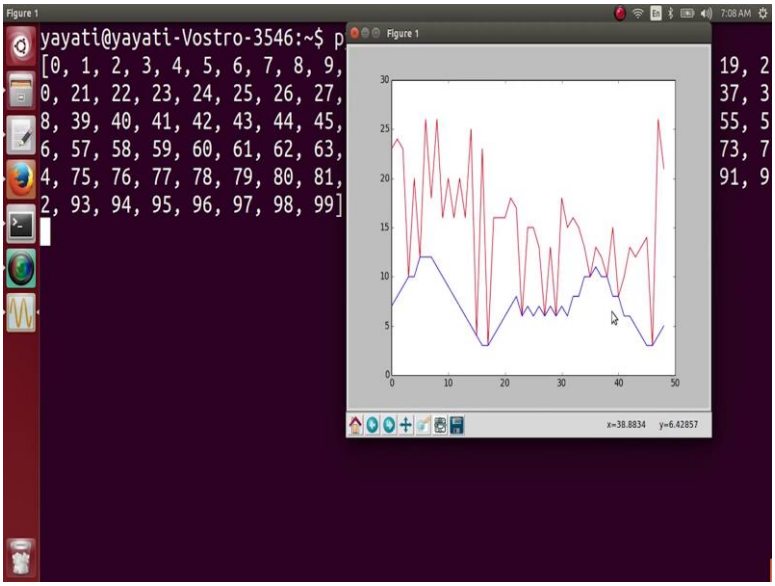
So, I will say my first node is u, for u in range and this range is on 0 to 49, 0 to 49 and for this, what I am going to do is my second which is a target node. So, u is the source node and target node is the diametrically opposite node and we can find it out by adding 50 to this u. So, how the pair will look like corresponding to 0 it will be 50. So, little bit changes here corresponding to 0 it will be 50, corresponding to 1 it will be 51 so on and

so forth. And, next what I will be doing is first I will perform a myopic search here, myopic search node from 0 to 40, but from u to v. And, next I am going to do an optimal search here which is again going to be from u to v. So, I am got the path according to my myopic search and I get path according to me optimal search and we do not need all these lines here; so, I delete them ok.

Next what I will do in my array m, which was corresponding to the myopic search I will append the length of the path which I got from myopic search. And, in my array o, I will append the length of the path which I got from my optimal search which is len(p1) and across the x axis I will simply append the t because, we need some numbers across the x axis and I increment this t by 1 and next what I have to do is at the simply plot this.

So, I will first I plot upload plt.plot I plot myopic search the path length corresponding to the myopic search on the x axis. So, x m and let say it should appear in red color. So, my myopic search of pl is in red color on this plot and plt.plot and then x. And, optimal the path length corresponding to the optimal search occurs in blue color. And, then I simply visualize this plot; let us run this code and see ring of py.

(Refer Slide Time: 09:31)



So, you can see here, here red colors are corresponding to the myopic search. So, there are we have taken 50 around 50 pairs of diametrically opposite nodes and we can always see that most of the times the myopic search takes quiet large number of steps as compared to the optimal algorithm.

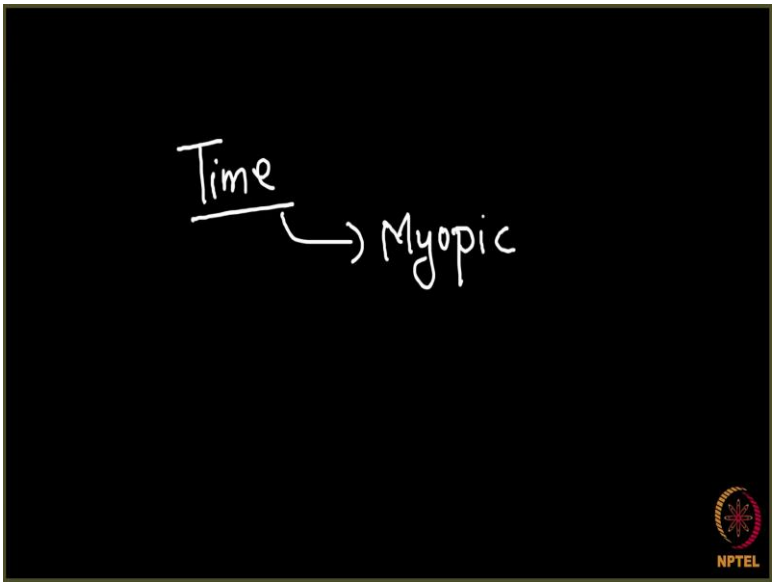
However, there are many points where myopic search actually performs optimally. So, out of these 50 points there are 11 points on which are myopic search it also performs optimally. So, this was the plot we wanted looked at.

Social Networks
Prof. S. R. S. Iyengar
Department of Computer Science
Indian Institute of Technology, Ropar

How to go Viral on Web
Lecture - 158
Time Taken by Myopic Search

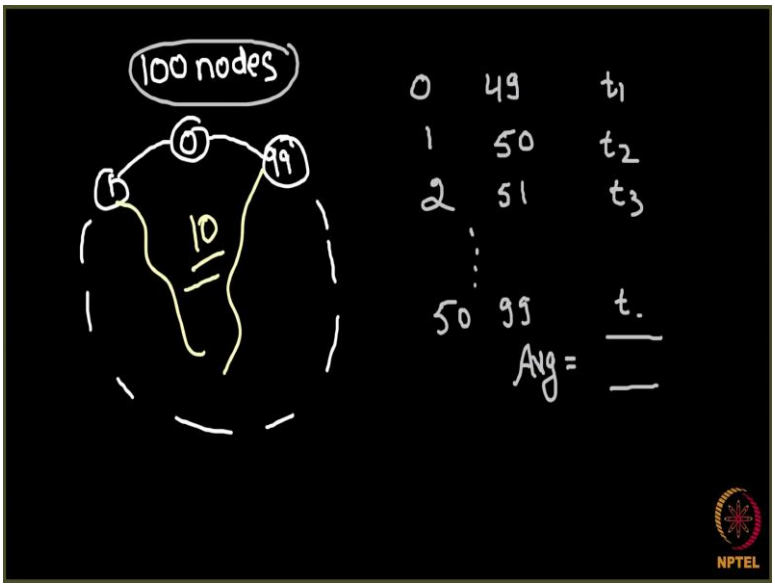
In the previous programming screencast, we compare the myopic search with optimal search.

(Refer Slide Time: 00:13)



Now next what we are interested in looking at is the Time Taken by the Myopic Search. So, if I perform my myopic search on a network of let us say100 nodes what is the time taken? On a network of 200 nodes what is the time taken so on and so forth; is this time linear in the number of nodes or logarithmic in the number of nodes or what. So, for doing this what we are going to do is first of all let us take a network having 100 nodes.

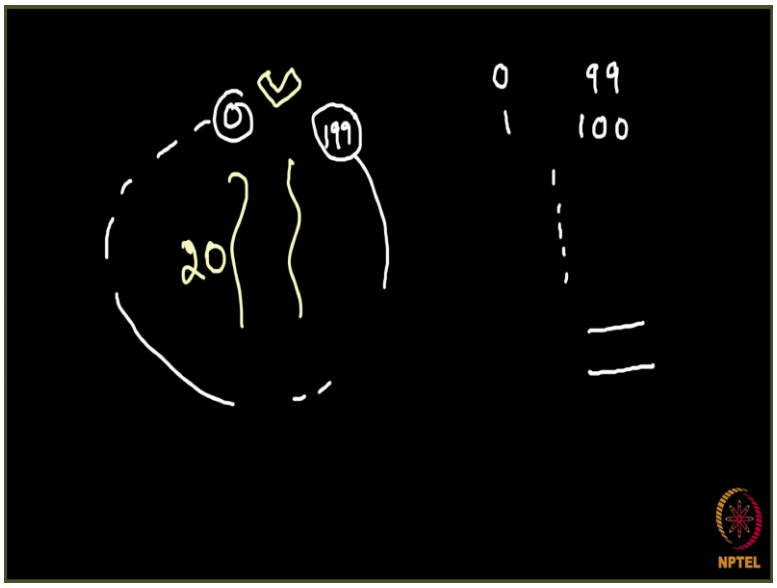
(Refer Slide Time: 00:41)



So, we take a network having 100 nodes as previous. So, we have nodes here 0 1 so on and so forth and here is 99 and we are going to do the same procedure what we did before. We will take the diametrically opposite points and look at how much time does myopic search take to work on this diametrically opposite points. So, we will be taking the point let us say 0 comma 49 and then we will see the time taken by the myopic search. Then for 1 comma 50 what is the time taken, for 2 comma 51 what is the time taken so on and so forth till 50 comma 99 what is the time taken.

Now, what will do next is we will take the average across all these times, that will give us on an average if we take 2 diametrically opposite points on this network; on an average what is the time taken by the myopic search. So, we calculate the average here. Now, this we have done for a network having 100 nodes, next what we will do we will repeat the same procedure on a network having 200 nodes. So, we will be taking a network having 200 nodes.

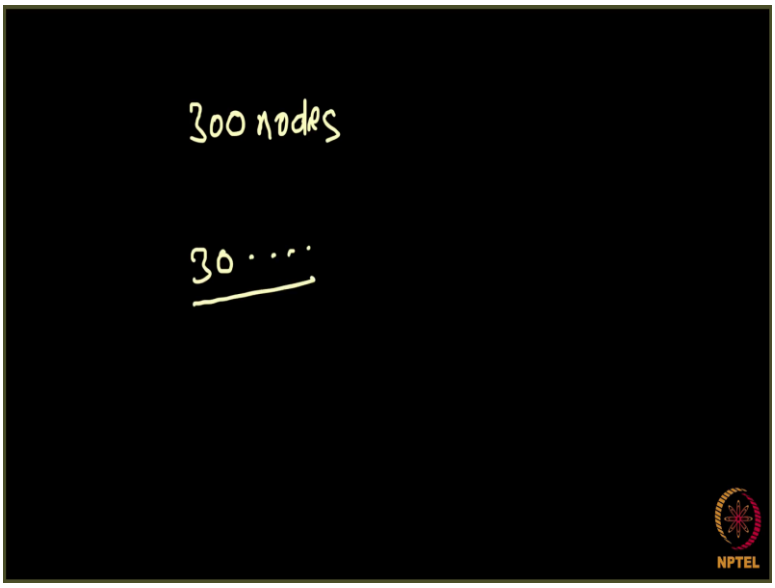
(Refer Slide Time: 01:57)



So, it will go from 0 to 199 and then we will again take the diametrically opposite points which here will be I will say 0 comma 99 and then 1 comma 100 so on and so forth. And, again we will take the average here and that will give us the time taken by myopic search on this network ok, while doing this we also take care of a small thing. So, when we have a ring a network over here right as shown you here the homophily based links and we know that there are going to be some weak ties as well.

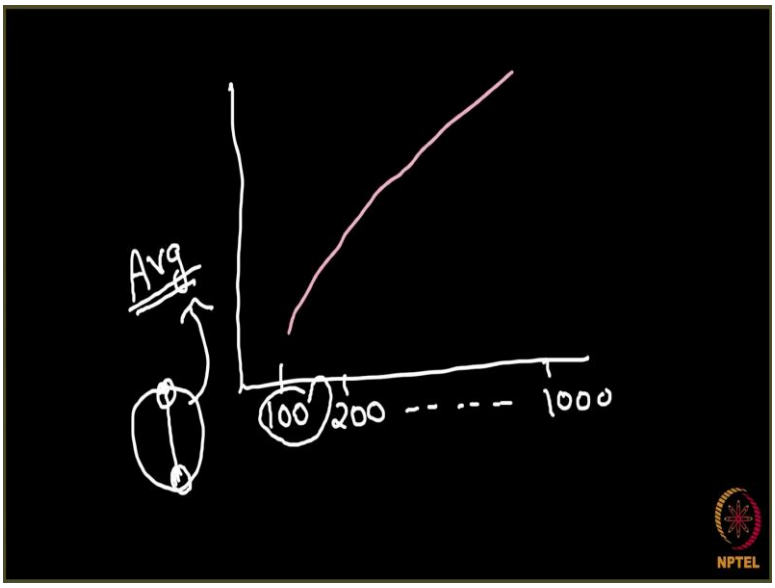
So, previously in this network we have taken 10 weak ties right, we have taken 10 weak ties. Now, if in this network is big right. So, what we are we will be going to assume in a programming screen cast is that the number of weak tie is 10 percent of the number of nodes in the network. So, for here we have 10 weak ties, here we will be making 20 weak ties in the next network which will be having 300 nodes.

(Refer Slide Time: 03:05)



We will be having 30 weak ties so on and so forth and then we will do this process. And, we will find the average over here and at the end what we will be plotting is, on the x axis we will be plotting the size of the network that is the number of nodes in the network 100 200 so on and so forth, let us say up to 1000.

(Refer Slide Time: 03:17)

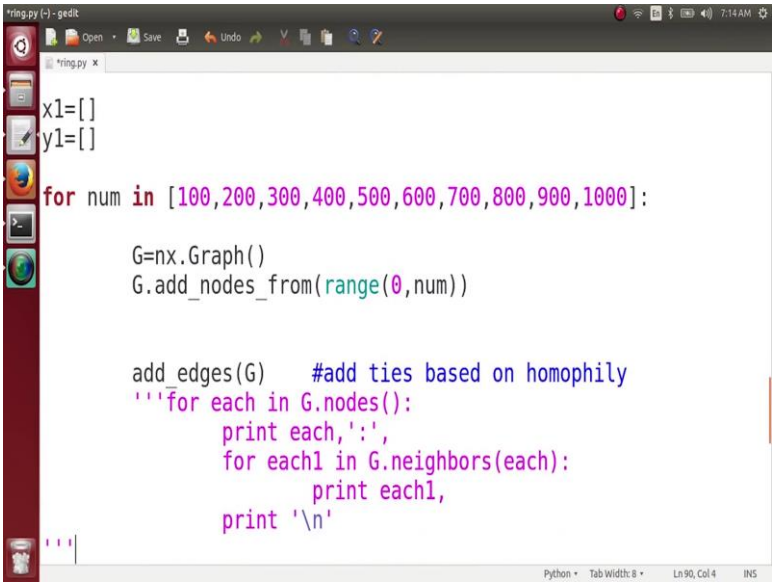


And on the y axis we will be plotting the average we have calculated. So, what is this average? It tells us that for one particular network, if we take the diametrically opposite points on this network. So, if we take this diametrically opposite points on this network

what is the average time taken by a myopic search and we will look at how does this plot look like is it linear, is it logarithmic or what. So, next what we want to do is now, we want to find out the time complexity of myopic search time which are myopic search takes for different size of networks.

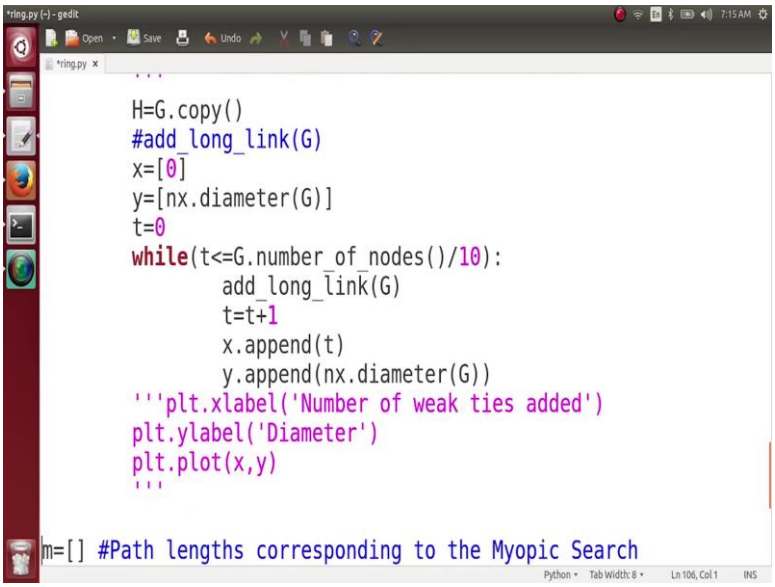
So, till now we have considered only1 network which was having 100 nodes. Now, we are going to do an experiment, we are going to build small world networks of different sizes 100 200 300 up to 1000 and we will do a myopic search there and we will see that how much time on an average does this take. So, at the end we want upload were on the x axis is the number of nodes and on the y axis is the average time taken by my myopic search. So, I make some arrays here.

(Refer Slide Time: 04:43)



So, I make an array x1 for the x axis which will be my number of nodes and an array y1 for the y axis which will be the time which my myopic search is checking. And what I have to do is now I have to create networks of different size and do this entire process for all these networks. So, I have to basically put all these code in a loop. So, what I am going to do is for num in and I am going to take a list here consisting of the values 100, 200, 300, 400, 500 6 up to 1000. And, then what I am going to do is the graph, I am going to create here is going to be from range 0 to num. So, first of all I will get a network on 100 nodes then 200 nodes so on and so forth and I will put everything inside this loop.

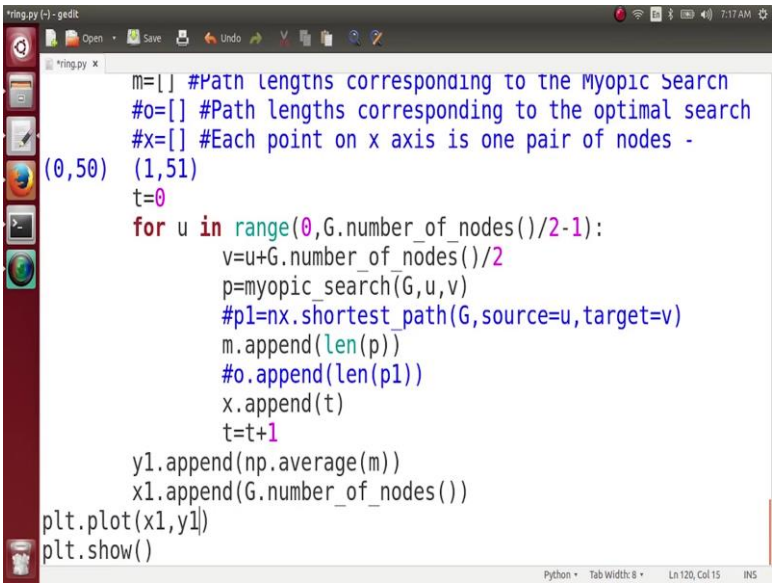
(Refer Slide Time: 05:49)



And, now we are one thing to be noticed is t here t here it is telling us the number of weak ties which we have to add. So, the number of weak ties should change according to the number of nodes in the network. So, instead of 10 what I am going to do here is, I am going to make it here while t <= G.number_of_nodes/10; that is I am going to have 10 percent of the edge is in the network to be weak ties.

Whatever are the number of a nodes in the network so, if there are 100 nodes so, the number of nodes so, if there are 100 nodes I will be having 10 weak ties; if there are 200 nodes I will be having 20 weak ties so on and so forth. And, then I am going to add these long range links and rest of the things remains the same.

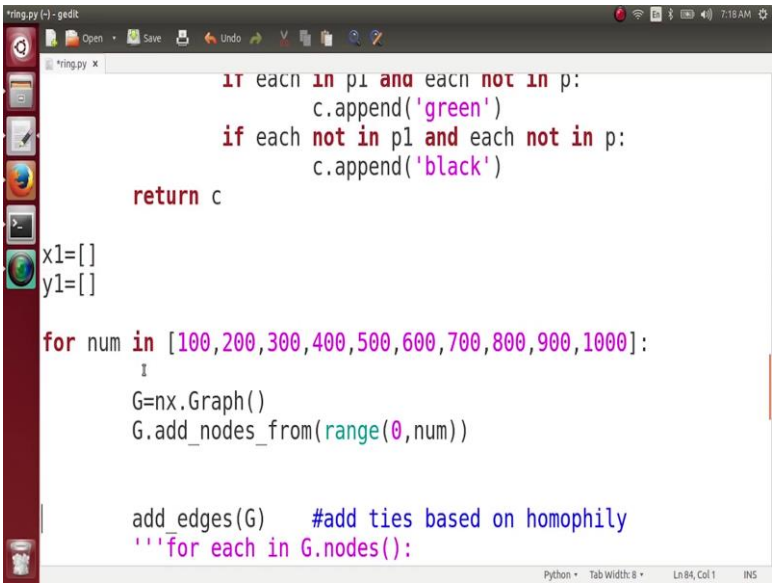
(Refer Slide Time: 06:59)



Now, mine this value u will range what is on 0 to 49, it will range from 0 to number_of_nodes/2 right. 0 to G.number_of_nodes/2 - 1 and v is going to be u plus G.number_of_nodes/2. And, then we are going to apply a myopic search here, find out the path we do not, no we are not doing an optimal search over here and then I am wrote append length of p and here is an array. So, so you can see that m is an array over here which holds the path length corresponding to the myopic search. So, you see what is happening over here, for some small world network here we are performing a myopic search. So, we have a small world network, we have done this myopic search.

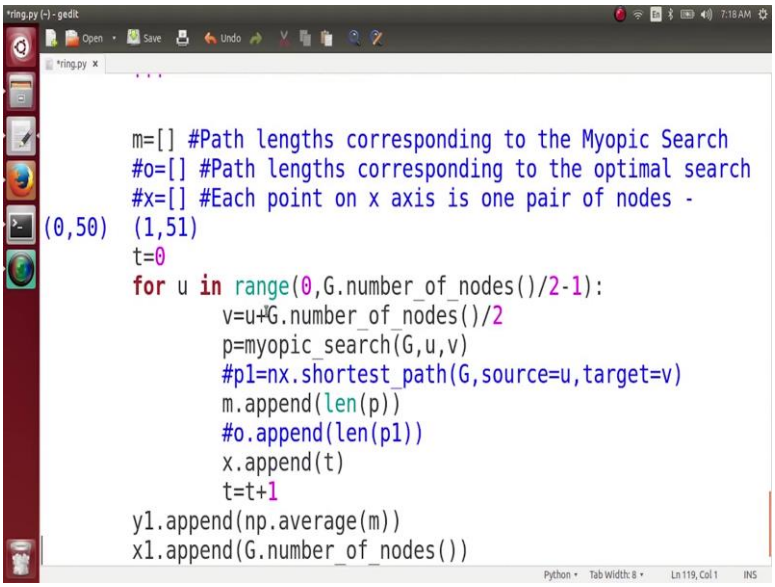
And we do it for many pairs of nodes and at the end what we want to append to our y axis is nothing, but the average of everything. So, what is going to come in our y axis is I will come out of this loop, what we are going to append in our y axis is the average which is np.average. And this is numpy, numpy.average of m right and what is going to be coming across x axis is nothing, but the number of nodes G.number_of_nodes. And at the end what will be, what we will be plotting is x1 against y1 y; I hope that this code is clear. What we have done is I will quickly recap it.

(Refer Slide Time: 09:39)



So, here are my 2 arrays x1 and y1, where x1 is the different. So, x1 is nothing, but thing here this complete will be x1 which is the number of nodes in different networks. And, y1 will hold the average path length which myopic search takes to connect a diametrically opposite points and then we make the small world network.

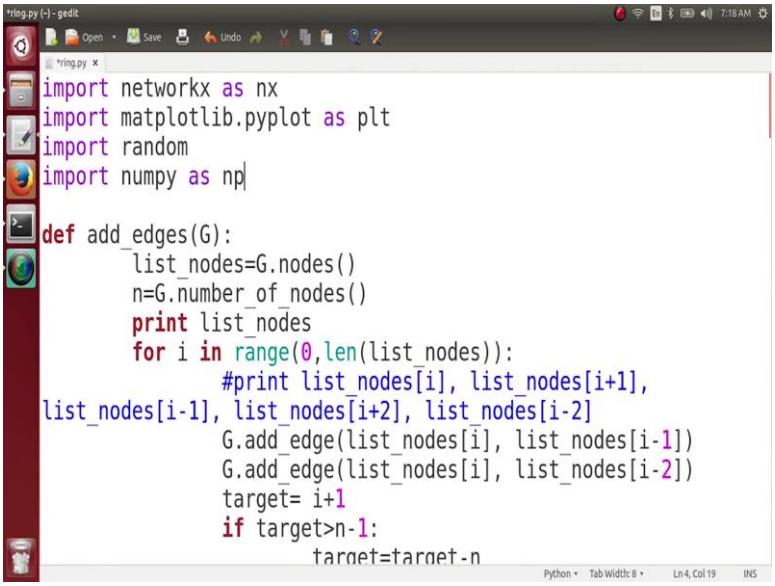
(Refer Slide Time: 10:05)



After making the small world network we take an array m be here and then we perform this myopic search for all the diametrically opposite points and keep a pending the path lengths in this array m. And at the end we take the average of this m and append it in y1

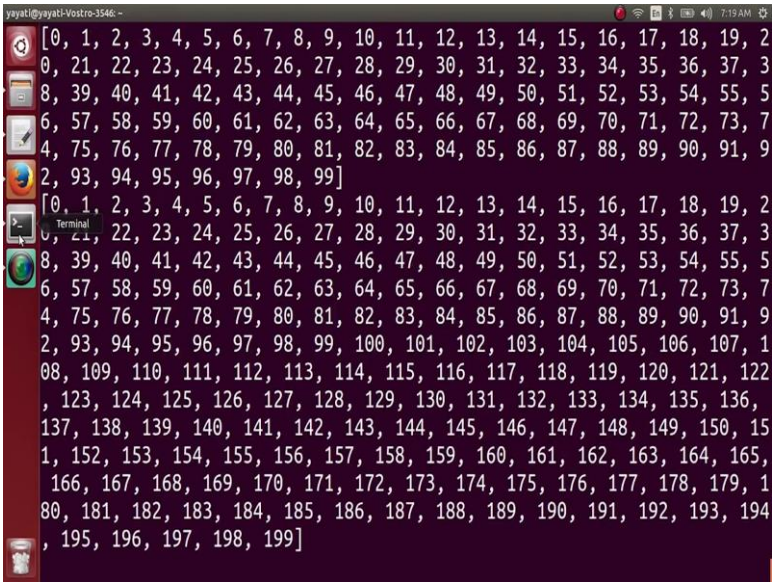
and this average of this m be append in y1 and x1 has nothing, but the number of nodes and we have using module numpy. So, that needs to be imported.

(Refer Slide Time: 10:35)



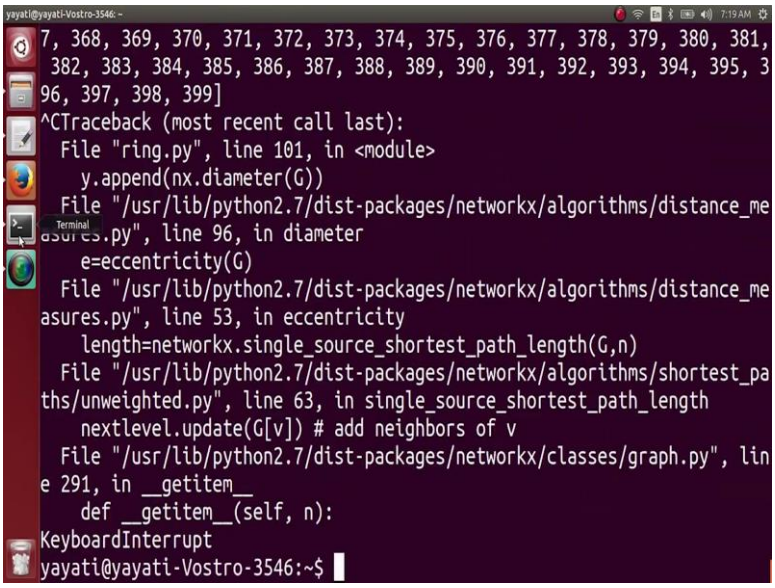
So, I import numpy as np means executes each.

(Refer Slide Time: 10:41)



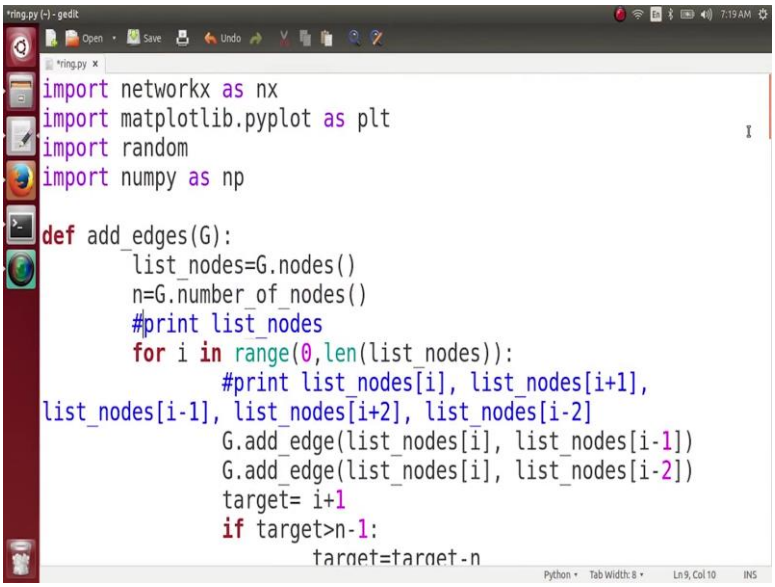
So, it will take some time.

(Refer Slide Time: 10:49)



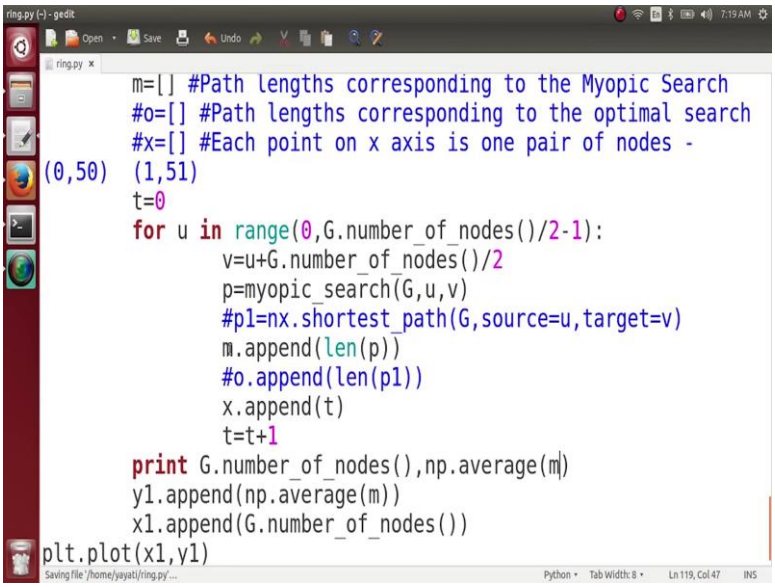
For the sake of clarity where there is code is running correctly or not, but we are going to do is where is my first print statement; we will remove that print.

(Refer Slide Time: 11:07)



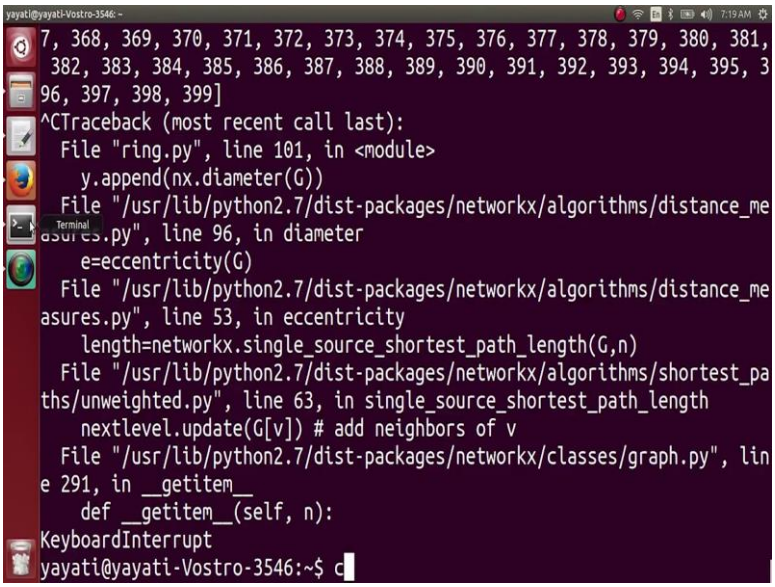
We do not print the list of nodes here, instead what we do here whatever we are appending we print here.

(Refer Slide Time: 11:15)



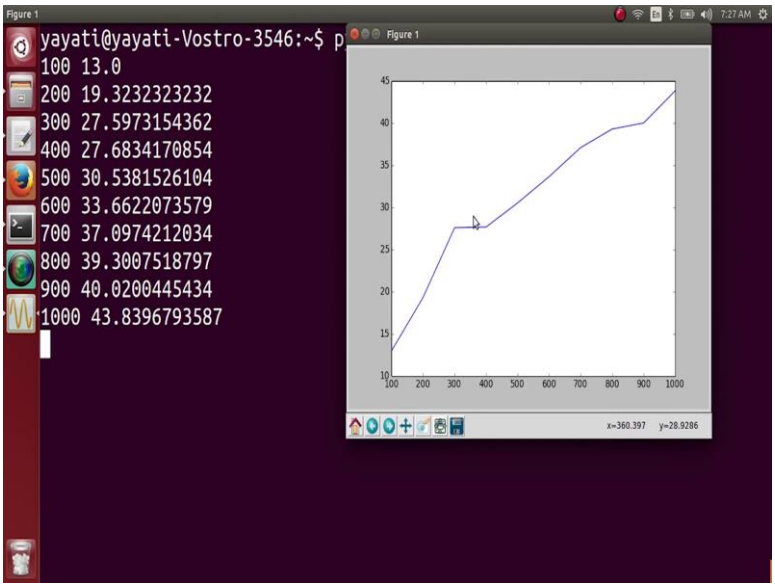
So, we print here whatever we have a appended in the x axis G.number_of_nodes and we print here np.average(m).

(Refer Slide Time: 11:29)



So, now.

(Refer Slide Time: 11:33)



So, when I have a network was of a was consisting of 100 nodes it took 13 steps 200 nodes 19 steps so on and so forth. It will be time to run and let us see the final output and you can actually do this process for a greater number of nodes. So, when you are code instead of 100 200 300, you can take 100 150 200 250 to get more data points and have a better plot ok. So, here you see plot and what if you will plot it for more data points you see the plot more clear and you can actually observe that this is not a linear plot rather it is a lower plot. So, as you increase the number of nodes, the time which myopic search takes to execute increase is logarithmically not linearly.

Social Networks
Prof. S. R. S. Iyengar
Department of Computer Science
Indian Institute of Technology, Ropar

How to Go Viral on Web
Lecture - 159
Pseudo Cores: Introduction

(Refer Slide Time: 00:07)



We have studied about cascades, we have seen how people follow each other and this following can be in terms of behavior, can be in terms of adopting an idea, can be in terms of adopting a product. And, we looked at an entire chapter based on this concept of following each other and cascading and we also talked about epidemics.

(Refer Slide Time: 00:29)



So, how do diseases spread from person to person and we have looked at various models for how we can model this like SIS model, SIR model etcetera. And, we have also talked about the subtle differences between the spread of an idea and the spread of a disease.

(Refer Slide Time: 00:47)



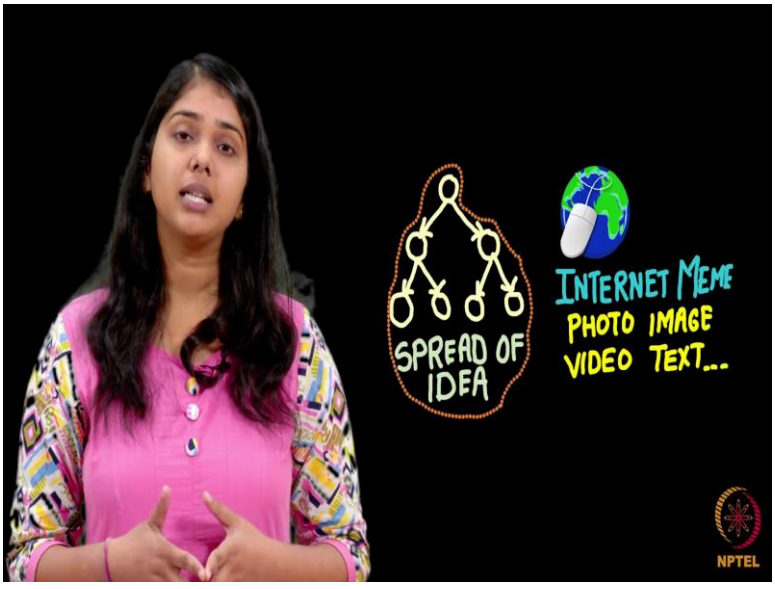
So, we have seen that when a disease spreads there is no choice, when an idea spreads there is a choice. So, if an idea comes to you, you make a choice if you do not like the idea, you have the liberty to say no.

(Refer Slide Time: 01:05)



I do not like this idea, but if it is a disease and somebody infected person comes to you and the contagion is happening, you cannot say I do not like this disease I am not going to take it. So, there you do not have any liberty. So, in this way the spread of a disease and the spread of an idea are different. Spread of an idea has become a complete research direction in its own; there are a lot of things we can study on the spreading of an idea.

(Refer Slide Time: 01:39)



And today ideas spreads like anything unlike the past days. Why? Because today all the ideas they spread on internet. So, whether it is an image or it is an video or it is an text or

let us say it is about a riot that people want to do. Everything today spreads with the help of internet through Facebook, through twitter, through WhatsApp and all these platforms. So, first of all I would like to tell you about a term internet meme, most of you might have heard about it also.

So, internet meme is actually not a technical word, it is not anything, it is just anything which spreads through internet is called an internet meme. It can be a photo an image or video or text or anything whatever spreads through internet is called an internet meme. And, today this internet memes they carry the capability of changing your life unlike the past days; if you want to become popular you do not need to find a right platform anywhere else, you can just upload a video on internet and become popular.

(Refer Slide Time: 02:53)



So, do you know the story of Justin Bieber? How Justin Bieber became Justin Bieber, the story is very interesting. So, initially in his when he was like 11 or 12 years old, he and his mom they used to record his videos. So, he used to sing at his school level or at his school levels and whenever he used to have a nice concert or nice singing they will record his video and they will post it on YouTube. And, when they post it on YouTube slowly the number of likes number of views started increasing.

And, then there was his producer or director which had his eyes on what is happening there. He looked there is a small, there is a talent a very small boy having this big talent and then he got and grab Justin Bieber and brought him to the singing industry. And,

today we do not also need a mentor like that to pull you out; you can mainly go and keep uploading your videos on internet. Your videos, pics or anything on internet and as it gains a greater number of likes you will become popular. So, we do not need anything else.

Social Networks
Prof. S. R. S. Iyengar
Department of Computer Science
Indian Institute of Technology, Ropar

How to go Viral on Web
Lecture - 160
How to be Viral

Although, I told you very easy way to become popular seems to be easy, do you think it is actually easy.

(Refer Slide Time: 00:21)



A population of 1 billion people everybody trying to appear on internet and getting popular, how easy is it for you to make your mark and go popular there; actually very difficult. There are lot of internet memes today and people have limited time. So, we cannot see all of those memes we just see the ones which may be our friends recommenders or which are really very very interesting and then go back to our work. So, in such a culture, in such a scenario, in such a situation it is actually very difficult to make your meme popular or rather we say it viral.

So, like diseases become viral some memes become viral and they are adopted by almost entire population. For example, Gangnam style meme is viral, Nyan cat meme is viral. So, how do you make your meme viral? How do you come to know that picture you updated on Facebook the last night whether it is going to become viral or not.

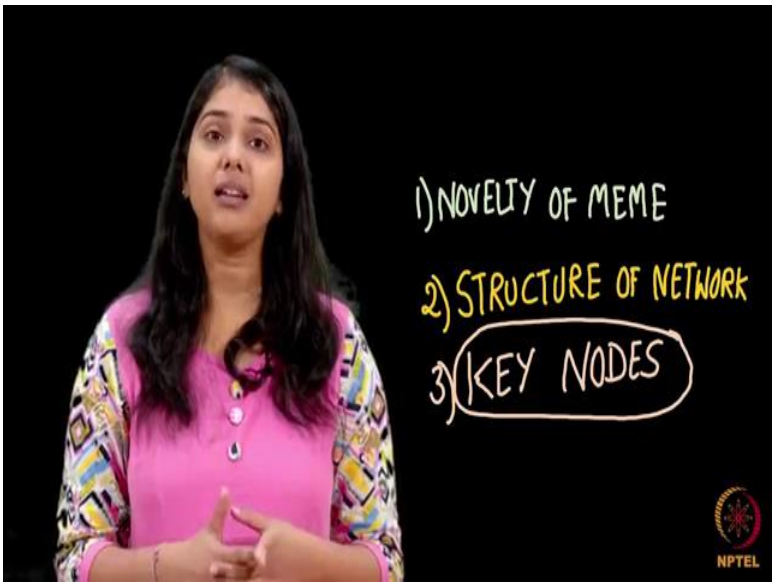
(Refer Slide Time: 01:23)



And, again we came back to the same question which we discussed in the chapter cascades, where we were concerned about the sport shoes my friends just now developed. So, he wants to know whether his sports shoes will become popular and will everybody buy them, or it will the cascade will quickly die out after some steps.

Now, how do we come to know that and like there the answer is again to 2 folds. So, one it is about the information you are sharing, it is about the quality of the sports shoes. So, it is about the quality of your meme. The novelty in your meme, do you have a nice background music or do you are you bringing something new to the platform; obviously, that matters what is inside your meme, what is inside your video or your picture your are releasing.

(Refer Slide Time: 02:17)



And, second, we looked at the structure of the network. So, some networks they are designed in such a way that they make everything popular and some networks they are designed in such a way that they do not let anything become popular.

But we cannot play here with the structure of the network structure of the network is same for everybody. So, the structure of the Facebook, structure of YouTube network, structure of Twitter network; structure means how the people in this network are connected. So, people are the nodes and the edges between these people that is the friendships and the followships are the ties. So, the structure is the same for every people and the structure is such that some of the memes become viral, some of the memes do not become viral. So, we cannot meddle much with the structure here. So, what do we do?

Do you remember the third idea which was particularly interesting, and the idea was about key nodes? So, if you want to get that cool bike it should be your dad whom you should be approaching not your spouse or your mother. In the similar way, if you want your information to become viral you probably make a key node, or a popular person talk about it. So, it is like if you convince Amitabh Bachchan or let us say Narendra Modi to re-tweet your post or to comment on your let us say your Facebook post no doubt it will become viral. No doubt more and more people will start liking it, will start sharing it, looking that somebody popular is liking your tweet.

So, that is the third way of doing; that is about choosing the right people for sharing your meme, infecting the right people with your meme.

Social Networks
Prof. S. R. S. Iyengar
Department of Computer Science
Indian Institute of Technology, Ropar

How to Go Viral on Web
Lecture - 161
Who are the right key nodes?

So, we now know that these should be the key people which we should be infecting, but how do we identify this key people. Who are these key people we should infect; assume I give you a network as shown here?

(Refer Slide Time: 00:19)



Can you look at this network and tell me which is the node here, which I should infect; so, that this meme which is travelling on this network which is rather diffusing on this network becomes viral, which should be the node here? And so, there are many possible criteria which you can look at, is it the node which is having maximum number of friends, having the highest degree we say. Should it be that node which we should look at and attack or should it be a node which is structurally closer to all the nodes in the network. Closer in the sense, if you look at its distance from every other node in this network the distance is small.

So, it can quickly reach all the nodes in this network that is actually called close the centrality and we will be discussed in another lecture. But, just for the intuition it just

says that this node is closer to all other nodes in the network or should it be a node having high betweenness. Betweenness we have looked at before, tells you how good you are in connecting people.

So, if you look at this node, here if we have these two components and there is this node in between. So, it has a high betweenness because, for going from this component to the other component you need to go through this node. So, should it be this node which I should be infecting or there should be some other criteria. So, how shall I find such a node in a given network; so, that if I infected with an idea or with my internet meme perhaps my meme should become viral.

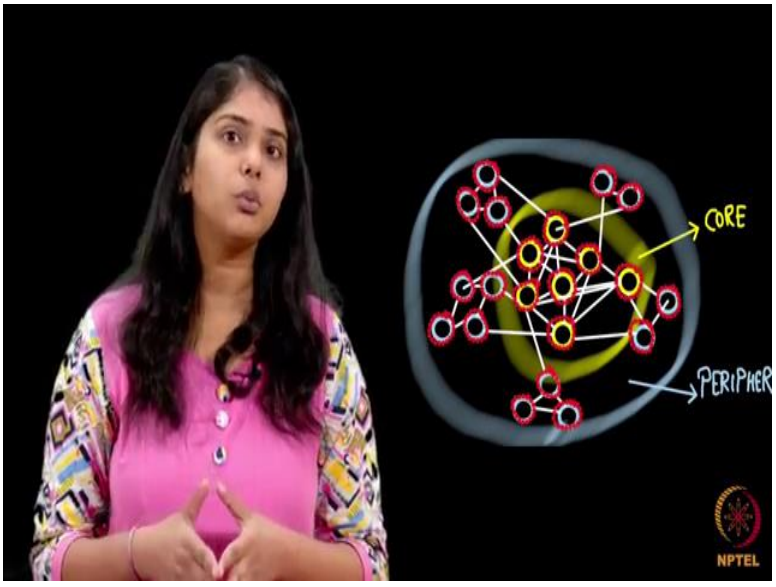
Social Networks
Prof. S. R. S. Iyengar
Department of Computer Science
Indian Institute of Technology, Ropar

How to go Viral on Web
Lecture - 162
Finding the right key nodes (the core)

So, here I tell you the answer how should you choose these key people. Do you remember we talked about this structure in one of our chapters in a small world phenomenon? We talked about this structure and it is called a core periphery structure.

So, we have looked that there are certain people in this world some small fraction of people who are rich, who have access to better resources, who can travel throughout the world and such people can manage more connections and rather connections to different-different parts of the world and they lie at the centre of a network something like this.

(Refer Slide Time: 00:45)

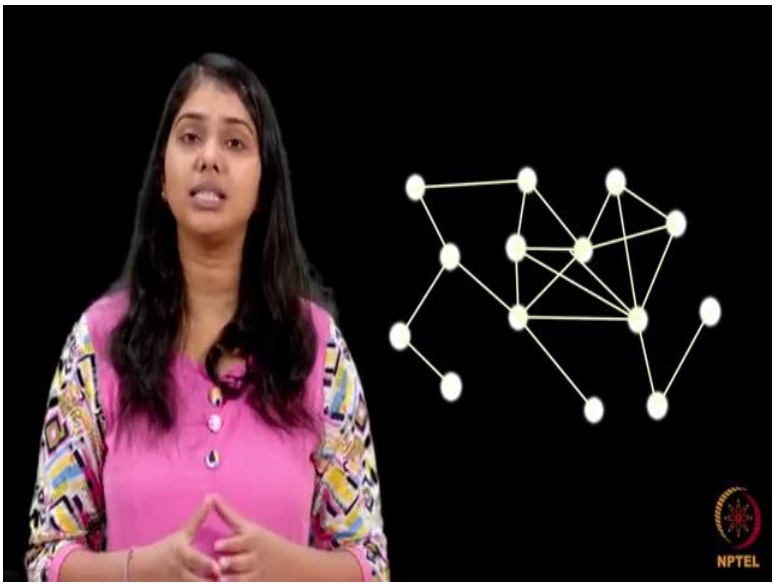


And rest of the people like a periphery are towards the periphery of this network they are towards the boundary. So, we talked about this structure before. This structure has a very big role to play here. So, it has been shown that if you infect one of these core nodes here you see what happens. So, if you closely look at this figure, you will see that these core nodes here they are very densely connected to the other core nodes.

So, if you infect one of these core nodes, it will quickly tell another core node, which will quickly tell another core node and soon this entire core will come to know of your idea. And we know that this entire core is now very nicely connected to all the people in the world to all the periphery. So, once this component is infected, your mean will go and reach all the periphery nodes as well. So, this should be the core nodes which you should be infecting ok.

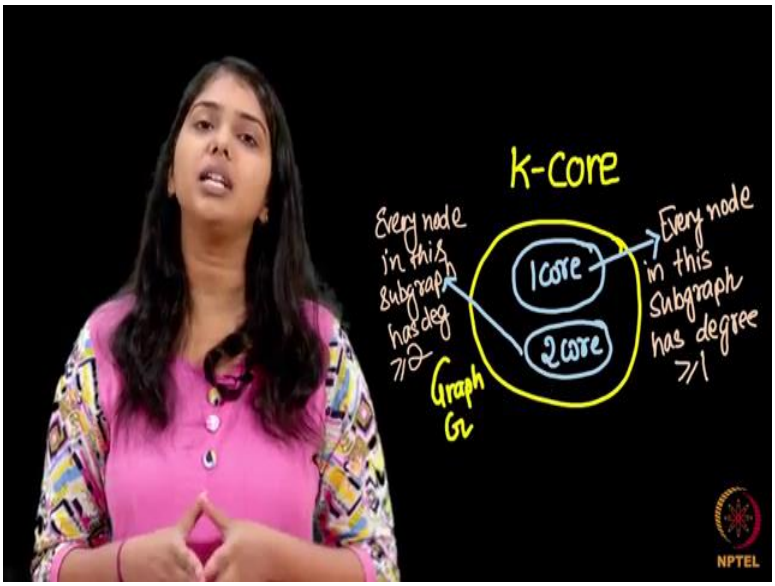
Till now, well and fine we come to know, we talk about the core periphery structure and we say that if this is the core which is infect my mean is going to become viral. Now I give you a network here can you tell me what is the core here?

(Refer Slide Time: 02:05)



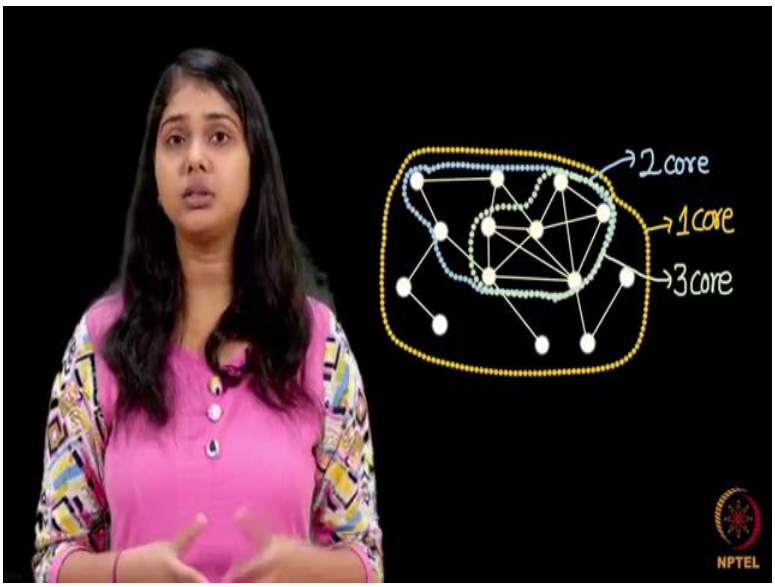
How do we even come to know what is core of a network, what lies in the core of a network? And then people have tried answering this question. And I will now discuss one of the most popular approach which you can use to identify these score nodes, identify these score nodes in this network. So, for that we will just switch to a definition a mathematical definition, but very simple and the definition is that of a K core.

(Refer Slide Time: 02:37)



So, given a graph let us say we are given a graph here, we say a node or we say not a node we take a sub graph from this graph and we say that the sub graph is 1 core. 1 core if every single node in the sub graph has the degree of at least 1. And similarly, you say a sub graph to be off 2 coress to be 2 cores if you look at every single node in this sub graph and every node has a degree of at least 2. So, for and so forth, so, you have 1 core, 2 cores, 3 cores, 4 core and so on. So, that is what defines a K core. You take a sub graph and every node in this sub graph should have a degree at least K ok.

(Refer Slide Time: 03:29)



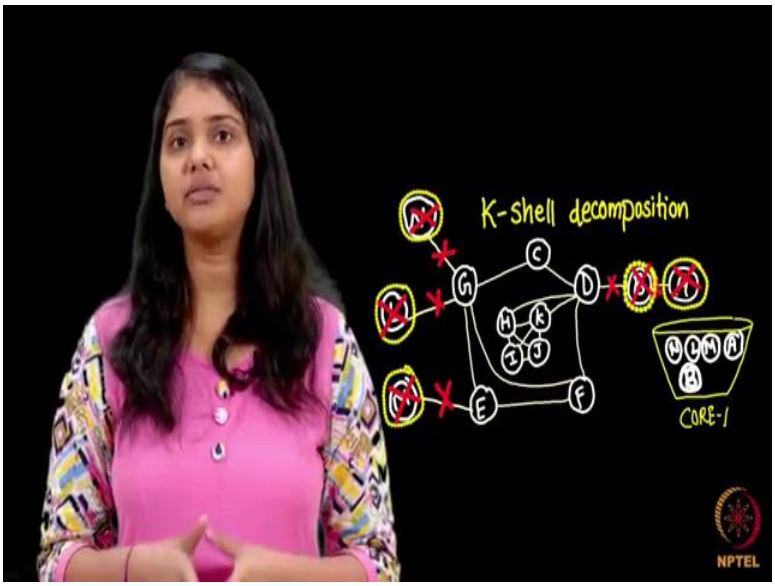
So, here is a graph. So, we can quickly see and try to figure out what is 1 core here, what is 2 cores, what is 3 coress. So, if we look at it and introspect it introspect on it a little bit we come to know that yes fine this is 1 core, this is 2 cores and this is 3 cores, but now what I am going to do? This is a small network. It was easy for you to look at it and then tell me what 1 core is, what is 2 cores, what is 3 coress.

(Refer Slide Time: 04:03)



I will give you a network having billions of nodes like Facebook or Twitter, I give you the network here and now can you tell me what is 1 core here, what is 2 cores, what is 3 cores and so on. It becomes very difficult. So, there is a very nice algorithm to do it. It is actually intuitive if you little bit think about it, you can converge to this algorithm on your own. And the algorithm is very interesting as well So, this algorithm determines what goes in 1 core, what goes in 2 cores, what goes in 3 cores and so on and it is known as K shell decomposition algorithm.

(Refer Slide Time: 04:39)



K shell decomposition algorithm because, it decomposes your network into multiple shells. Now, let us see how does this algorithm work. So, you look at your network. So, here is your network. And let us say your network is some instead of nodes I use the word ball here. So, your network has many balls and these balls are connected to each other.

Then just for some time I am using the word ball instead of the word node. And then I come with a bucket and I label this bucket as 1 core and put this bucket here. And then I look at this network, I look at all the balls, all the nodes which are having degree 1; which means which have only 1 friend, we also call them pendant vertices if you know graph theory otherwise you can just leave it.

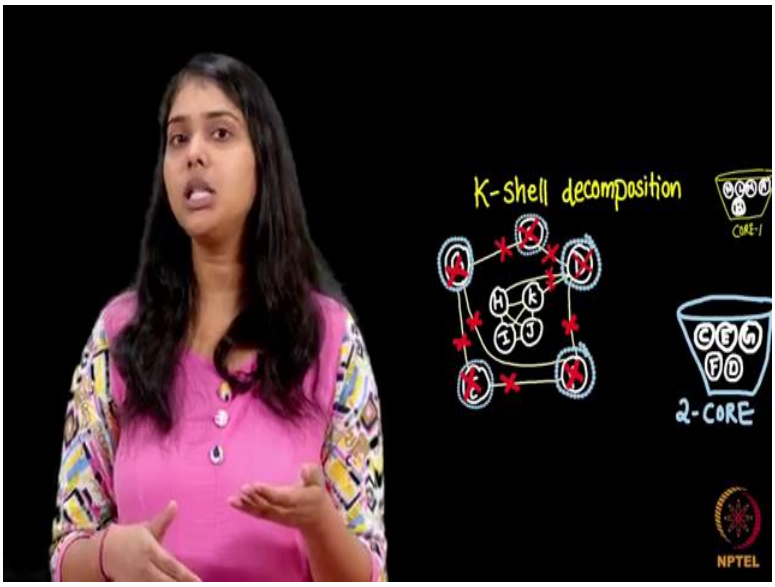
So, the just have degree 1 they are kind of attached to the network with just 1 edge. I take these nodes I plug them out and put them in this bucket. So, when I plug a node out please note that all the edges which are outgoing all the edges which are incident on this node also gets plugged out.

So, when I plug out a node, I cannot have a hanging edge in this network right. So, that edge is remove and I put that ball in this bucket. And then I keep doing it, I look at another ball which is having degree 1, put it in the bucket, I keep doing with all the balls and I look at all the balls having degree 1 and put it in the bucket.

Did you notice something here? When I pull a ball having degree 1 another ball which was previously having degree 2 can now become a ball having degree 1. So, that is when you keep removing these balls with degree 1, new balls keep immerging in your network which has degree 1. So, if you look at this node A here, when I plug this node A out, keep it here, you see this node B, it has now become of degree 1.

So, what do you do? You also remove node B and put it in the basket. So, mainly you keep doing this business of plugging out the nodes and keeping them in the basket till there is no degree 1 ball in your network. Till all the nodes here they have degree 2 or a higher degree. And once you achieve such a graph which has nodes of degree 2 or higher degree you stop and you put your bucket aside then you bring a fresh bucket and then we label this bucket as 2 cores.

(Refer Slide Time: 07:21)



And you now can guess what I am going to do is. So, I put this bucket having label 2 cores here and I again locate all these balls in my network. And now I look at the balls which have degree which have degree 2 that is with 2 edges they are connected to this network.

So, for example, the node C here, you can see that it is connected with degree 2 to this network with 2 edges. So, I plug C out and keep it in the basket. I do it for all the balls which are having degree 2. I take all those balls and put those balls in this basket. And again did you notice the same thing happens when you plug a ball out which is having a

degree 2, a new ball having degree 2 might immerge and rather a new ball having degree 1 might immerge.

So, you see here when I remove this ball having this when I remove this ball E from this network having degree 2, this node F immerge is which has just degree 1. What I do, what should I do now with this degree one node? Shall I keep it in my previous bucket or shall I keep it in this bucket? So, I will tell you I keep it in this bucket. This market means the bucket label 2 cores. So, I keep removing the balls having degree 2 or lesser. So, in this iteration what all goes in this bucket 2 all the nodes having degree 2 or lesser.

(Refer Slide Time: 09:09)



And you keep plugging them out and new balls keep immerging which might have a degree 2 or lesser you plug those balls also out and put them in this same bucket and you keep doing; so till there are no nodes of degree 2 or lesser in the network. So now, in this network there will only be nodes having degree 3 or a higher degree.

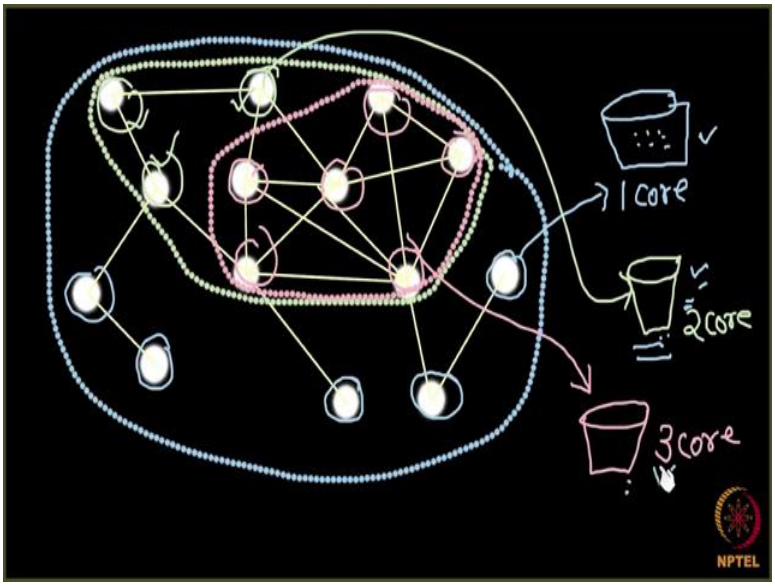
And when this is done, I pack this bucket and keep it aside and I bring the third bucket in picture, I label it as 3 cores and I start repeating the same procedure again. Plug all the nodes having degree 3 from this network and new balls might immerge which might be having a degree 3 or lesser and I keep all of them in the same bucket and I keep doing it. And we keep doing this procedure till my network becomes empty.

So, we are plugging nodes at every step. So, when we keep plucking nodes obviously one state should come when my network becomes empty and that might be at some K core at some K-th bucket, we do not know it can be a 10th bucket or it can be the 9th bucket.

Now, we have K cores. What we talk previously was about core and periphery; we were interested in finding the core of the network. So, what is the core of this network? So, when we do this K shell decomposition to find 1 core, 2 cores and 3 cores and so on. The nodes which are removed at the last step the most let me see the innermost core of the network, innermost shell of the network that is called the core.

So, in this network where we were having 1 core, 2 cores, 3 cores, the core of this network is this the 3 cores. And when we do k shell decomposition whatever goes in the last bucket, whichever nodes go in the last bucket form the core of that network. I want to tell something important related to K core, 2 cores and K shell decomposition.

(Refer Slide Time: 11:07)



So, we have looked at this network and, in this network,, we have looked at which node comprise of which nodes coming 1 core, which nodes coming 2 cores and which nodes coming 3 cores. So, we have seen that in this network the sub graph in which all the nodes have degree greater than or equals to 1 come in come in 1 core.

So, this entire network used to come in 1 core. So, this is the 1 core of this network. And then we have looked at 2 cores in which all the nodes have degree greater than or equals

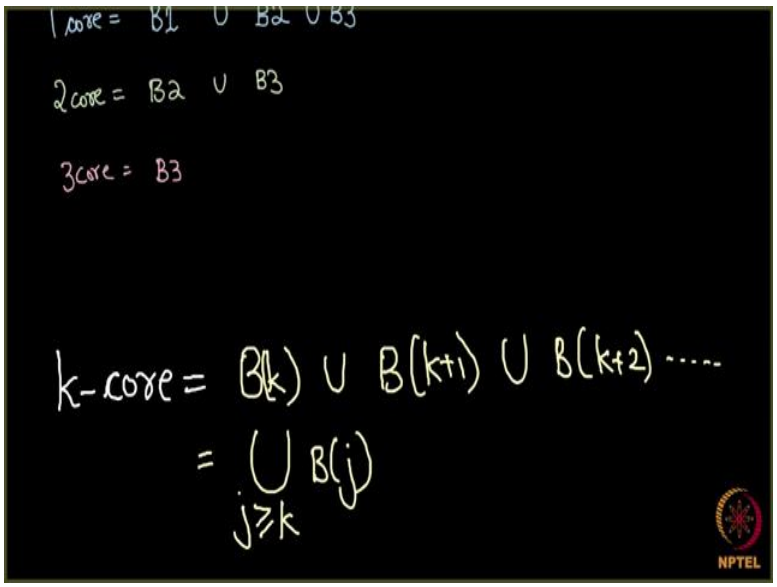
to 2. So, this particular portion in this network is the 2 cores. And then similarly we have looked at that this particular portion in this network is the 3 coress. And then next we will discuss the techniques to determine the 1 core, 2 cores and 3 coress in the network which was called K shell decomposition. Now what is important here is if you remember that in K shell decomposition, we made several buckets right. So, there was a bucket corresponding to 1 core.

So, let us say this was a bucket corresponding to 1 core and then we had a bucket corresponding to 2 cores and 3 coress also. So, in this bucket corresponding to 1 core, which all were the nodes in K shell decomposition and if you will apply K shell decomposition over this network, you will note that these nodes over here were the ones which were present in this bucket corresponding to 1 core.

Similarly, if we look at the bucket corresponding to 2 cores then we saw that these nodes over here 1, 2, 3 1, 2, 3, so these 3 nodes were in bucket of 2 cores. And then similarly we had a bucket of 3 cores over here So, this is a bucket for 3 cores and all the nodes, so 1, 2, 3, 4, 5 and 6, the 6 nodes over here were in the bucket for 3 cores So, we had these 3 buckets.

Now, what is important here is please note that all the nodes which are present over here in the bucket for 1 core. So, if you are asked over this network what is the 1 core of this network?

(Refer Slide Time: 13:31)



So, 1 core of this network is according to K shell decomposition 1 core of this network is the nodes which are present in bucket corresponding to 1 core. Let us say that bucket 1 union bucket corresponding to bucket corresponding to 2 cores which is B 2 union bucket corresponding to 3 cores which is B 3 right.

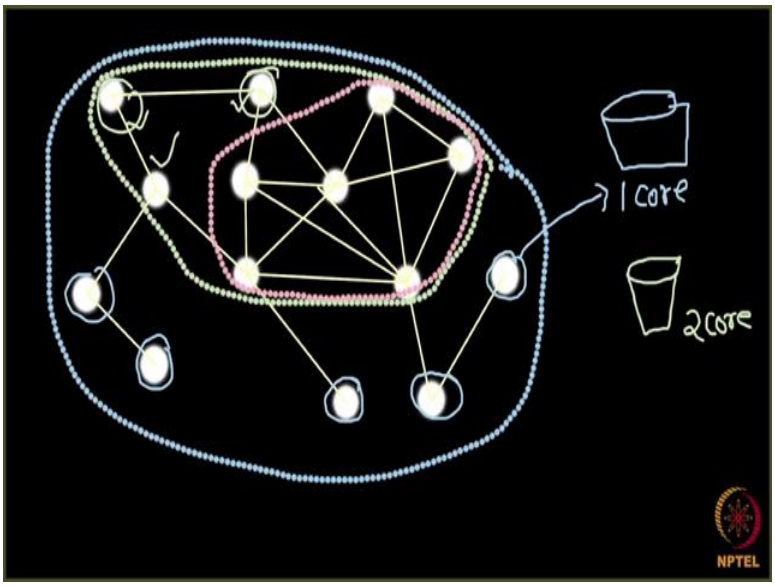
So, in this network if you see over here what is 1 core, so the 1 core in this network is all the nodes over here, union all the nodes over here, union all the nodes over here because, whatever is in this bucket for 2 cores is definitely is trivially present in the bucket for 1 core. So, here we have all the nodes which have degree greater than or equals to 2, so automatically this will be having degree greater than or equals to 1.

So, whatever is in this bucket is automatically in 1 core, whatever is there in this market is automatically in 1 core. Similarly, whatever is there in this bucket is automatically in 2 cores. So, the 2 cores of the network is, so if I want to write 2 cores in this network is so 2 cores is bucket 2 union bucket 3.

And if I want to write 3 cores in this network is 3 cores is my simply bucket 3. If you want to generalize this formula and write; if you want to generalize the formula and write it down we can say that when we try to determine the K core in a network with the help of K shell decomposition, we can say that K core of the network is, what is K core of the network is? It is bucket k, let us write it like this, bucket k union the nodes in bucket k plus 1 union the nodes in bucket k + 2 so on and so forth; that it is the $\bigcup_{j >= k} B(j)$.

So, that is how would how you can determine the K core in network with the help of K shell decomposition.

(Refer Slide Time: 15:41)

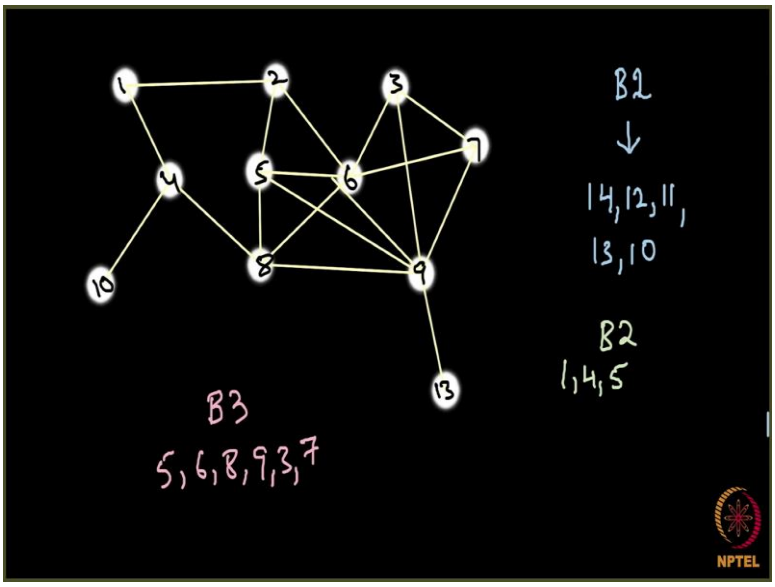


So, the overall motive of this video was to tell you that the buckets here do not correspond to the actual core numbers here; so, if you actual K core here. So, if you want to determine 1 core in this network, you cannot simply take the nodes which are present in bucket 1, you have to union it with the nodes in the bucket 2 and bucket 3 so on and so forth till the maximum bucket.

Social Networks
Prof. S. R. S. Iyengar
Department of Computer Science
Indian Institute of Technology, Ropar

How to go Viral on Web
Lecture - 163
Coding K-Shell Decomposition

(Refer Slide Time: 00:05)



In the next programming screen cast we are going to implement facial Decomposition. So, I hope you remember you remember what was the algorithm for facial decomposition. First of all, we take bucket 1 and then after taking bucket 1 we start removing the nodes having degree 1.

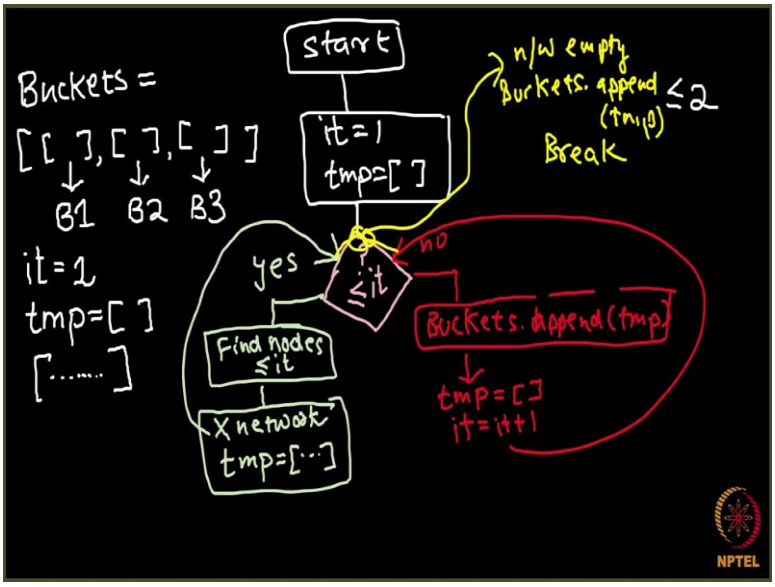
So, we see that this node over here these nodes have degree 1, this node has degree 2. So, these 3 nodes have degree 1, so what we are going to do, we are going to remove these 3 nodes and put them in bucket 1. So, let me number these nodes also. So, what I am going to do is let me number these nodes first ok. So, we are going to take bucket 1 and in bucket 1 will recursively put the nodes who have degree 1.

So, first of all nodes 14, 12 and 11 go to bucket 1 because, they have degree 1 and then we delete these nodes from this network. So, we cut node 14, 12 and 11 from this network and we put them. And next again we see that there is degree 1 in this network and then we recursively remove these nodes also from the network and put them in

bucket 1; similarly, we determine bucket 2 and bucket 3 in this network. So, bucket 2 will comprise of the nodes 1, 4 and 5 and bucket 3 will comprise of the nodes 5, 6, 8, 9, 3 and 7.

So, how do we write a code for this? So, before writing the code here I am going to tell you the structure of the code, so that it will be easy for you. Decode is a little bit involved, so it is important to understand its structure before writing the piece of code ok. So, let us take this network and we do not even need this network ok.

(Refer Slide Time: 02:15)



So, what we will be doing is let us say that here a good start. So, this is starting. What we are going to do is we know that we have to recursively remove the nodes having degree 1 or less than 1 till there are no degree 1 nodes in the network. Next, we have to recursively remove the nodes having degree less than or equal to 2 from the network till there are no nodes having degree less than or equal to 2 in the network so on and so forth.

So, in the beginning I am going to take some variables. So, I have a first of all I will take a list here and I name this list as buckets. What is this list bucket? This list buckets is going to be a lists of lists. So, it will be further consisting of many lists and this will be the list corresponding to my bucket 1. This will be my list corresponding to bucket 2, bucket 1 here means bucket corresponding to pour 1, bucket corresponding to pour 2, and bucket corresponding to pour 3.

So, this is my array buckets and the aim of our code is to fill this array buckets. And then I am going to take a variable let us say it which is initially 1. What this variable tells me? This variable tells me at a particular time in a code which degree nodes I am removing. So, if the value of it is k let us say it means that I am removing the nodes having degree less than or equals to k.

So, that is my variable it. And then we have a array or we have a list tmp. What this list tmp is going to hold is, so we are going to determine 1 of bucket at a time. So, I might be filling this bucket at a particular time, I might be filling this bucket or this bucket. So, t is going to temporarily hold this bucket. So, initially the value of tmp is going to be initially, so tmp will start with empty and then whatever values will put in tmp will be the values in B1.

So, we will fill tmp, we will keep filling tmp and a point will come where all the nodes have been degree less than or equals to 1 will be removed from this network. At that time we are going to close tmp and we are saying that we will say that this tmp is nothing but bucket 1 and we will append tmp here, it will be further clear ok.

So, what we are going to do is we have a start here. After start we are going to check a condition. So, initially we have it equals to 1 and tmp is nothing, but empty ok. Next what we are going to do? We are going to check a condition and what is this condition? This condition is whether there are nodes of degree greater than sorry, whether there are nodes of degree less than or equal to it; in this network whether, there are such nodes in the network and the answer can be yes or no.

So, let us say here the answer is yes and here the answer is no. So, if the answer is yes what we are going to do? If the answer is yes it means that, still there are nodes of degree less than or equals to it in this network. So, we have to prune those nodes from the network. So, what we will be doing is what we will do is find those nodes.

So, we find these nodes which have the degree less than or equals to it and what do we do? We remove these nodes from our network. So, we remove these nodes from our network and we append these nodes in our array tmp, in our list tmp which is the currently being filled bucket.

So, we append these nodes in tmp. That is what we do and after doing this we return back to here. So, when the value of it was 1 here what do you do, you find the nodes which have degree less than or equals to 1, you delete those nodes from the network and put them in your bucket and after that you again see whether you; I am very sorry, you come here you come here ok.

So after that, after removing these nodes having degree 1, you again see whether still there are nodes having degree less than or equal to 1 in your network and again if the answer is yes you remove those nodes also and you keep doing this till there are no nodes having degree less than or equals to 1 in the network. So, if the answer is no what will you do? It says that your bucket is over.

The bucket which you are currently filling is over, so what you do? At this point if the answer is no you append your tmp to your buckets. So, buckets.append(tmp) that is your current bucket is full, so you do this and what else you do. Now we have to start filling a new bucket. So, we set back tmp to empty and also, we have removed all the nodes having degree it. Now we have to start removing the nodes having degree it plus 1. And after doing this we go back to these things.

So, I hope the code structure is clear now. So, this loop will keep running. So, it will check for the value of it equals less than or equal it will be 1. So, we check whether there are nodes of degree less than or equals to 1. We keep doing this, we keep coming back, keep removing nodes having degree less than or equals to 1 till there are no such nodes in the network. And when there are no such nodes in the network, we change it to 2 and then we check the nodes having degree less than or equals to 2, we will keep doing it.

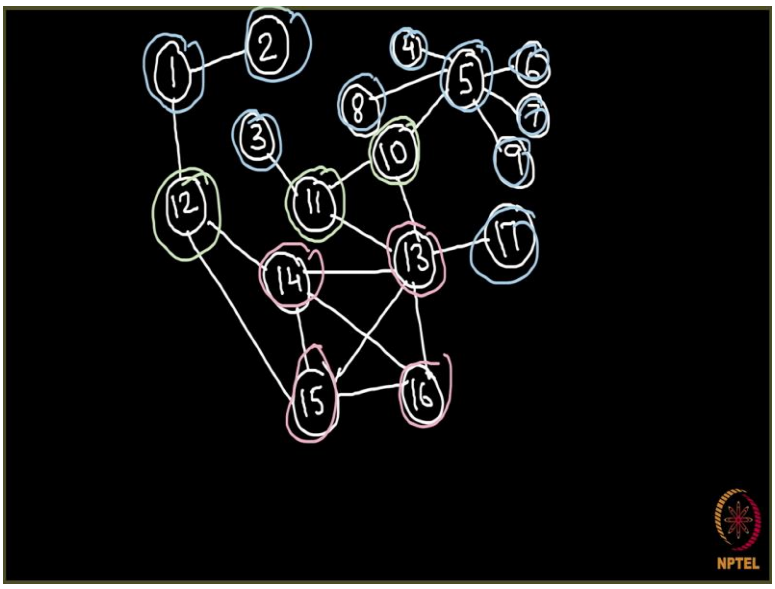
Now do you see this code will keep running forever, there is some problem with this code. At one time all the nodes from this network will be removed and that is the time where we should stop. So, what we are actually going to do is before coming to this step, before coming to this step and before checking whether there are such nodes in the network we can actually put a small condition here.

So, before doing this we can actually put a small condition there and this condition will check whether my network is empty, whether my network has become empty. Whether I have removed all the nodes from this network and if I have removed all the nodes from this network what I will do? I will simply append my temp to buckets.

So, see what is going to happen is, let us say that 5 is the maximum degree in this network and you are removing the nodes having degrees less than or equals to 5. And you remove these networks and at a particular time when you remove a node the network becomes empty, but your tmp will be holding the last nodes present, last nodes which you have removed from this network are still present in tmp, So, that bucket we still have to append.

So, at this time we will append tmp to buckets buckets.append(temp) and you break. Now we will be implementing this and I am sure this will be further clear when you look at the code, you might want to keep this in front of you while writing the code which will make your process easier ok. So, next we are going to write the next we are going to see the code for facial decomposition and please note we need a little bit more complicated network.

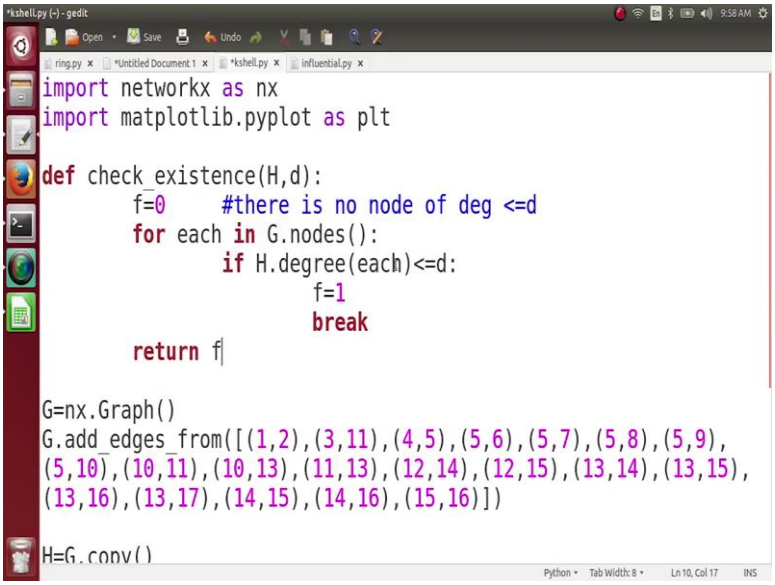
(Refer Slide Time: 10:05)



So, for doing facial decomposition and for all the further codes we will be going to use this particular network which is the slightly a little bit complicated. So, here if you look at the nodes present in nodes which will be removed in the bucket 1, so in bucket 1 these are the nodes which should be removed. So, first you remove this node 2 and 2 and all these nodes 4, 8, 6, 7, 9 and 17 over here and 3 over here and in the next iteration you will remove one and five as well and then your bucket 1 will be over.

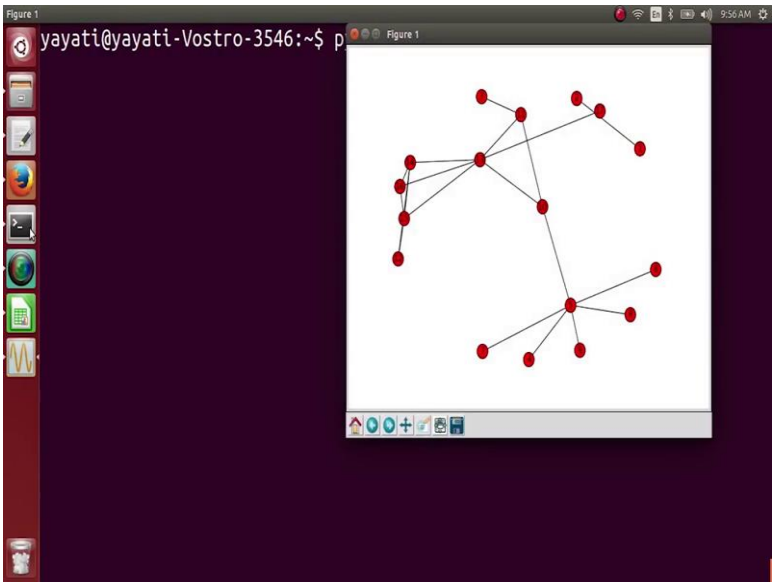
And then in your bucket 2 will be going to remove these nodes 11 over here and 12 over here. And then once you remove 11, you will have to remove this node 10 over here also. So, these nodes should be in your bucket 2 and finally, your bucket 3 should comprise of these 4 nodes over here. So, we will be using this network for our programming screen casts. So, now, we are going to write the code for facial decomposition.

(Refer Slide Time: 11:13)



So, as usual we import network x as nx and we import matplotlib pyplot as plt. What do we do next is we create a graph G = nx graph and we add some edges to this graph? So, this graph which I am going to take is the same graph which we have seen previously in this video. So, the graph which I am going to take is this. You can actually verify and see that it is the same graph. So, we can also draw it and see nx.draw(G) and then plt.show and then we can execute and see it.

(Refer Slide Time: 12:02)



So, this is the network over here and we have to look at various shells in this network ok. What we do? Next is we have already discussed what we are going to do is first of all we are going to create a copy of this network H = G.copy and whatever we are going to do? We are going to do it with H and then we take our variable iteration corresponding to the degree which is initially 1. And then we have a list tmp corresponding to the bucket which we are filling currently.

So, it is corresponding to the let me write it down here. It is for the bucket being filled currently ok. And then we have our overall added bucket which is going to be a list of lists and each list here is a bucket ok. Next what we are going to do is we are going to run a while loop while 1.

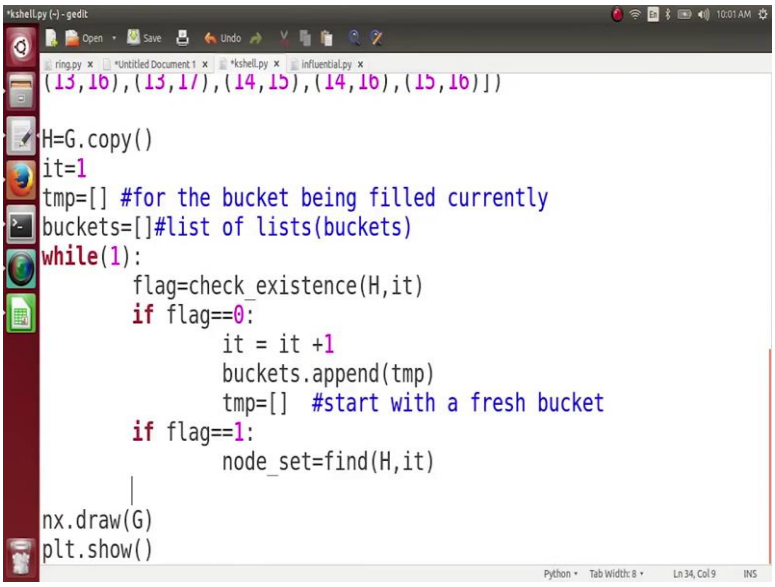
So, this loop will keep running till a terminating condition comes and Now, we put a check condition whether there are nodes of degree less than or equals to it in this network. So, for that I use a function and let us call this function as check existence. So, flag equals to check existence and what do I pass here a graph h and the degree. And let us define this function first here. Define check existence, where I have passed the network H and the particular degree and how do I define it? It is very simple.

So, what do I do? Initially I said f equals to 0 which means that f is going to be 0 till the end of this function if in the case there is no node of degree less than or equals to d in the network right. And then what we are going to do for each in G.nodes we check each and

every node in this network and as soon as we find a node in this network whose degree is less than or equals to d, it means that still there is a node having degree less than or equals to d in the network immediately we set f to 1 and break this loop right and then we return f.

So, this f is going to be 0 till the end of this code if there is no node of degree less than equals to d and it is 1 if there is even 1 node of degree less than or equals to d ok.

(Refer Slide Time: 14:53)



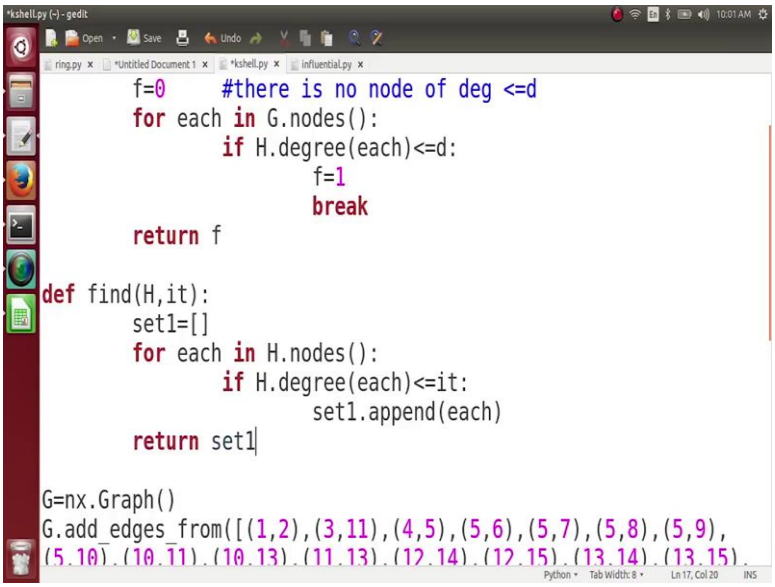
So, flag equals to check existence h comma it. And next what we have seen, flag can be 0 or 1. If flag is 0, it means that we have looked at all the nodes having degree less than or equals to it, even recursively we have seen we have pruned again and again and there is no node of degree less than or equals to it. In that case what we have to do? We have to mark our bucket as complete.

So, if flag is equals to 0, what do we do? We move on to the next degree, we set it equals to it plus 1 and we say that the current bucket is complete. So, we append our temp to buckets, mark it as complete and now we start with a fresh bucket.

So, this is nothing, but start with a fresh bucket right ok. And in another case if flag was 1, what we are going to do? If flag is one it means that we have to keep pruning the nodes having degree less than or equals to it. In that case first of all we should find out such nodes in the current network we have to find the nodes which are having degree

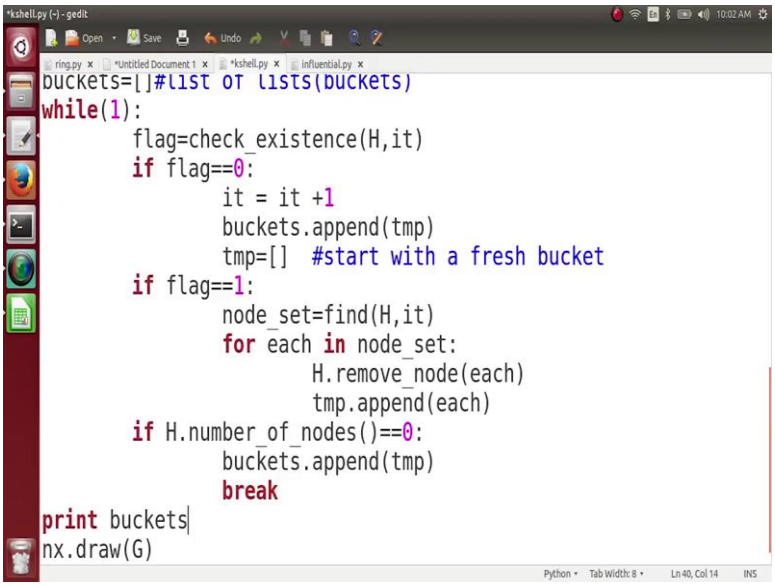
less than or equal to it. Let us call a function for that and let us name this function as find and so in the graph H finally, nodes having degree less than or equals to it and we can quickly write code for this function define find h comma it and how do I define it.

(Refer Slide Time: 16:26)



So, initially this set is empty, and it is simple. For each in H both nodes if H.degree(each), if H.degree of H is less than or equals to it what we are going to do is set 1.append each and then return set 1 right. So, we have find out we have found out all the nodes having degree less than or equals to it. Next, we have to prune these nodes on the network and add them to our bucket. So, what do we do? Prune these nodes from our network and add them to the bucket.

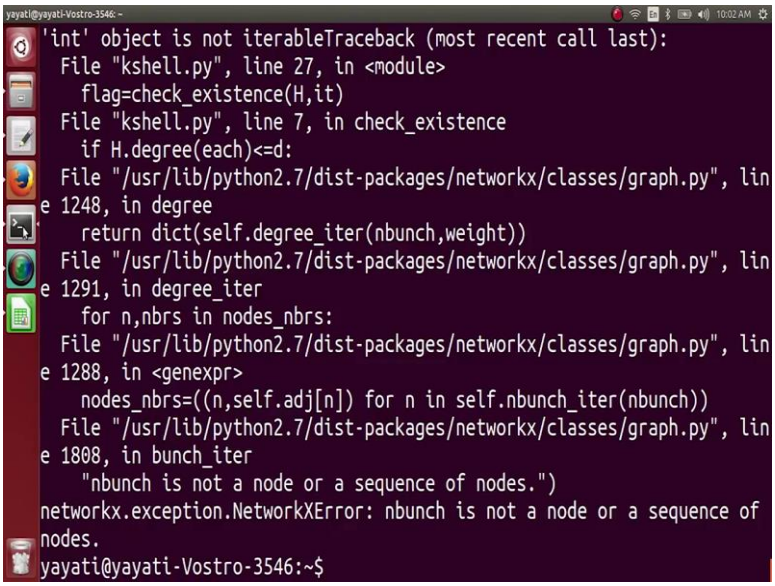
(Refer Slide Time: 17:23)



So, for each in node underscores set what we are going to do is H.remove_node(each), we have removed it from H and we have to put it in our current bucket which is being filled which is nothing, but tmp.append(each) right. In this case we will keep pruning the nodes and the last part of the code is a terminating condition. So, if by pruning these nodes our network it becomes empty what we have to do?

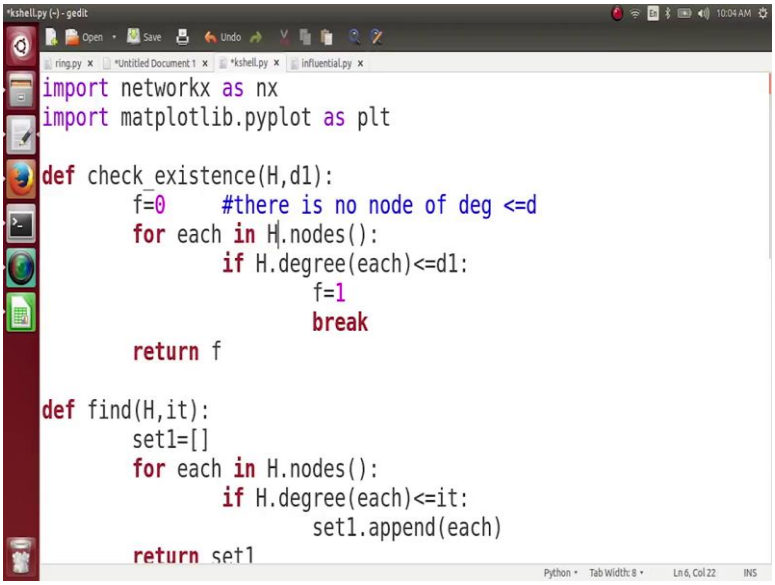
So, if network becomes empty is if H.number_of_nodes if it becomes equals to 0 what we have to do? What we have to mark the current bucket as filled. So, buckets.append(tmp) and then break ok. And now we want to see whether our buckets are correct or not. So, for that what we can do is we can simply print buckets. It tells us the different buckets in our network.

(Refer Slide Time: 18:44)



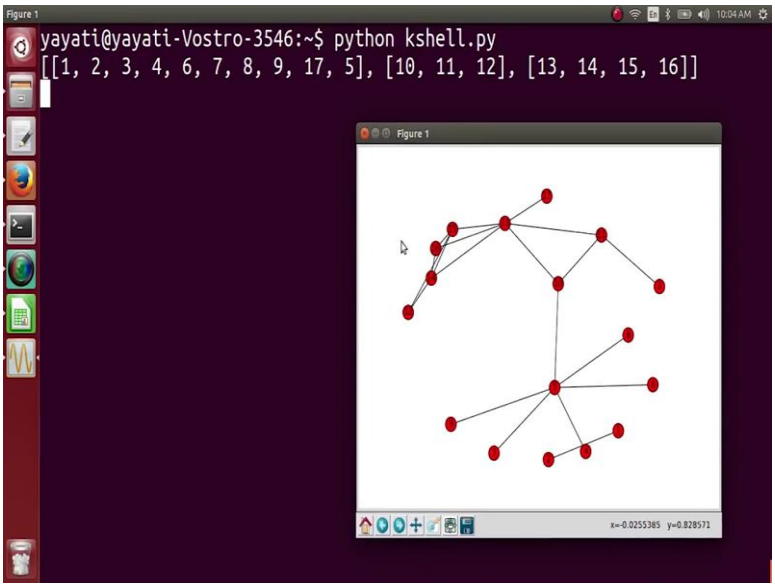
So, let us execute this code and see. An adder at line number 27. Check underscore existence H comma it ok. If it was 0 for each in G.nodes, so if I do not think this (Refer Time: 19:20) ok. Some error let us try to figure it out, some very small error. So, it what is it here? It is a number right a particular degree d for each in let us use some other variable here d1 ok.

(Refer Slide Time: 20:11)



See a mistake here; here we have used G.nodes, so here we have to use H.nodes because, if we are using G.nodes here we might here be talking about a node which is not there in H, H has less nodes than G So, it should be H.nodes here and let us execute it ok.

(Refer Slide Time: 20:32)



So, you see it here, your 1 core is the first bucket is from 1, 2, 3, 4, 6, 7, 8, 9, 17, second is this and third is this and that is the result.

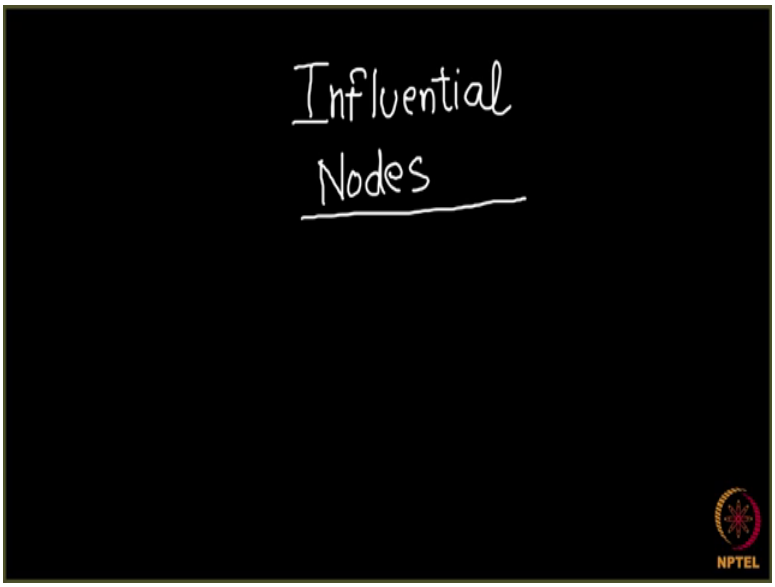
So, we have implemented facial decomposition, if you want you can also color these nodes with different colors to see the differentials and so on.

Social Networks
Prof. S. R. S. Iyengar
Department of Computer Science
Indian Institute of Technology, Ropar

How to go Viral on Web
Lecture - 164
Coding cascading Model

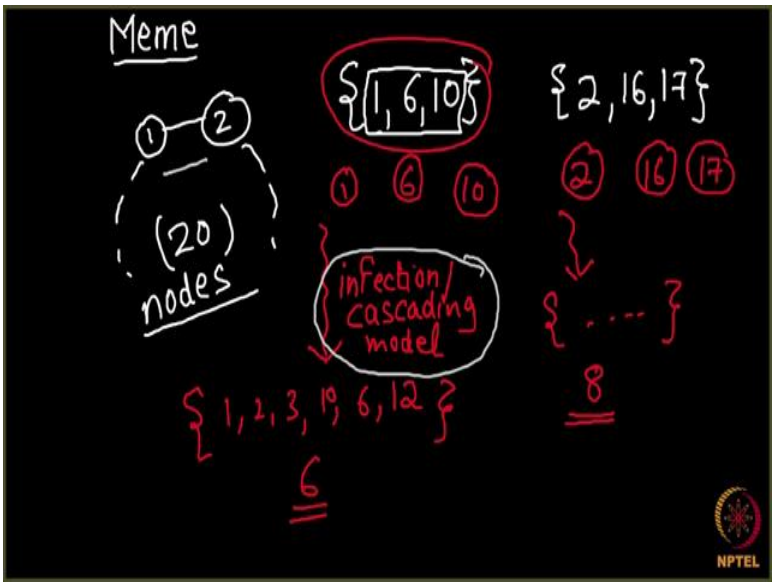
So, next what is our aim? Our aim is to determine the influential pair of nodes. For actual question is we want to see that which nodes in our network are the most influential.

(Refer Slide Time: 00:18)



So, we want to determine the influential nodes in a network, but how do we know which nodes are influential. And what does it mean when I say that these bunch of nodes are influential. What we are going to do that is; experimentally speaking, programmatically speaking, what we are going to do is we can have certain set of (Refer Time: 00:45) nodes from where our infection will start.

(Refer Slide Time: 00:47)



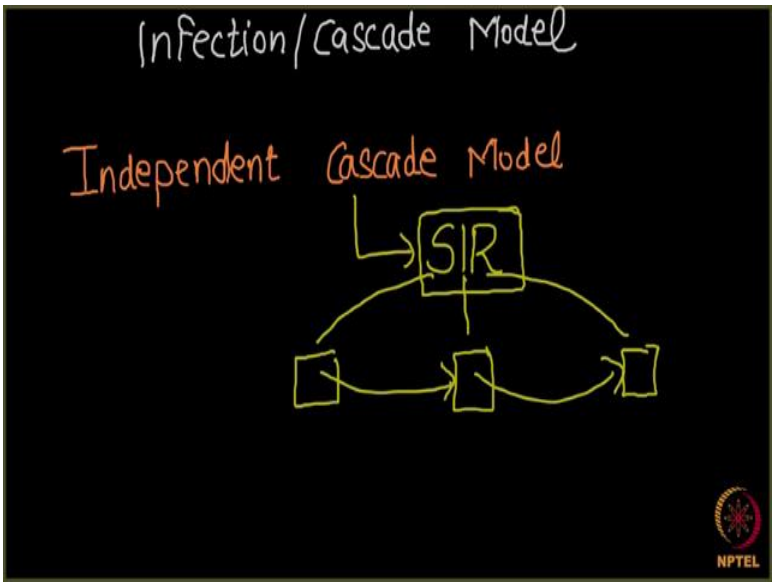
So, let us say there is a meme which wants to traverse on a network which is traversing on a network, let us say the network is something like this consisting of let us say some 20 nodes, 20 nodes some network. And now, we want to see out of these 20 nodes which nodes are the most influential? What we can do for that is if I want to compare let us say 2 sets of nodes, I have a set 1, 6 and 10, so these 3 nodes and I have another set let us say 2, 16 and 17. And I want to see that which out of these 2 sets is most influential.

What I am going to do is initially in this network I infect these 3 nodes with my main, so I will say that node 1 is infected with the main, node six is infected and node ten is infected. And then I will run an infection model, I run cascading model infection or a cascading model. And I will see when this model stops, when this process stops how many nodes in whole are infected. So, let us say that this is the set of nodes which are finally infected, let us say these are the set this is a set of nodes which is finally infected.

So, there are 6 nodes in this set. So, I can say that the influential power of this set is 6. And then to determine the influential power of this another set I can again do the same process. I will initially, in fact these 3 nodes to 16 and 17 in the network, I run my cascading model. And will see at the end how many nodes are infected and let us say 8 nodes are infected; it means that the influential power of this bunch of nodes this set of nodes is 8 ok.

So, this is the way we are going to find which set of nodes is most influential. So, to find which is most influential, we probably will take all possible set of nodes on these 20 nodes network and then determine their influential power and then see that which one creates the maximum cascade. And that is actually very time consuming process ok, but here we talked about the infection or cascading model. So, what is this infection or cascading model, let us talk in a little bit of detail.

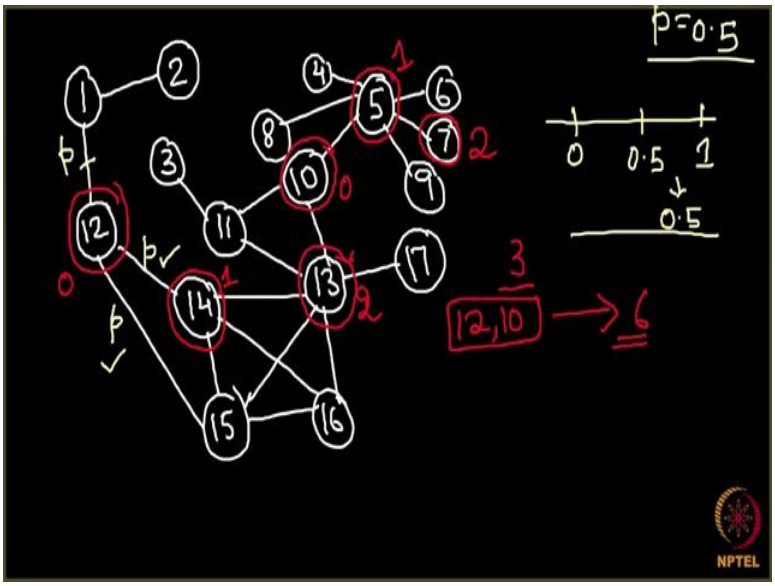
(Refer Slide Time: 03:23)



So, I need an infection or cascade model. And what is this infection or cascade model how do I code it and how does it work. So, that we are going to implement in our next programming screencast. And we are going to look at a cascading model which is known as the independent cascade model, independent cascade model. And this independent cascade model is actually very similar to the SIR model we discussed in the chapter epidemics, very similar to the SIR model.

Do you remember what was SIR model? There were 3 kind of nodes in the network susceptible, infected and recovered and nodes used to pass from this chamber to this chamber infected chamber and then to the recovered chamber. So, our model is very similar to the sir model, the node exactly similar. Let us look at what is independent cascade model now.

(Refer Slide Time: 04:25)



Assume that this is a network given to you here. So, I give you a network here and I want to say that let say the seed nodes in this network are 12 and 10. So, the infection in this network starts from these 2 nodes 12 and 10. And now I want you to implement independent cascade model on this network, how will you do that? So, at time equals to 0, these 2 nodes are infected, nodes 12 and 10 are infected. How the modal proceeds is very simple. What is going to happen at the next time stamp is this node 12 is going to look at all of its neighbors.

So, node 1, node 14 and node 15 and it will infect each of its neighbors independently with a probability p. And let us say p is 0.5, we can set the value of 2p to be anything. So, it infects each of its nodes, each of its neighbors with the probability of p and we have done it previously right. How can we simulate this? We can actually toss a coin for this edge if we get an head then 12 infects 1 else node toss a coin for this edges well and toss a coin for this edges well. That is that was how we can implement these and in terms of programming also it is very simple.

So, what we can do is we can generate a random number from 0 to 1 and then we look whether this random number is greater than 0.5 or node. So, what is the probability that this random number is greater than 0.5, it is nothing, but 0.5. So, this is how we can do it in our coding. So, we are we generate a random number from 0 to 1 and if the number is greater than 0.5, we will in fact, the corresponding node. Ok that apart.

So, a time iteration 1, this node 12 over here we try to infect each of its neighbors with the probability p and let us say it succeeds in infecting this node 14 over here. And then node 10 is also doing the same, it will look at all of its neighbors and infect them with the probability p.

So, it again it has 3 neighbors and let us say it succeeds in infecting the node 5 over here. So, a time iteration 1 this node 14 over here and this node 5 over here gets infected ok. What's now going to happen in next iteration is node 12 and node 10 have done their job. So, they were given 1 chance to infect their neighbors and their chance is now over. So, these 2 nodes it remains infected they are infected, but they do not get any further chance of infection.

So, it is very similar to a case where in a school some students are getting infected. So, initially at the first way the students who are infected, they infect the people around, but during the second day at second day at second day they do not get any chance to infect anybody. So, nodes 12 and 10; now we will not infect anybody and the just infected nodes, which are the just infected nodes, so it is time iteration 1. Just infected nodes are 14 and 5.

So, 14 and 5 now gets a chance get a chance to infect their neighbors. So, this node 14 over here looks at all of its neighbors and tries to infect each of them with a probability p except for the node 12 because, node 12 is already infected. So, we cannot infect it further. So, for remaining neighbors which are 3 in number, so for each of these neighbors it sees whether it can infect them or node and let us say node 14 succeeds in infecting this node 13 over here.

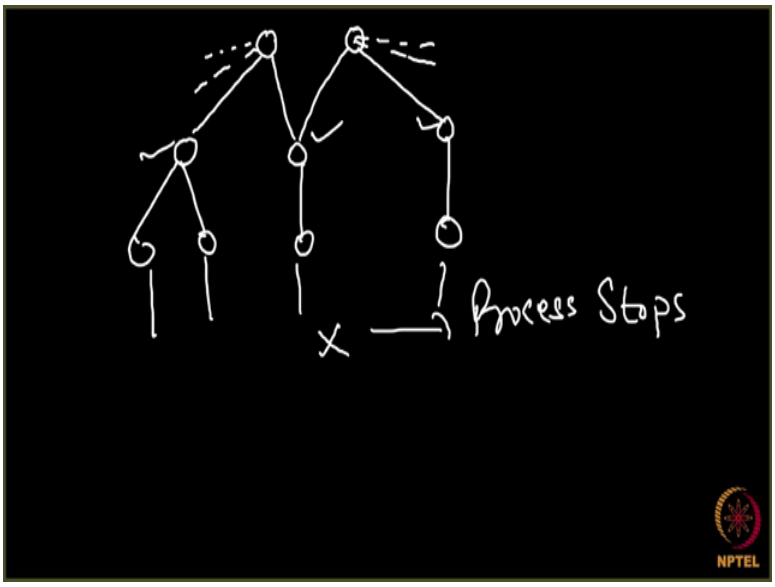
So, time iteration 1 node 13 gets infected. And in the same iteration this node 5 over here also tries to infect its remaining neighbors. So, there are 5 neighbors and let us say it succeeds in infecting this neighbor 7. So, at time iteration 2, so a time iteration 2 this node 13 over here and this node 7 over here gets infected and again in the next iteration these node 7 and 13 look at their neighbors and try infecting them and what will happen then, let us say that this node 13 is unable to infect anybody and this node seven over here is also unable to infect anybody right.

So, nobody gets infected in iteration 3. What will happen next? So, these will be just now infected nodes which were infecting their neighbors. 13 and 7 have also got their due

chance of infecting their neighbors, but they could not infect anybody. So, now, there is nobody in the network which could look at their neighbors and infect them and hence the process stops.

So, do you see where did the process stop? The process stopped when we reach a iteration, we reach an iteration where nobody is infected. And in this particular case you saw that we started with these nodes 12 and 10 over here which were initially infected and the size of the cascade is 6. So, the influential power of node is set 12 and 10, we can say 6. So, this is how an independent cascade model is going to work. Let me revise it quickly.

(Refer Slide Time: 09:44)



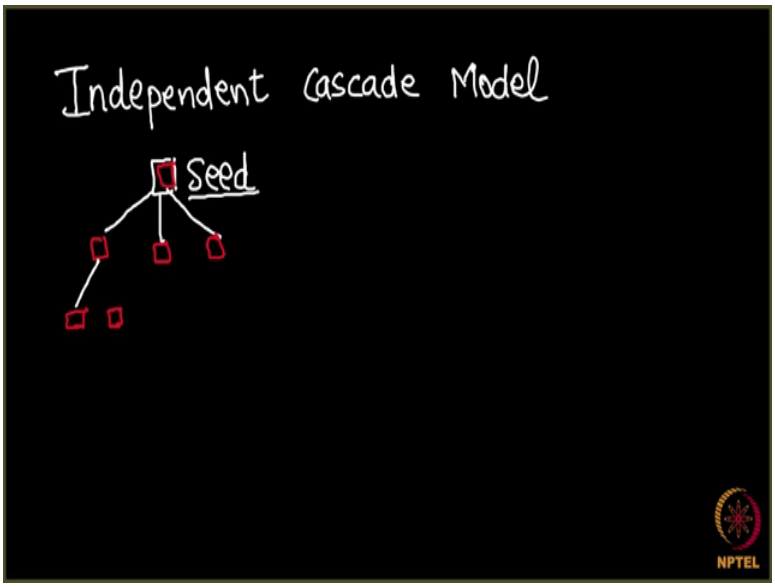
So, what is happening is initially there are certain nodes which are infected let us say these 2 nodes are infected. In the next iteration these nodes will look at their neighbors. So, this can be a possible case here right. So, these nodes will look at their neighbors and infect each of them with the probability p and let us say that these 3 neighbors got infected.

So, they might have other neighbors as well and let us see out of all these neighbors only these 3 are the ones which got infected. Then these three nodes over here will repeat the process and in fact, all of their neighbors with probability p and they might succeed in infecting some people and let us say here for more people go to infected. And let us see in the next iteration nobody goes to infected. So, in the next iteration nobody goes to

infected. So, what will happen is your process will stop here. So, this is what is known as an independent cascade model.

And why is it called an independent cascade model is actually very clear whether this edge over here get passes the infection or node is independent of whether this says over here passes the infection or not. So, hence it is known as the independent cascade model. And next we look at how can we code this independent cascade model. So, let us now see how we can implement this independent cascade model. So, before writing the code for independent cascade model let us look at the structure of the code.

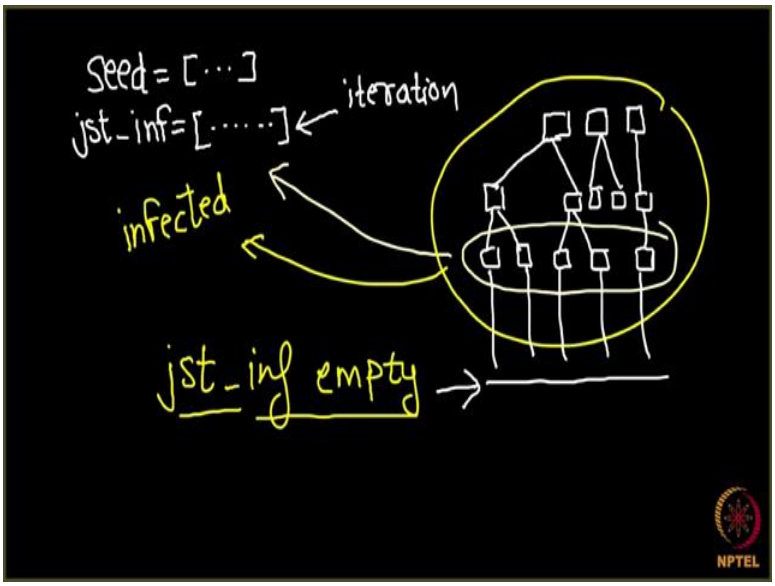
(Refer Slide Time: 11:07)



So, we are going to see how can we implement independent cascade model, independent cascade model. And what we are going to do is we know that we know the entire procedure right, we know that there is a set of nodes which are our seed nodes, which are our initially infected nodes. And the seed nodes are then going to look around all of their neighbors and ending up infecting few of its neighbors and ok.

So, this is a C and they end up infecting few of their neighbors and then all of these neighbors are going to look at their neighbors and then they are going to infect few of their neighbors. So, these are the neighbors which are not already infected. So, some new nodes over here are infected and this process will stop as soon as we reach an iteration where no new node is infected. So, how can we write a code for this? While writing a code for this we are going to use 3 lists.

(Refer Slide Time: 12:06)



So, let me tell you one list is obviously the seed nodes, which holds which of the nodes are initially infected, so this list is seen. Another list which we will be considering is let recall it just infected. So, another list is just infected. What does this list hold is, this list holds the moves which are most recently infected in any iteration?

So, so this particular list it iteration dependent. If you look at this list seed, it is always going to be the same. So, these are the nodes which were initially infected. It is just infected list it is going to change with time. So, in iteration 1, when C is going to infect certain nodes then this list will change, and whichever nodes will be infected at the first iteration will come in just infected. So, let us say again I will make the same diagram the seed is here and then this let us say there are 3 nodes in seed set and these 3 nodes they end up infecting some more loads. So, these are some more nodes which got infected life. So, at this particular iteration this will be my set of just infected nodes.

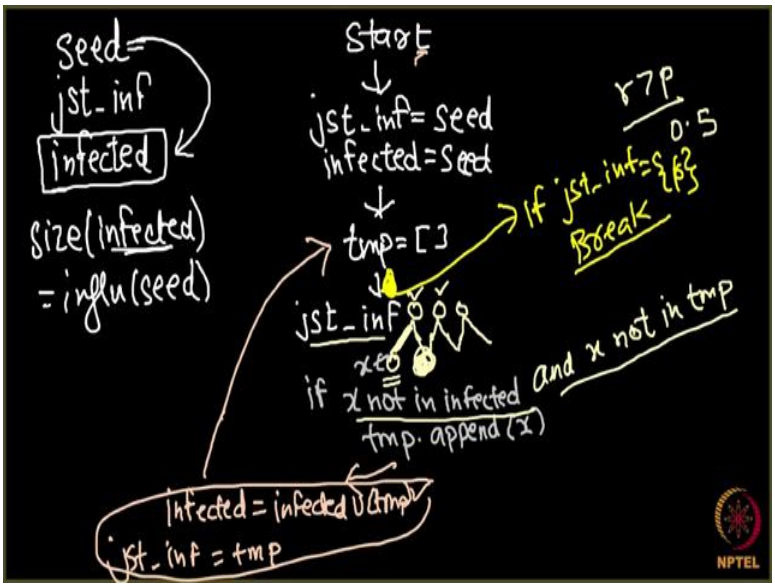
So, at this iteration this becomes the least just infected right and then we will also be using another list which is infected. And this list holds all the nodes over overall the process throughout the process till now whichever nodes are infected comes in this infected list. So, you see what is what is infected currently. So, this entire these all the nodes are infected right.

So, this is my list infected, this is my list infected. What will happen in the next iteration? What will happen over next iteration is let us say so, these whole nodes over

here they might end up infecting some more nodes. So, let us say in iteration 2 these are the nodes which are infected. Now what is just infected? So, these nodes over here these nodes are my just infected and then this entire set of nodes is my infected right.

So, we will be using these 3 lists, and now can you tell me when will this process end? This process will end at an iteration when this list just infected becomes empty. So, as soon as this just infected becomes empty will break the loop and come out. So, after this particular iteration over here, let us say that in the next iteration none of the nodes gets infected. So, just infected fact, it is empty here and the process will come to an end ok.

(Refer Slide Time: 15:01)



So, how we are going to implement it let us see. To implement it as we saw before we have 3 lists. So, 1 is corresponding to seed, one is the nodes which are just infected and at the end we have all the nodes which are infected. So, this particular thing we are going to return at the end. These are the overall nodes which are infected throughout the process. And the size of this list infected is what we called the influential power of this set seed.

We will be starting our infection with seed and at the end certain nodes will be infected. And the size of these infected nodes is nothing but the influential power over seed ok. And this is what we are going to return. Now what we will be doing is to implement our independent cascade model in the very beginning, so let me put a start over here. So, here we start, after starting I sit the value of just infected to be seed right, when nothing

has happened, so these are the seed nodes which are just infected and which have to create my further infections and the value of infected is also equal to seed because, when nothing has happened these are only the seed nodes which are infected.

What will happen next is we will enter a loop and what will happen in this loop is these all nodes which are in just infected. So, we will look at this just infected set and let us say ok. So, before doing this a small step and it take a temporary list. And let me call it tmp and you will understand why I am taking this empty list over here. So, this is my tmp. What will happen is I will take all the nodes which are in just infected. So, let us say that these are all the nodes which are in just infected and then all these nodes will look at their neighbors right, all the nodes will look at their neighbors. And then they will infect each of their neighbors with a probability of p.

So, you can do this by how do you infect them with a probability p, we have discussed it earlier. We are going to generate a random number from 0 to 1, a real number and if the number turns out to be greater than p, if r > p we are going to insert the node over here else node right. So, when p (Refer Time: 17:32) 0.5 will generate a number from 0 to 1 and if this number is greater than 0.5 it means that this node over here should be infected ok.

While infecting it, but few things we have to take care is, first of all this node should not already be infected right. So, we put the desired loops here. Overall what we have to do at this step is we have to look at this list just infected and then for each of its neighbors over here for each of the neighbors, so first we take this node over here and then for each of its neighbor we generate this random numbers and then we see whether this node over here should be infected or not.

And if this node is not already infected that is this node should not belong to a set infected, then we are going to append it to tmp. So, let us say that let me write it down, if a particular neighbor, so let me call this node over here as x, if x node in infected right. So what I am going to do is tmp.append(x) right and I am going to do this for each of the nodes over here. So, first I will do it for this x then I will do it for this x, then i will do it for this x, this x and so on.

And one thing you might note here now it seems it is a graph, it is not a tree like structure what can happen is let us say this node over here it is the neighbor of both the

node, it is the neighbor of this node as well as this node. And let us say when we came here this node over here got infected, so we have appended it to tmp right.

So, when we were looking at the neighbors of this node, we saw whether it should be infected and then we see that yes this node should be infected it is not already infected. So, we append it to a list tmp, but then when this node is looking at its neighbors and it might also see that it should be infected right, it is not till now it is not infected because, it is note in this list infected it is in list (Refer Time: 19:41). So, we might again appended to temp. So, to avoid this, so that sort of duplication what we can do it if x noting infected and x noting tmp. This is not a very important step, but just to avoid duplication we can write it here.

So, first of all this node over here, it should not be previously infected, second it shouldn't be in tmp. In that case we append x to tmp. So, we get this list tmp at the end. What is this list tmp telling us? tmp is telling us that in the current iteration these are the nodes which got infected. So, what we do next is we go to our set of nodes in tmp, which should be infected next.

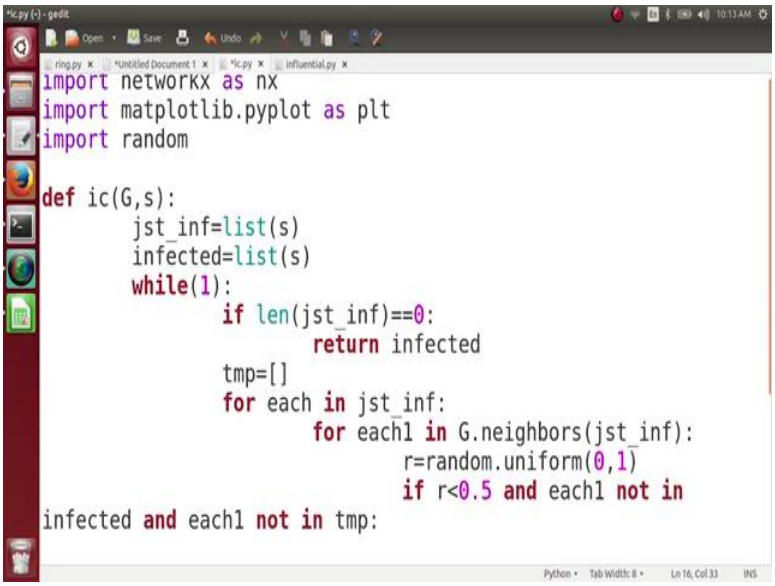
So, we will simply for all the nodes in tmp we set our list infected right. So, these all node should be infected. Infected becomes infected union tmp right. So, these all nodes become infected first of all then. Secondly, you notice that now my set of just infected should change right. So, previously my just infected were the node which were infected in that iteration, next what should become just infected now? Just infected should become equal to tmp right and then we jump back.

So, after doing both of these steps we jumped back here ok. So, you see what is happening, we start from here, we start from here and then we set just infected to be seen infected to be seen, we take a temporary list. For initially just infected is seed and then we seeded which node should get infected in the current iteration. We append them to the list infected and then we said just infected equals to tmp, then we come here. Then for all the nodes which were infected in the previous iteration we keep repeating this process ok. Now it has become an infinite loop right, it will keep running keep running keep running.

So, one should this process come to an end when this list just infected becomes empty. So, we can put as small terminating condition over here. Over here we can put

terminating condition that if your list just infected is 9, it has no element than you break right. So, this is the terminating condition over here. So, this is my entire function for implementing independent cascade model. And at the end we can return the list infected which holds all the nodes which are infected during this entire process ok.

(Refer Slide Time: 22:27)



So, now we are going to write a code for login independent cascade model. So, already made a file here and we are going to use the same graph which we have used previously. So, some part of the code I have written very basic part. And next let us look at how can we implement independent cascade model here. So, you should have some seed in this network.

So, we can randomly define some seed and let us say my seed is node 3 and let us say node 8. And next what I want to do is, I want to call my independent a function independent cascade I see which runs the independent cascade model on this graph G taking as input the seed c and gives me as output a list. And this is the list of the people who are infected finally, at the end.

So, that is list 1 ok. How do we implement it? So, define independent cascade G comma seed ok. What we are going to do is, we have to so, we have 1 list seed, another list we know we had a list called just infected and which is nothing which is initially going to be the same as the seed node, let us call it as here. It is same as the seed node s and we know that the people who are infected currently are also nothing, but the seed s right

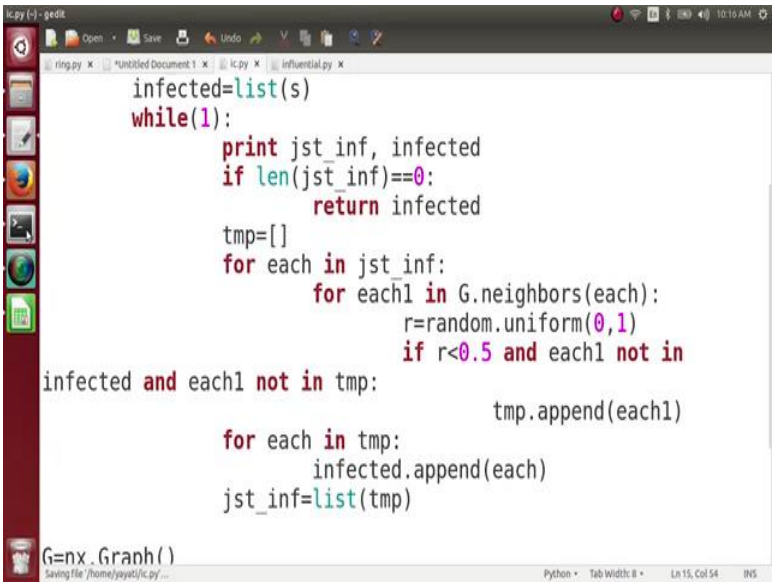
initially, both of these nodes have the same value as c. And then we have a loop while 1, while 1 what are we going to do, we know that what is the terminating condition for this loop. So, there should be some persons who are just infected. If nobody is just infected, we should break.

So, if length of just infected is equals to 0, we are going to what we will do, we will return our list infected that is all the infected people right, that is sufficient we will return ok. That is the end of this function otherwise, what we have to do? We have to find out the new people who will get infected. So, we have a temporary array tmp for that and how do we find these people? We look at everybody for each in this list just infected; we are going to look at their neighbors.

So, for each 1 in G dot neighbors of just infected people what are we going to do? Now, so here is an edge from 1 person here to 1 person here and we have to do. We have to see whether the person will get infected or not that depends upon the probability of infection right. And we have seen before how do we model the probability of infection was with the help of random variable.

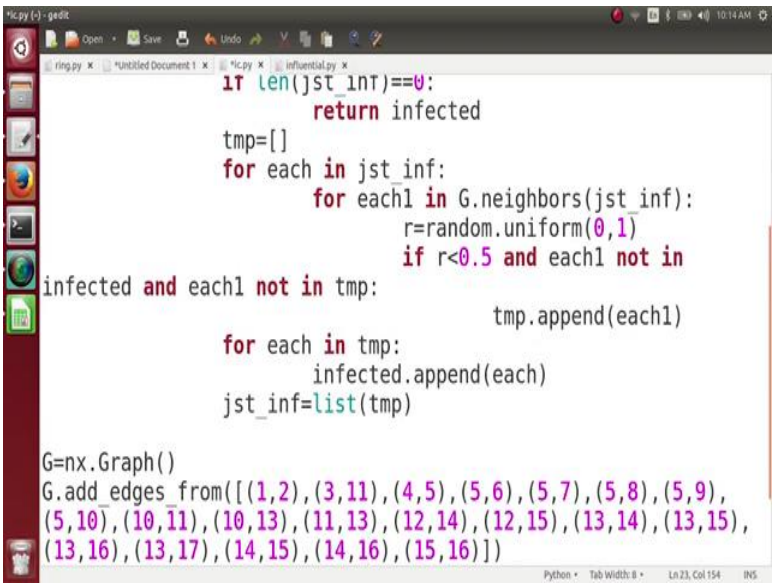
So, we import random here and what do we do is we generate a random number r, random real number r from 0 to 1 and what do we do next is if, so when is the infection going to occur when r is less than 0.5 let us say the probability of infection is 0.5 and what is the other condition, this node each 1 should not be previously infected. Each 1 not in infected and as we discussed previously to avoid any duplication in tmp, 1 not getting infected by the 2 nodes which are in just infected we do for each 1 node in tmp also, in tmp we do not need any duplication. So, what we do here, if this happens is the tmp dot append each one.

(Refer Slide Time: 26:59)



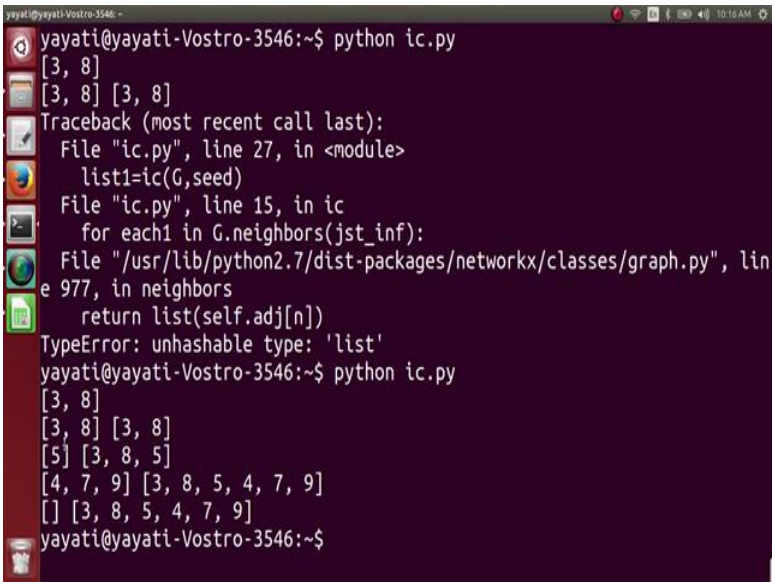
So, this each 1 is the node which is going to be infected in this iteration ok. So, we have found out all the nodes which get infected in the current iteration then, what we have to do for each in tmp. These nodes which got infected just now, what are we going to do infected dot append. So, these nodes are now infected. So, we append them to this list infected, infected dot append each and our list just infected now get tends to tmp and that is done. So, this is a very simple code and right. So, we want to see how does this code run.

(Refer Slide Time: 27:54)



So, for that what I am going to do is let us print the value of just infected at every iteration and see how this is going. So, we print both the nodes, let us say we print just infected first and then let us print infected the total notes infected and at the beginning let us print s right ok, s just infected, infected and here before returning infected, here also let us print infected. Actually, there is no need for these leave it as it is ok. Let us run this code and see ok.

(Refer Slide Time: 28:42)



We see a problem here just infected is a list here right. So, we have done here, for each in G dot neighbors it should be each here right, for each in just infected for each 1 in G dot neighbors each right. (Refer Time: 29:27).

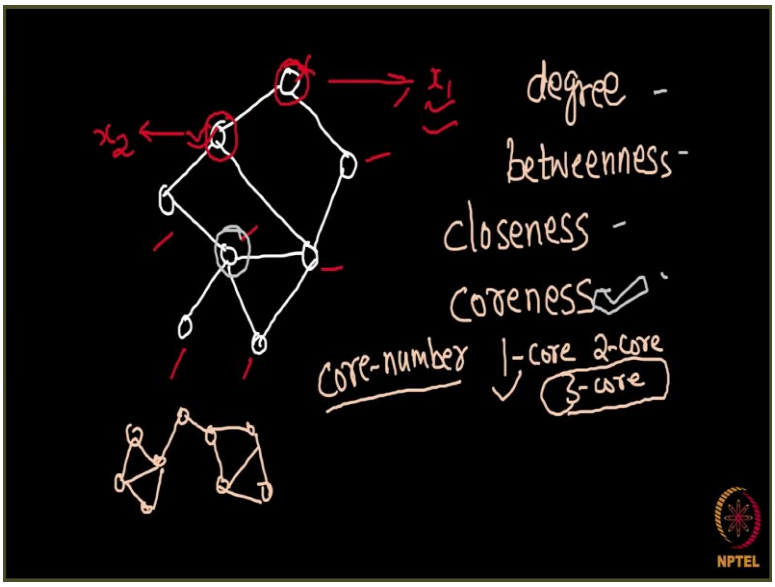
And then you can see here initially 3 and 8 were seed nodes 3 and 8 were the seed nodes, so just infected are also 3 and 8 and infected and 3 and 8 then one of these infected 5 and 5 also became infected. Then 5 infected 3 more nodes 4, 7 and 9 which got I did and 4, 7, 9 could not infect anybody. And hence, these were the finally infected node and the process finally stopped.

Social Networks
Prof. S. R. S. Iyengar
Department of Computer Science
Indian Institute of Technology, Ropar

How to go Viral on Web
Lecture - 165
Coding the importance of core nodes in cascading

So, next we are going to do an interesting experiment. What we are going to do is we are interested in looking at which are the most influential nodes in a network. And we have our answer also right.

(Refer Slide Time: 00:19)



So, given a network over here we saw that to find that let us say in this network over here this network over here I want to see which nodes over here are the most influential. So, what can I do is we know that how do we define influence for a seed set; given a seed we will start our independent cascade model from there and we will see that at the end how many nodes got infected.

So, to say that which nodes over here are most influential what we can do is first of all we say that this is my seed. This is my seed and then I perform the independent cascade model here and I see that at the end how many people got infected and let us say I found a value x 1. Then I want to see the influential power of this node. So, I said the seed to be this now this is no longer the seed.

So, this over here is the seed only 1 node and then I will repeat the independent cascade model and I will see towards the end how many people got infected. I will do for this, I will do for this, do for this, do for this and do for this and at the end we will see that which of the x is maximum. And, whichever x is maximum that particular node is the node, which is the most influential in this network, but doing this kind of an experiment it is a very time consuming process right.

So, taking every node to be seed node and again and again repeating the independent cascade model and then finding out influential node is a time-consuming process. Here we have another very interesting question; can you look at this network and then guess which is the node most influential over here? I have a feeling for example, this node over here it has 1, 2, 3, 4 neighbors and no other node or maybe this node over here.

So, they have 4 neighbors. So, probably might be they are most influential, since they have more number of neighbors they can infect more people right. So, it might be the case that these nodes are most influential. So, that is an easy way of doing this same process right. So, taking every node over here and then running independent cascade model as opposed to just looking at this network and looking at its structure, looking at the properties of certain nodes and then saying that: yes, this node is influential is quite easy.

So, now the question is which nodes are the most influential over here. So, we have various centrality measures ok. What is centrality measure? It is just a complicated term. So, centrality measure is we can measure the values of notes over here based on certain things. For example, we can look at the degree of each node right.

So, is it the degree can I say that the nodes which are having the maximum degree, they are the most influential nodes or we have another measure let us say betweenness, I hope you remember what was betweenness. So, betweenness tells me if I look at a node, if I look at a node how important is it in connecting to different parts of the network?

So, let me use another example. Let us say there is a community structure, this is one community, and this is 1 community right and there is a node over here. So, this node over here is very instrumental in connecting these 2 portions of the graph. So, this node has a very high betweenness over here right. So, are these the nodes which are having a

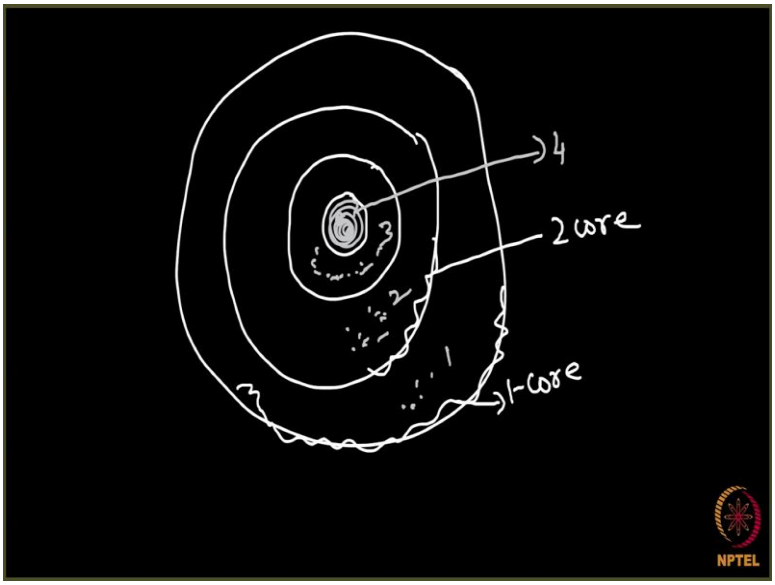
very high betweenness, which are the most influential and we will create the biggest cascade or are these the nodes having high closeness.

So, closeness is something like I take this node and I see that how close is it to other nodes in the network. For example, this node is very close to these 5 nodes and then still closer to this node and then far away from these nodes. So, you can look it up there is a very nice way of calculating this closeness centrality, but the intuition is simple. We see that how closer I am to the entire network that is closeness.

So, are these the nodes which are having a high value of closeness, which are the most influential oh we have a last measure which we discussed coreness. So, are these the nodes which are having maximum coreness, which are most influential? What do I mean by maximum coreness? Maximum coreness is we know that all the nodes in this network are in 1 core right. We define a value which is called core number, what is core number? It is nothing but the.

So, see a node can be part of 1 core, a node can be a part of 2 core, 3 core right. Whatever, is in 3 core, whatever node is in 3 core that is automatically in 1 core So, let us say there is a node which belong to three core I say that its core number is 3, right call number is 3 So, the inner you are in the network, the innermost core you are towards oh see let me put it like this ok.

(Refer Slide Time: 05:37)

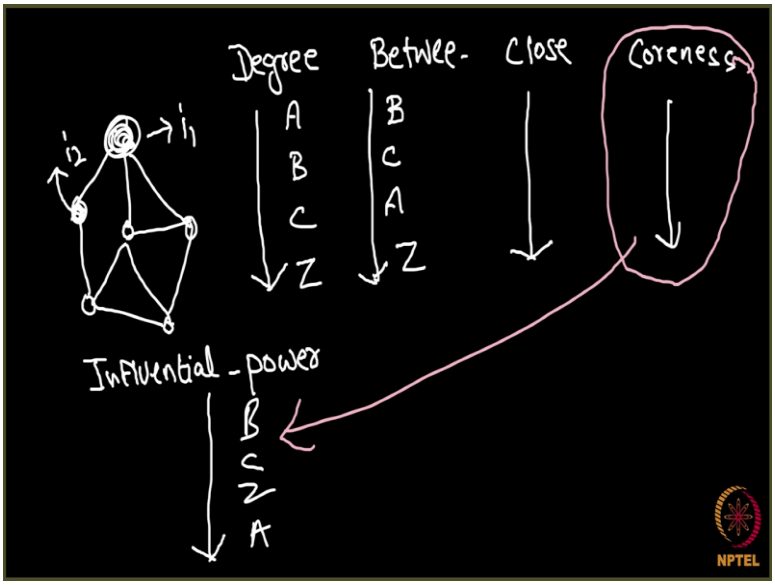


Let me say that this is the network, and then in this network, this is the so, this in this is 1 core right. So, whatever is inside this particular circle is in 1 core, whatever is inside this particular circle is in 2 core so on, and so we have 4 cores in this network. Now we know that the node which is present over here right, a node which is present over here it is a part of 1 core as well as 2 core as well as 3 core as well as 4 core right, but it is the innermost.

So, I say that the core number of this node over here is 4, the core number of all the nodes which are present over here is 3, the core number of all the nodes which are present here is 2 and which are present here is one. So, are these the nodes which lie over here which are the most influential? Has the influential power something to do with the coreness of a node which is the core number of a node?

So, this thing is what we want to look at. So, how we are going to do it is we are going to take a network. So, I take a network over here.

(Refer Slide Time: 06:47)



We will be doing it next in our next programming screen cast. So, I will take a network over here, what we are going to do? We are going to take degree of these nodes and we are going to arrange these nodes in descending order of their degree.

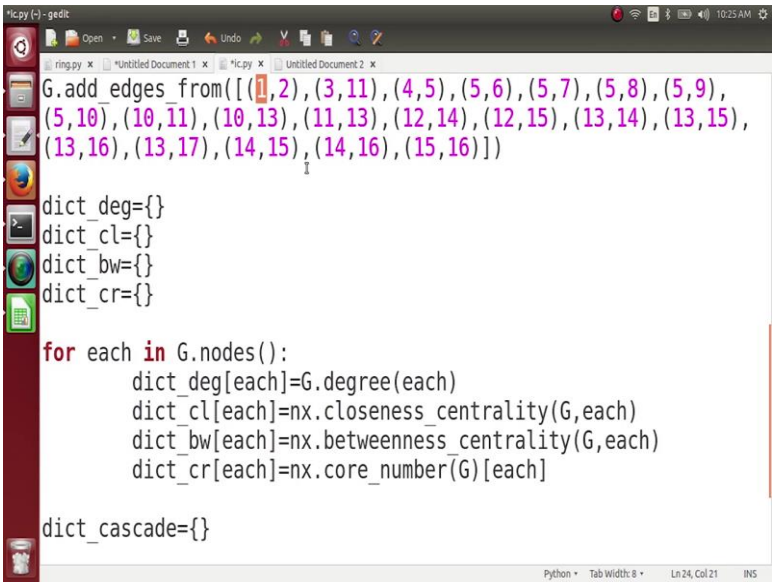
So, node which has the maximum degree comes over here and the nodes which has a minimum degree comes over here; let us say something like this. And similarly, we are

going to look at betweenness and we are going to arrange these nodes, nodes having the highest betweenness over here, least betweenness over here, similarly we are going to do for closeness, and we are going to do for coolness. And that aim is to see which out of these four measures is most related to the influential power of a node.

So, we will take a new list corresponding to the influential power. An influential power will actually take nodes and determine their influential power. So, I will take this node over here and look run independent cascade model from it and then see what is its influential power say; i_1 going to do this for this same node i_2 and then here i will arrange the nodes in the descending order of their influential power. So, let us say be caesar day right.

So, arrange the node here according to their influential power and then we will compare these lists. So, we will see at whether these are the nodes having highest degree which have the highest influential power or these are the nodes having highest betweenness which have highest influential power and to a surprise not surprised cannot call it supplies, but we see that actually these are the nodes having the highest value of coreness which best defines which are the most influential nodes in a network.

(Refer Slide Time: 08:39)



So, what we are going to do now is we are going to look at different-different nodes, arrange them according to certain degree closeness, betweenness, coreness and their cascading power and then see which of the centrality is matched the most with influential

power of a node. Where influential power is over noticed, if you select that particular node as seed for your infection, how many nodes are infected by the end of your process.

So, what we are going to do here now is we are going to use the same code which we have written previously for the independent cascade model ok. So, we are going to make 4 dictionaries now for holding different values. A dictionary for holding the degrees of nodes, a dictionary for holding the closeness of nodes; let us call it cl for simplicity and then dictionary for holding the betweenness of nodes. Let us call it bw and then a node a dictionary for holding the core numbers of nodes which are found by a shell decomposition and let us call that coreness denoted by cr ok.

So, we have these 4 dictionaries and now we are going to flood these dictionaries. How do we put in the values for these dictionaries for each in G.nodes what we are going to do is dictionary for degree the value corresponding to the key each which is the node e is the degree of that node, G.degree(each) right. And then dictionary corresponding to the closeness each, so there is a function in networkx you can directly calculate the closeness of a node and it is called nx.closeness_centrality you can look it up and then stored closeness centrality.

And then we are going to put G comma each here and there is a similar function for betweenness bw is nx and this is simply called betweenness centrality. So, betweeness and a surprise for you I hope you did not know it earlier otherwise you would not have heard the previous programming screen cast. You can actually get the core number of a node directly with the help of networkx with the help of the function known as core number, which performs k shell decomposition on a network and gives you the core number for a v_1 node right.

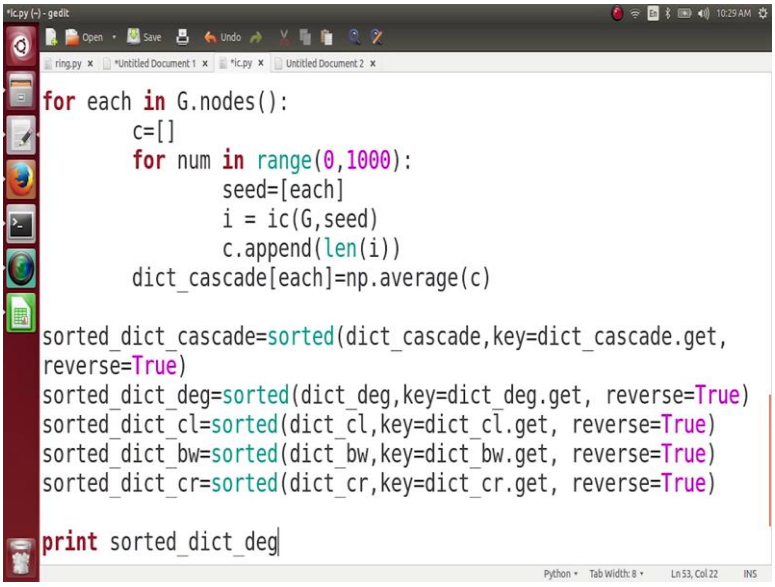
And here you see that the notation is a little bit different why. So, when you use nx.core_number(G) is nothing but a dictionary. In that dictionary we want, so these are these are the values. So, these are the functions which returns you the values. nx.core_number is a function. So, this is a function which returns your dictionary and from that dictionary you want the value corresponding to the node each.

So, that is why we have wrote it in this manner. So, we got the dictionaries corresponding to all these force and centralities. And at the end what we need we are creating our own dictionary which holds the cascading power of node. So, we have a

dictionary for that and how do we flood these dictionary? We have to flood this dictionary using our independent cascade model. So, how do we do that? So, in this 1 by 1 we will select every node in this network as seed. So, we will take 1s seed, 2s seed and what we will do to as we know when we run this independent cascade model, it is quite a random process right. So, we will run it some 1000 times.

So, we will take 1 s seed a run independent of cascade models from this 1000 times and take the average number of nodes which are getting infected.

(Refer Slide Time: 12:57)



So, what do we do? For each in G.nodes what we are going to do is c is currently empty. And then we take a loop which runs 1000 times for numbing range to 0 to 1000 and what we are going to do we said the seed of the process to be equal to each and then we call our independent cascade model with graph G and seeds.

So, now this value i over here is nothing but a list which holds the final people infected when the seed was the 1 node each here. So, what do we do? We append this length of i inner array c right. So, some 1000 times we get different-different people who are infected for different-different processes and for all these 1000 times we append the number of people who are infected in this array c.

So, this is array c we will be consisting of 1000 entries and each entry will correspond to 1 process where this node each was infected, and some people were infected by the end

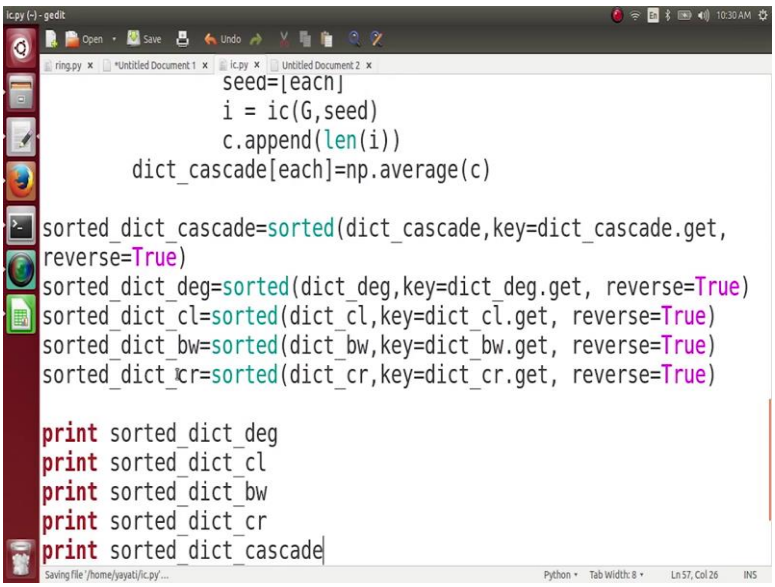
of the process. And now what I am going to append in my dictionary cascade dig the cascade each is the average, average from c, that is on an average how many people got infected, if I started my infection from this node each right.

And then what I am going to do now is I have find out I found out 5 dictionaries, 1 for degree closeness betweenness cornice and 1 for cascading and now I want to see which out of these 4 correlates the most with this 1, which is our degree for cascade.

So, what I am going to do is I am going to sort all of these dictionaries. So, sorting is easy with 1 command, so I will let first sort this dictionary corresponding to cascade So, there is a function sorted which you can call on a dictionary. So, you pass your dictionary cascade here right and we want to sort according to the keys are going to be nothing, but dictionary cascade.get and we want to sort them in a reverse order. So, we said reverse equals to true and we do this with all our dictionaries with all the remaining 4 dictionaries.

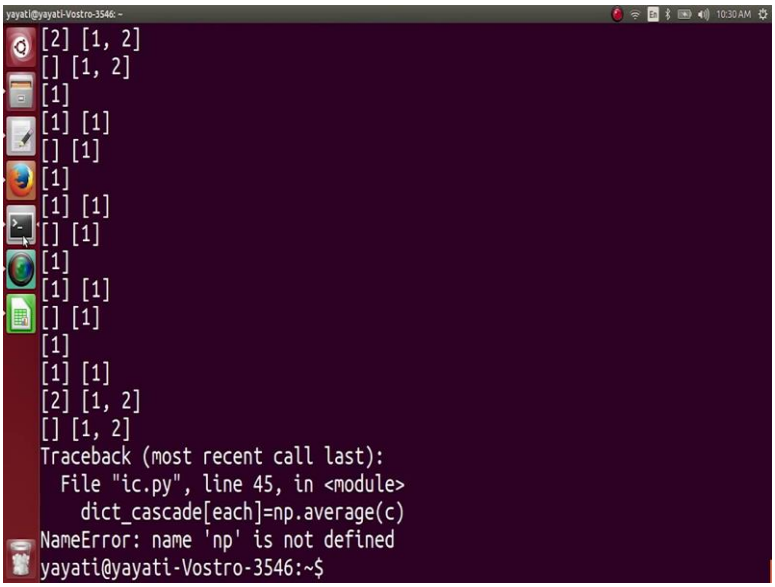
So, this cascade thing we can core degree we will replace it with degree and for closeness we replace it with cl and for betweenness we replace it with bw, bw and here also bw right and four corners we replace it with cr. And we can actually print all of these and see what is happening.

(Refer Slide Time: 16:29)



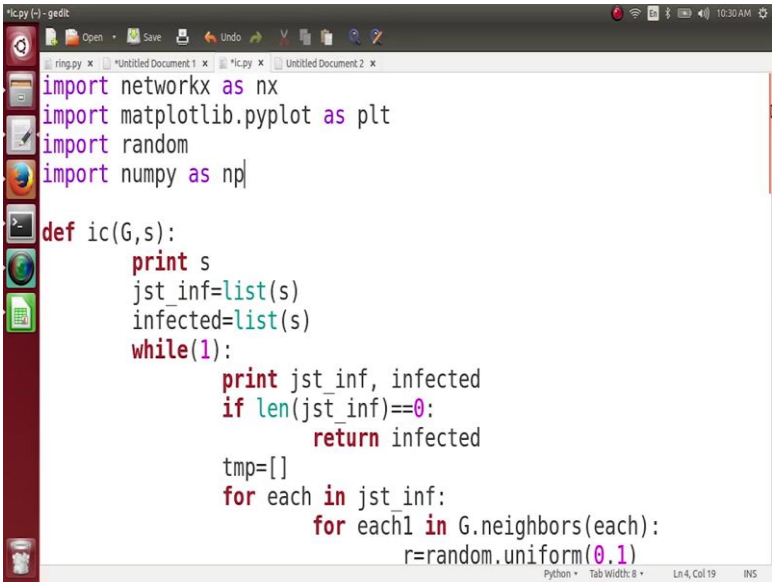
Print let us print degree first and then let us print degree and then let us print closeness and then let us print betweenness, and then coreness, and then cascade right. Let us implement it and see no ok.

(Refer Slide Time: 16:55)



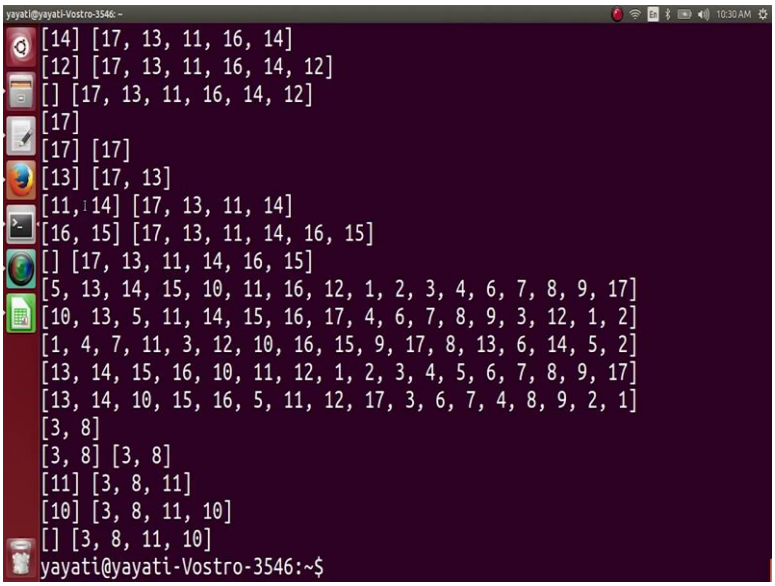
So, we have used this numpy module for finding the average we need to import it.

(Refer Slide Time: 17:05)



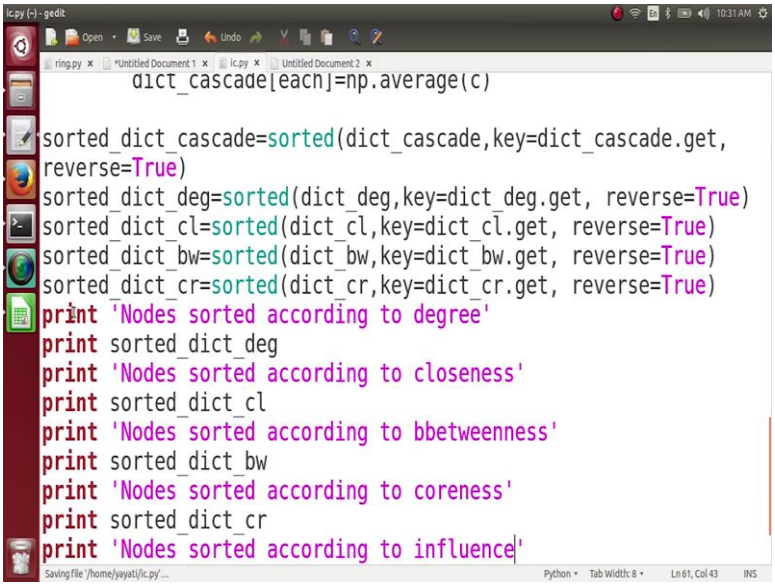
So, import numpy as np and actually we were printing some extra values where we print some extra values not sure, we are actually printing ok.

(Refer Slide Time: 17:31)



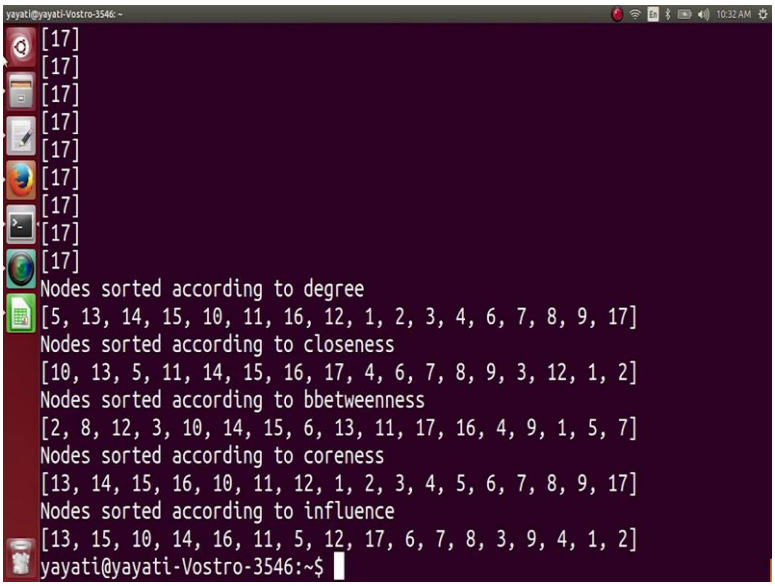
So, we do not to see all these values because, so many times our process have run ok. So, we can actually if we do not need this portion of the core and let us now run it and see. And we do not know whether where one particular list is ending. So, let us say here print, so take degrees.

(Refer Slide Time: 17:59)



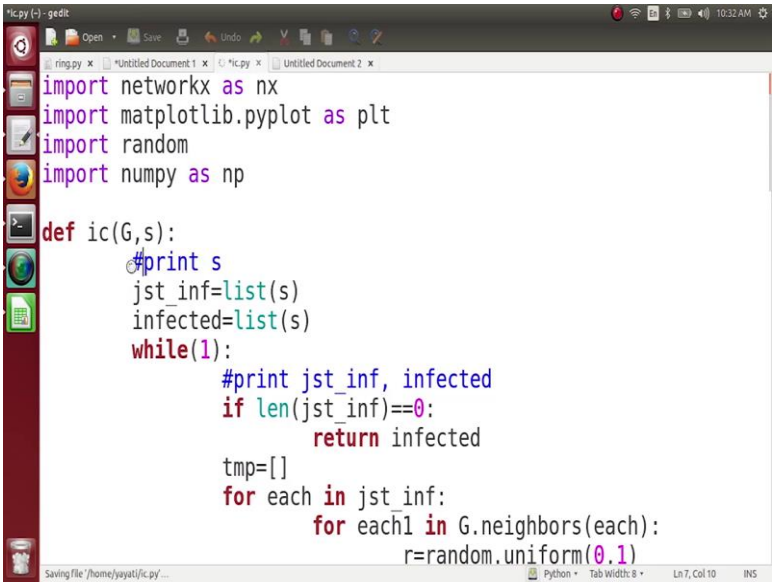
So, these are your nodes sorted according to degree, according to closeness, according to betweenness, according to coreness and according to we can call it nothing but the influence fine.

(Refer Slide Time: 18:45)

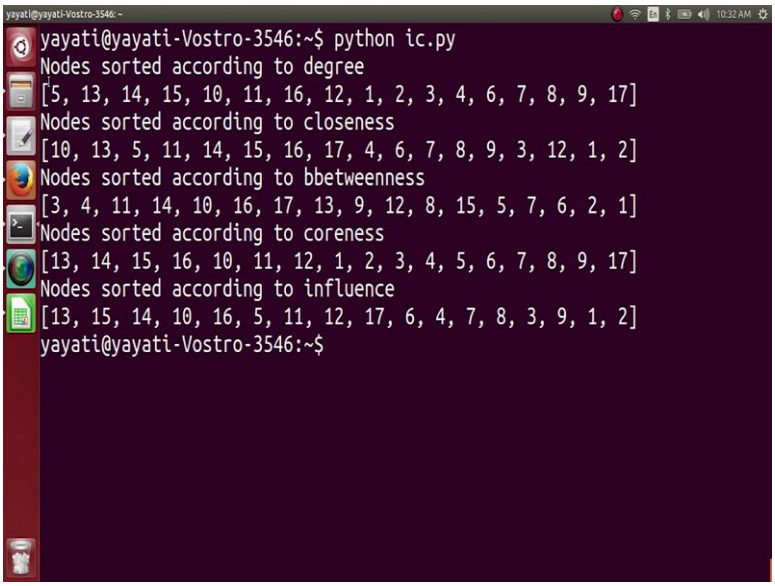


So, we have many extra things printed here which we actually do not want. Somewhere this print statement is coming ok, so it is coming from here we can actually comment it and run it again. Still something is sprinting, s is sprinting here, we comment it also see here.

(Refer Slide Time: 19:07)

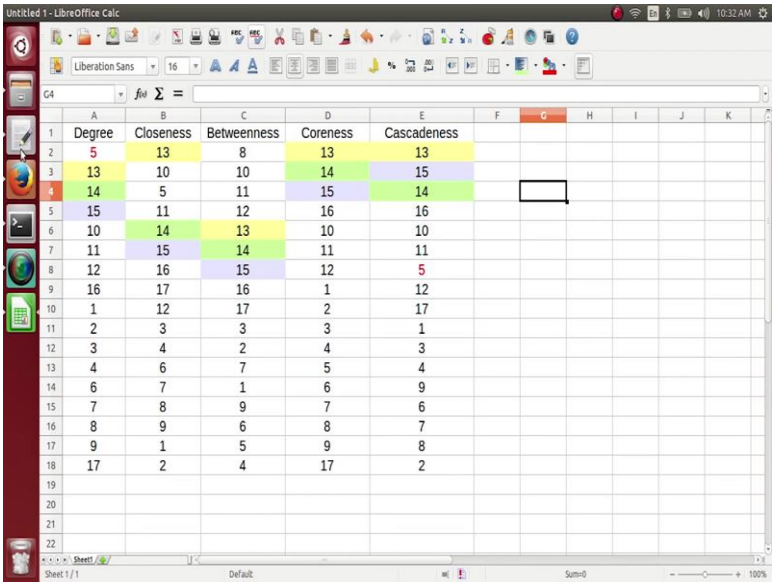


(Refer Slide Time: 19:11)



So, these are your nodes sorted according to your degrees, node sorted according to closeness between this coreness and influence. So, what I have done? I have actually run this process and I have kept these values here. So, these values might not be exactly the same.

(Refer Slide Time: 19:35)



Degree	Closeness	Betweenness	Coreness	Cascadeness
5	13	8	13	13
13	10	10	14	15
14	5	11	15	14
15	11	12	16	16
10	14	13	10	10
11	15	14	11	11
12	16	15	12	5
16	17	16	1	12
1	12	17	2	17
2	3	3	3	1
3	4	2	4	3
4	6	7	5	4
6	7	1	6	9
7	8	9	7	6
8	9	6	8	7
9	1	5	9	8
17	2	4	17	2

So, for degree and closeness these are going to be the exactly the same. So, if you see a 5, 13, 14, 15 it is going to be the same 5, 13, 14, 15. For finding out the betweenness we use a random process, the algorithm uses a random process.

So, the value here might not be very same as what is here and for the coreness it is going to be the same. And for influence it is mostly going to be the same because, we have run this process a 1000 times. So, the results which we are going to get will be the same 13, 14, 15, 10 and you can see 13, 14, 15, 10 and. so on. Ok let us analyze what's happening. So, here I have taken a spreadsheet, here I have put nodes according to a descending order of degree, descending order of closeness and so on.

Now I want to show you here something. Do you see here this cascade ness? It is actually nothing but the influence, so it is the influence here ok. So, if you see here you see the node which is having the highest influence is the one having the highest coreness. And actually the highest closeness it means that maybe closeness and coreness relates to influence, but if you look at the second highest node here which created the second highest cascade it as a high coreness, but it does not have a high closeness.

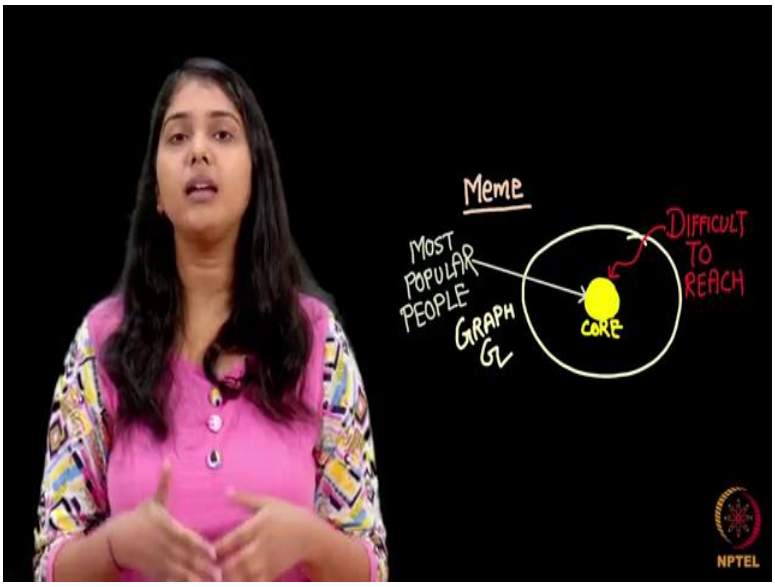
So, overall, you can see that this influence over here is mostly related to the coreness and degree. So, closeness if you see nodes 10, 5 and 11 here this goes quite down here and then this node edge here comes over here. So, the influence is maximally correlated with the coreness and degree here and out of coreness and degree although you can see that it is mostly related to coreness. This node 5 over here has a very high degree, but it has a very less influential power. So, this result it validates it is that: if you want to look at the maximum influential power of a node you should look at the nodes which are having the maximum coreness.

So, I have done this experiment over here for 1 network. You can actually replicate this experiment for various different networks rather you can use some real-world data sets also and verify this.

Social Networks
Prof. S. R. S. Iyengar
Department of Computer Science
Indian Institute of Technology, Ropar

How to go Viral on Web
Lecture - 166
Pseudo core

(Refer Slide Time: 00:09)



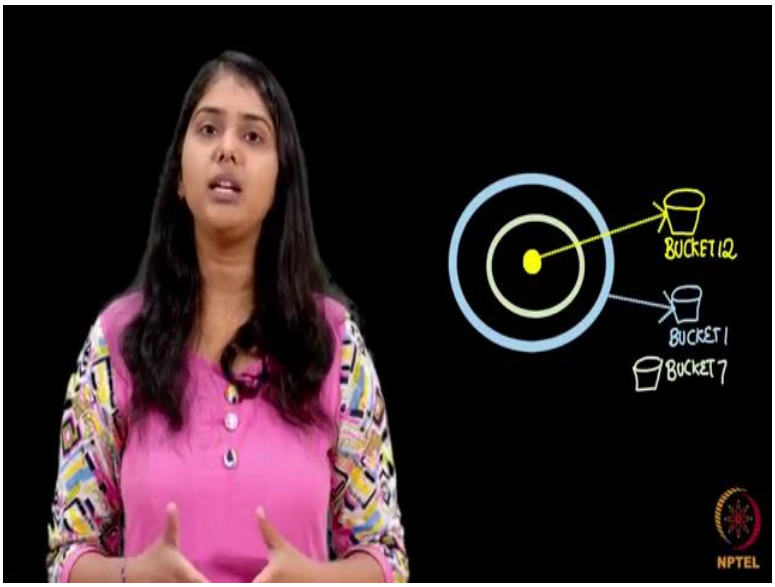
I have this internet mean which I really want to become popular. I look at my network, I identify the core nodes and I try and convince them to adopt my idea. But, is that so easy? So, generally in this core are the most popular people of a network like the public figures, Bollywood celebrities let us say Amitabh Bachchan and Shahrukh Khan kind of people. Can I go to them and really ask them to share my meme further? Why will they listen to me? We know that it is very difficult to reach such people.

So, how do we use this strategy? So, one idea is to obviously, pay them a lot of money like this different advertising companies do. So, if you want to let us say your next soap to become popular, you try to give some money to a say of Bollywood heroine and ask her to act in your shoot and so on. But, can we use a smarter approach? I might not have that much amount of money, how do I meet these core nodes when I do not have enough amount of money to give them, I do not know.

So, reaching these core nodes is actually very difficult and even sometimes with your money they might also not be ready to adopt your idea. That is mostly because these people are inside, they form the crux of this network they are in the innermost part of this network. So, it is really very difficult to infect such people but see what now next we are going to do, we looked at this (Refer Time: 01:46) decomposition algorithm.

So, we have seen that there were multiple buckets. So, we had this bucket number 1, 1 core, bucket number 2, 2 core, bucket number 3, 3 core. What does this bucket actually tell us? These buckets actually tell us the different influence levels in this network.

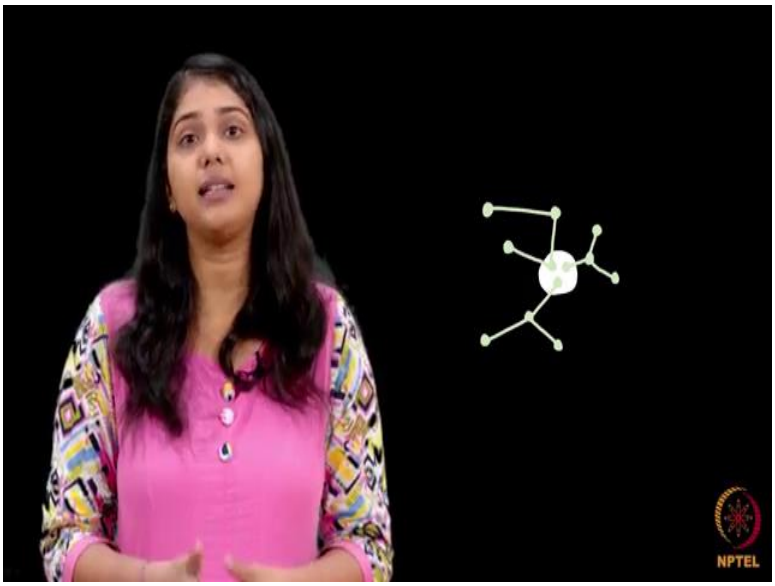
(Refer Slide Time: 02:08)



So, if you look at this bucket 12; let us say that twelve is the last bucket we formed. If you look at this bucket 12, it has the most influential people of the network the innermost core. If you look at bucket 1, it consists of the least influential people in the network. And if you look at let us say bucket 7, then it is somewhere in middle based on its influence.

So, we can actually look at this network in this way where, based on your buckets your network gets divided into multiple shells, outermost shell corresponds to bucket 1, innermost shell corresponds to bucket 12 and then here are our different buckets. So, we have here this network divided into multiple shells. And then we have seen that it is very difficult to convince these people in the innermost core.

(Refer Slide Time: 03:05)



What next we will try to do if what we will do if we will look at these different, different shells and we will examine the influential power of all shells, what do I mean by that?

So, we have done this the done that experiment right where we choose some c nodes which are having the highest degree and then looked at how many people go to infected at the end or we looked at the nodes having highest closeness. And then looked at how many people got infected at the end. We do the same procedure here. We first of all choose nodes from the innermost shell; perform our cascade and look at how many nodes get infected at the end.

(Refer Slide Time: 03:46)



Then we take this second shell here. So, let us say the innermost shell is shell 12, we took shell 12 look at the how many people go to infected towards the end. Then we take the shell number 11, again take some nodes from here infect the network and then look at how many people got infected and we term this just a term.

So, this final number of nodes which got infected here, we call it as the cascade capacity of a shell, of a shell. Cascade capacity means if you start infecting the network from the shell choosing the seed nodes from this shell, what is the final amount of network that you have infected.

And we do it for all the shells. What do you think is the result, should it be if i start the infection from the outermost shell, shell number 1, very less people get infected towards the end, cascade capacity is very less? If I start from this second shell cascade capacity is a little bit more, third shell a little bit more and then it keeps increasing.

We expect the result should be something like that but when you do it, so what we did is we took some we can actually take some real world networks we can take some synthetic networks also, but since we are talking about the internet meme propagation in real world social networks, it is better to take our network which are which correspond to the online social networks.

So, we can take; so all these networks as we have seen before in our chapter data sets are now available online. You can get a graph for Facebook, you can get a graph for google plus, you can get a graph for twitter and almost every other online social network. So, you choose some of these social networks and then applying (Refer Time: 05:41) decomposition is very easy here. And then do this experiment. Take the nodes from different, different shells and look at the final cascade capacity.

And when you plot this graph, plot this graph on the x axis is the shell number, on the y axis is the cascade capacity what you observe? You see that it is not linear like what we expected. We will see what is happening here; It starts increasing starts increasing and add some shell it reaches its maximum and then it becomes constant. What does that mean? That means that to infect this network to make your meme go viral, it is not always necessary for you to go to the innermost core and convince these people.

So, if you look at some of these shells outer from this core shell, some of these they have actually the same cascade capacity as the core nodes. So, instead of these core nodes we have found a lot more people here which are actually we can called pseudo course, we call these people as Pseudo course. So, there is a big chunk of the pseudo core people, whom you can actually infect to make your meme go viral.

THIS BOOK
IS NOT FOR
SALE
NOR COMMERCIAL USE



(044) 2257 5905/08
nptel.ac.in
swayam.gov.in